

CS 185: Deep Reinforcement Learning

by Matthew Cai

Lecture 1

idk didn't take notes on this one

Lecture 2

Markov Property

In RL problems, *states* follow the Markov property, i.e. the future state depends only on the current state, not the previous states that led up to it. In other words, the state is *memoryless*.

Partially observed RL

Actions a_t , states s_t , observations o_t .

$$\begin{aligned}\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t) \\ p(\mathbf{o}_t \mid \mathbf{s}_t) \\ p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)\end{aligned}$$

Fully observed RL

$\mathbf{o}_t = \mathbf{s}_t$, so observations are unnecessary.

$$\begin{aligned}\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t) \\ p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)\end{aligned}$$

Behavioral Cloning

What if we wanted to do supervised learning? When collecting data, we can measure several *demonstration trajectories*

$$\{(\mathbf{o}_1^{(i)}, \mathbf{a}_1^{(i)}, \dots, \mathbf{o}_H^{(i)}, \mathbf{a}_H^{(i)})\}_{i=1}^N$$

Behavioral cloning, a form of *imitation learning*, involves maximizing the log-probability of the policy—this is effectively just supervised learning/maximum likelihood estimation with a neural network.

$$\arg \max_{\theta} \sum_{i=1}^N \sum_{t=1}^H \log \pi_\theta(\mathbf{a}_t^{(i)} \mid \mathbf{o}_t^{(i)})$$

The algorithm itself would take the raw data (e.g. image) and process it with an encoder to produce the *action distribution parameters*. For discrete actions, these parameters are logits that are then processed with a softmax. For continuous actions, these parameters are actual distribution parameters, i.e. they are values like Gaussian means and covariance that parametrize the distributions of each action.

$$\begin{aligned}\text{Output: } \begin{bmatrix} f_1(\mathbf{o}_t) \\ \vdots \\ f_A(\mathbf{o}_t) \end{bmatrix} &\implies p(\mathbf{a}_t = 1 \mid \mathbf{o}_t) = \frac{\exp(f_1(\mathbf{o}_t))}{\sum_{i=1}^A \exp(f_i(\mathbf{o}_t))} \\ \text{Output: } \begin{bmatrix} \mu(\mathbf{o}_t) \\ \Sigma(\mathbf{o}_t) \end{bmatrix} &\implies p(\mathbf{a}_t \mid \mathbf{o}_t) = \mathcal{N}(\mathbf{a} \mid \mu(\mathbf{o}_t), \Sigma(\mathbf{o}_t))\end{aligned}$$

Does Behavioral Cloning Work?

The main issue with behavioral cloning is that, in RL, data is *not i.i.d.*, i.e. earlier actions in the sequence affect later states. However, after a slight divergence from the training trajectory (due to a small error from the model), the error will be *amplified* across the future trajectory; as the model diverges further from the training trajectory, its error subsequently grows. This problem may be addressed in several ways:

- Change the algorithm (Dagger)
- Use better models that make fewer mistakes
- Smarter data collection/augmentation
- Multi-task learning

But first, we formalize the notion of this divergence, which is termed **distributional shift**, i.e. $p_{\text{train}}(\mathbf{x}) \neq p_{\text{test}}(\mathbf{x})$. At some point in testing, $p_{\pi_\theta}(\mathbf{o}_t) \neq p_{\text{data}}(\mathbf{o}_t)$. We'll now try and quantify just how bad this distributional shift can be for behavioral cloning. Note that this theoretical view is *not* necessarily representative of real-world, typical behavior.

Assume the data is produced by a good policy $\pi^*(\mathbf{s}_t)$. (Note that we are using the observed case for simplicity). We define a cost function

$$c(\mathbf{s}_t, \mathbf{a}_t) = \begin{cases} 0, & \mathbf{a}_t = \pi^*(\mathbf{s}_t) \\ 1, & \text{otherwise} \end{cases}$$

The expected value of our cost at time step t w.r.t. actions sampled from π_θ is thus

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} = \sum_{\mathbf{a}_t} \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) c(\mathbf{s}_t, \mathbf{a}_t) = \pi_\theta(\mathbf{a}_t \neq \pi^*(\mathbf{s}_t) | \mathbf{s}_t)$$

In other words, this represents the probability of mistakes. Then, the expected total number of mistakes is

$$\mathbb{E}[M] = \sum_{t=1}^H \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t), \mathbf{s}_t \sim p_{\pi_\theta}(\mathbf{s}_t)} [c(\mathbf{s}_t, \mathbf{a}_t)]$$

Note that $\mathbf{s}_t \sim p_{\pi_\theta}(\mathbf{s}_t)$ represents the distribution of possible states $\mathbf{s}_t$ at time t for the policy π_θ , and it *depends* on past actions.

Let's assume $\pi_\theta(\mathbf{a} \neq \pi^*(\mathbf{s}) | \mathbf{s}) \leq \epsilon, \forall \mathbf{s} \in \mathcal{D}_{\text{train}}$. In other words, the probability of making a mistake is $\leq \epsilon$ for each state in the dataset. Then,

$$\begin{aligned} \mathbb{E}[M] &\leq \epsilon H + (1 - \epsilon)(\epsilon(H - 1) + (1 - \epsilon)(\dots)) \\ &\leq \epsilon H^2 \end{aligned}$$

In other words, the expected number of mistakes increases *quadratically* in the horizon. This assumption, however, does not generalize to most datasets.

Instead, we can make a more realistic assumption, in which, for any state sampled from the training distribution (not just the states observed in the training dataset), we are unlikely to make a mistake, i.e.

$$\mathbb{E}_{\mathbf{s}_t \sim p_{\text{train}}(\mathbf{s}_t)} [\pi_\theta(\mathbf{a}_t \neq \pi^*(\mathbf{s}_t) | \mathbf{s}_t)] \leq \epsilon$$

Then, we can note the probability of some state $\mathbf{s}_t$ as

$$p_{\pi_\theta}(\mathbf{s}_t) = (1 - \epsilon)^t p_{\text{train}}(\mathbf{s}_t) + (1 - (1 - \epsilon)^t) p_{\text{mistake}}(\mathbf{s}_t)$$

We may subsequently calculate **total variation divergence (TVD)**, a measure of distance between two distributions similar to *KL divergence*

$$D_{\text{TV}}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)| \leq 1$$

Through some simplifications, we note that

$$\begin{aligned} D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) &= \frac{1}{2} \sum_{\mathbf{s}_t} |p_{\text{train}}(\mathbf{s}_t) - p_{\pi_\theta}(\mathbf{s}_t)| \\ &= \frac{1}{2} \sum_{\mathbf{s}_t} |p_{\text{train}}(\mathbf{s}_t) - (1 - \epsilon)^t p_{\text{train}}(\mathbf{s}_t) + (1 - (1 - \epsilon)^t) p_{\text{mistake}}(\mathbf{s}_t)| \\ &= (1 - (1 - \epsilon)^t) \frac{1}{2} \sum_{\mathbf{s}_t} |p_{\text{train}}(\mathbf{s}_t) - p_{\text{mistake}}(\mathbf{s}_t)| \end{aligned}$$

From the upper-bound on $\text{TVD} \leq 1$ and the identity $(1 - \epsilon)^t \geq 1 - \epsilon t$ for $\epsilon \in [0, 1]$, we find

$$\begin{aligned} D_{\text{TV}}(p_{\text{train}}, p_{\pi_\theta}) &\leq (1 - (1 - \epsilon)^t) \\ &\leq \epsilon t \end{aligned}$$

In other words, the distance between probability distributions increases linearly in t . This bound allows us to derive, after several simplifications, that

$$E[M] \leq \sum_{t=1}^H \epsilon + 2\epsilon t$$

or that $E[M] \in O(\epsilon H^2)$. Thus, the error increases *quadratically* with the horizon.

□

pessimistic

Solution 1: DAgger: Dataset Aggregation

We'd like to make $p_{\text{data}}(\mathbf{o}_t) \approx p_{\pi_\theta}(\mathbf{o}_t)$, i.e. reduce distributional shift. DAgger focuses on being smart about $p_{\text{data}}(\mathbf{o}_t)$ for this purpose—its goal is to effectively collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$!

1. Train $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ from human data $\mathcal{D}$.
2. Run $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ to produce the model dataset $\mathcal{D}_\pi$.
3. Ask a human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$.
4. Aggregate $D \leftarrow D \cup \mathcal{D}_\pi$.
5. Repeat.

This iteratively aligns the training dataset to further match the states/observations that the policy π_θ experiences.

The main difficulty with DAgger, however, is the production of labels by humans. This is not only expensive, but humans may not even be able to classify each observation, depending on the task.

A common variant of DAgger aimed to fix this issue is to replace human labeling with human intervention.

1. Train $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$ from human data $\mathcal{D}$.
2. Run $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$.
3. Ask a human to *intervene* or take over at some time step t .
4. Add all $(\mathbf{o}_t, \mathbf{u}_t)$ examples to the human data $\mathcal{D}$.
5. Repeat.

Lecture 3

Solution 2: Better Models, Less Mistakes

Why might a model fail to fit to the expert (training data)? The human expert may exhibit...

- Non-Markovian behavior
- Multimodal behavior

Non-Markovian Behavior

Instead of $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t)$, a human's policy resembles something more like $\pi_\theta(\mathbf{a}_t \mid \mathbf{o}_1, \dots, \mathbf{o}_t)$, in which the current behavior depends on all past observations, not just the current observation. This sort of knowledge can be encoded by using some sort of sequence model, e.g. transformer, to process each observation "frame" in history.



Info

If there is a bijection between observations and states (i.e. $\mathbf{s}_t = \mathbf{o}_t$), then there exists a Markovian policy (i.e. doesn't use history) that is optimal. If the observations are not sufficient to infer the full state, then the optimal policy may require knowledge of history.

However, including history can *degrade* model performance. Why?

- Access to more (unnecessary) information can cause mistakes.
- It may induce overfitting as the model must naturally be larger and have more capacity.
- It can increase the chance of *distributional shift*, as even one small deviation remains in the history forever, and therefore contributes to the shift from then on.

Causal confusion is a particularly notable consequence of an excess of information. In essence, a model's action A causes an event B , which results in A and B showing up together very frequently. As the model makes *associations*, not causal relationships, the occurrence of B can actually influence the model into doing A . This phenomenon is known as **spurious correlation**.



Brake Indicator

When the car brake is depressed (A), the brake indicator lights up (B). As the brake remains depressed over the next several time steps, so too will the brake indicator remain lit up. Then, during inference, if the model sees the brake indicator light up, it may likely choose to depress the brake because it has associated action A with event B .



Dagger?

Dagger, surprisingly, is effective for fixing causal confusion for the simple reason that Dagger addresses distributional shift.

Multimodal Behavior

Multimodal behavior describes environments where there exist states for which there are *multiple reasonable actions*. This becomes an issue with *continuous actions*, in which a unimodal distribution (e.g. Gaussian) would essentially *average* the reasonable actions together to produce an action that may not be reasonable at all! There are a couple solutions to this.

- Discretize the continuous action space → softmax correctly identifies all reasonable actions
- Use more expressive continuous distributions → multimodal distributions that capture all reasonable actions

Discretization

Can we always just naively discretize continuous action spaces? No, because the number of discrete "bins" is exponential in the dimensionality of the action space. For low-dimensional action spaces, however, this is a great solution.

One may suggest discretizing one dimension at a time to avoid the exponential explosion; however, this is not necessarily effective because it suggests choosing along each dimensions independently when, in fact, the dimensions may covary with each other.

Example

For instance, consider a highway with a fast lane on the right and a slow lane on the left. One dimension of the action space is {fast, slow}, and another dimensions of the action space is {right, left}. However, the two dimensions are not independent—the only good actions here are (fast, right) and (slow, left).

Autoregressive discretization provides a computationally efficient (linear, not exponential) solution for high-dimensional action spaces. In essence, this discretizes one dimension at a time, but *in sequence*, rather than independently. In essence, the sequence model (e.g. transformer), when predicting dimension i of the action space, is fed the original input (state/observations) and the *previously predicted dimensions*, i.e. dimensions $0, \dots, i - 1$. In short, the dimensions are discretized one a time and then decoded autoregressively (hence the nomenclature).

This is valid by the **Chain Rule of Probability**. If we denote dimension i of action $\mathbf{a}_t$ as $a_{t,i}$, then

$$p(\mathbf{a}_t \mid \mathbf{s}_t) = p(a_{t,0}, \dots, a_{t,n} \mid \mathbf{s}_t) = \prod_{i=0}^n p(a_{t,i} \mid \mathbf{s}_t, a_{t,0}, \dots, a_{t,i-1})$$

In other words, this is equivalent to predicting all dimensions of the action at once based on the state while still providing an efficient method of discretization.

Expressive Continuous Distributions

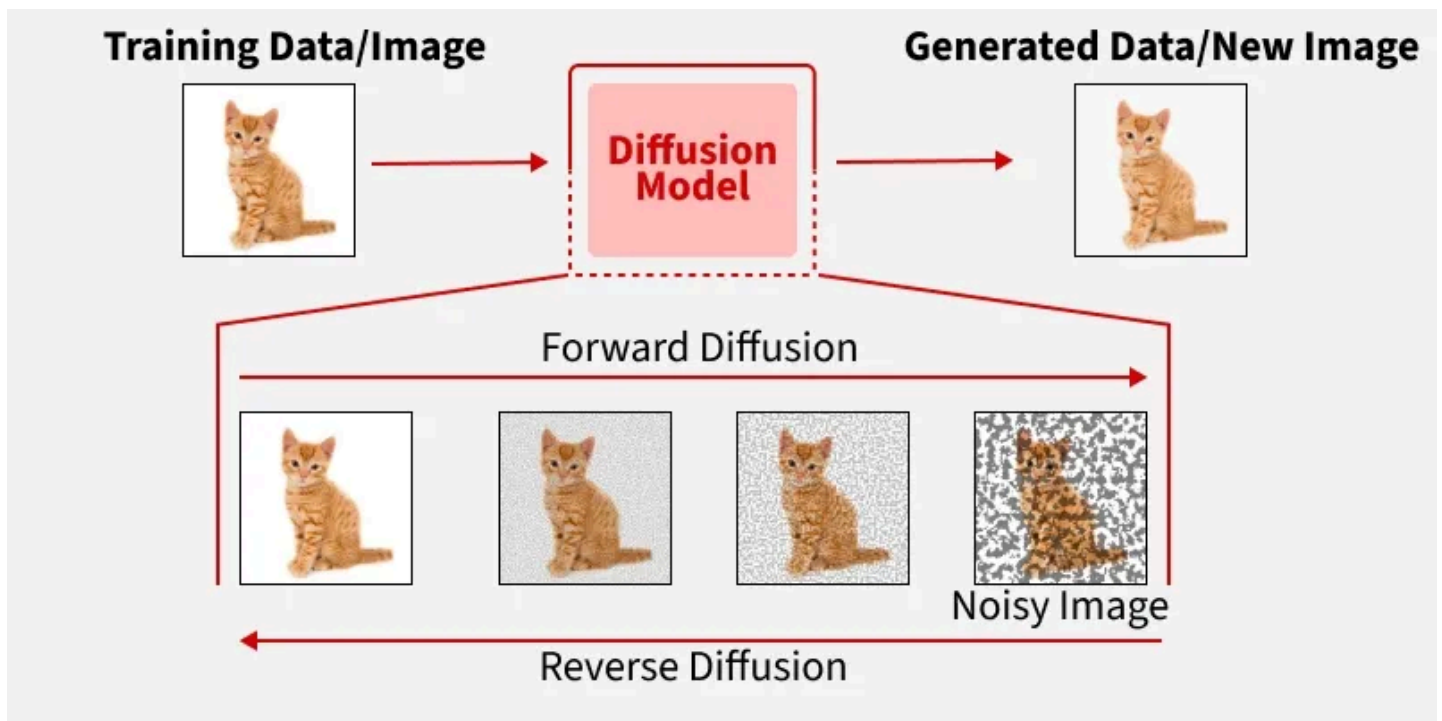
Generally speaking, it's hard to model multimodal distributions. Instead, we can include an additional input that "decides" the mode that the model chooses; this additional input can simply be some *random noise*.

The main challenge with this method, however, is that we must train the model to actually use the random noise to decide the mode—otherwise, of course, the noise is useless. There are several different solutions for this.

- Variational autoencoders
- Normalizing flows
- Diffusion/flow matching

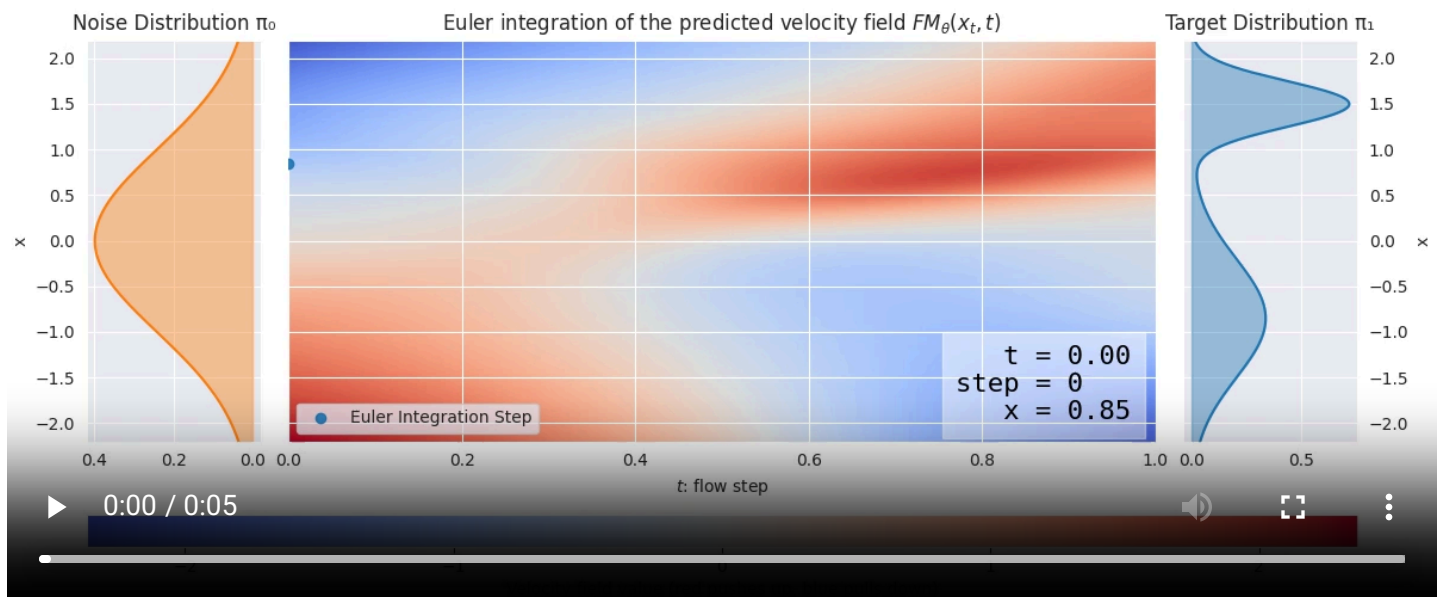
Diffusion/Flow Matching

Diffusion is the most popular approach for generating high dimensional, continuous data. The key principle underlying diffusion is that turning random noise into a specific image (e.g. image of a dog) is hard, but turning an image of a dog into random noise is easy (just add more noise). So, to train a model to generate images of dogs, we take real images of dogs, iteratively add noise, and then train the model to simply go the other way!



Source: GeeksforGeeks

Flow matching is the modern version of diffusion, with essentially the same intuition, except it's easier to implement. In essence, we want to train a model to start with a noise distribution, e.g. Gaussian, and transform samples from it into a desired data distribution. It relies on the same idea of starting with real data samples and adding noise, and simply training the model to reverse this process. Formally, flow matching learns a vector field $v(\mathbf{x}_t, t)$ that allows "sampling from" (modeling) $p(\mathbf{x})$ by sampling $\mathbf{x}_0 \sim p_0(\mathbf{x}_0)$ (noise distribution) and integrating the vector field to produce $\mathbf{x}_1 = \mathbf{x}_0 + \int_0^1 v(\mathbf{x}_t, t) dt$. In practice, this integration is discretized as **Euler integration**. Below is a demonstration of flow matching in action.

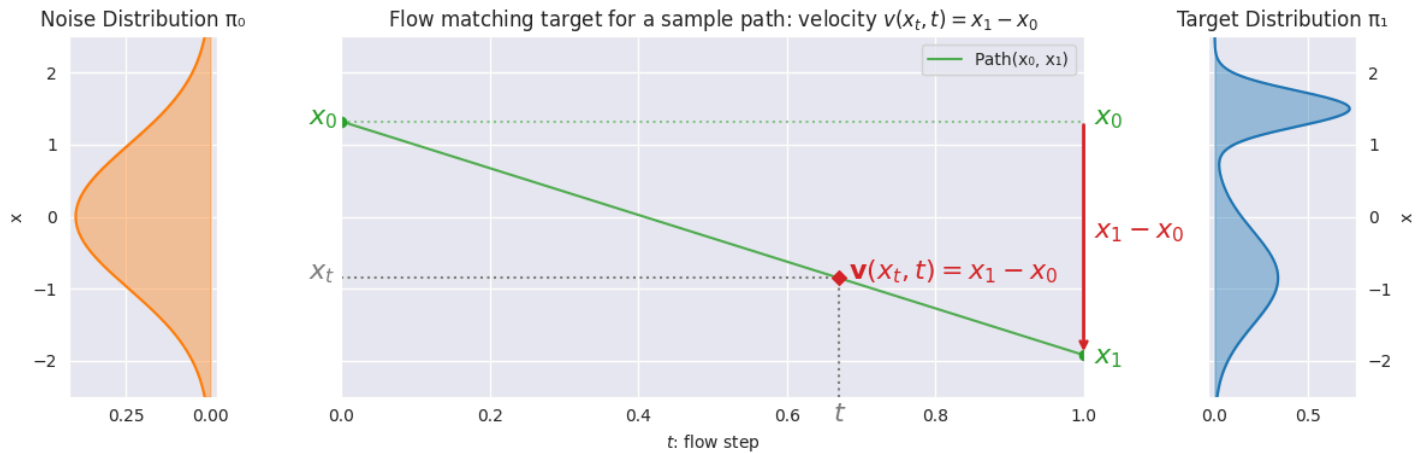


Source

But how do we train flow matching?

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$ (i.e. Gaussian noise distribution)
2. Sample $\mathbf{x}_1 \in \mathcal{D}$ where $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ (data distribution)

3. Sample $t \sim p(t)$ where $p(t)$ is defined only over $[0, 1]$ (e.g. $p(t) = \mathcal{U}(0, 1)$), as this is the selection of the "time step" t of the vector field in-between $t = 0$, the noise distribution, and $t = 1$, the data distribution. See the below diagram for clarification.
4. Compute $\mathbf{x}_t = t\mathbf{x}_1 + (1 - t)\mathbf{x}_0$ (i.e. just draw a line)
5. Update v with $\nabla \|v(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2$, or update the velocity (that point in the vector field) to point more towards $\mathbf{x}_1$ (i.e. intended velocity is the slope of the line $(\mathbf{x}_1 - \mathbf{x}_0)$)



The point of this with regards to fixing the averaging problem is that the vector field produced by flow matching will *eventually diverge* to produce one of the reasonable actions. In earlier "time" steps (t closer to 0), the vector field appears more averaged across reasonable actions. When t is closer to 1, the vector field starts to diverge. In essence, the supervision of many linear vector fields produces an overall vector field that effectively maps the noise distribution to the target distribution, effectively representing multimodality.

In RL, flow matching is applied as follows.

1. Construct minibatch. For each element in batch j ,
 1. Sample $(\mathbf{o}_t^{(j)}, \mathbf{a}_t^{(j)})$ from dataset (data distribution)
 2. Sample $\mathbf{a}_{t,0}^{(j)} \sim \mathcal{N}(0, \mathbf{I})$ (noise distribution)
 3. Sample $\tau^{(j)} \sim p(\tau)$ (time step)
 4. Compute $\mathbf{a}_{t,\tau}^{(j)} = \tau^{(j)}\mathbf{a}_t^{(j)} + (1 - \tau^{(j)})\mathbf{a}_{t,0}^{(j)}$.
2. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}$, where $\mathcal{L} = \sum_{j=1}^B \|v_{\theta}(\mathbf{o}_t^{(j)}, \mathbf{a}_{t,\tau}^{(j)}, \tau^{(j)}) - (\mathbf{a}_t^{(j)} - \mathbf{a}_{t,0}^{(j)})\|^2$.

That is, when applying flow matching to RL, the model is predicting the *vector field/velocity* itself. In particular, the model is represented by $v_{\theta}(\mathbf{o}_t^{(j)}, \mathbf{a}_{t,\tau}^{(j)}, \tau^{(j)})$.

Action Chunking

Action chunking asks a model to predict a **chunk** or sequence of the next K actions, after which the actor will execute the first k actions, where $1 < k \leq K$. It's unknown exactly why or when this improves performance for imitation learning, but it frequently does!

? Why only execute k actions?

There is no definitive reason, but it seems that it provides additional training signal for the model.

Solution 3: Narrow vs Broad Data

In practice, how you collect/augment your datasets has a substantial impact on model performance.

Mistakes and Corrections

One common method is to intentionally add mistakes (and their corresponding corrections) to the dataset. This ensures the dataset isn't "too good," and that the model can learn to recover from the mistakes, and the idea is that the inclusion of corrections helps more than the mistakes hurt the model. This addition of data may even be synthetic.

Pre-Training

Essentially, the motivation is the same as the mistake-augmentation; we want to show the model bad situations and how to recover from them, but not to enter those situations. So, we run two steps of training.

1. Pre-training phase: train the model on a very large, but low quality dataset that may include many mistakes.
2. Post-training phase: train the model on a smaller, but curated/high quality dataset that only includes good examples. (This is known as **fine-tuning**).

Solution 4: Multi-task Learning

Consider training a car to drive to a single point $\mathbf{p}_1$, with policy $\pi_\theta(\mathbf{a} \mid \mathbf{s})$. An example of a multi-task version of this problem would be training a car to drive to any of a set of points $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$, with policy $\pi_\theta(\mathbf{a} \mid \mathbf{s}, \mathbf{p})$, i.e. conditioned on some choice of $\mathbf{p} \in \{\mathbf{p}_1, \dots, \mathbf{p}_n\}$.

The most obvious benefit of this is that this allows a much *larger* and *varied* corpus of training data, and this is more formally known as **goal-conditioned behavioral cloning**, in which the model is trained on a dataset of several tasks, but it takes in, as input, the desired goal—in this case, which point $\mathbf{p}$ the car should drive to.

Lecture 4: Reinforcement Learning Basics

The Markov Decision Process

The **Markov Decision Process (MDP)** is a way of describing reinforcement learning problems.

- Markov: for the Markov property
- Decision: because the model makes decisions

Info

For now, we will assume $\mathbf{o}_t = \mathbf{s}_t$, i.e. our policy is $\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$.

The critical difference between imitation and reinforcement learning is the lack of demonstration training examples for the model to learn from. Instead, in an MDP, we're provided a **reward function** $r(\mathbf{s}_t, \mathbf{a}_t)$, which represents the (state, action) pairs that are preferred. In fact, an MDP may be defined wholly by the following parameters.

- States $\mathbf{s}_t$.
- Actions $\mathbf{a}_t$.
- Rewards $r(\mathbf{s}_t, \mathbf{a}_t)$.
- Probabilities $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$.

Markov Chains

Markov chains originated as a representation for *dynamical states* that satisfied the Markov property, i.e. $\mathbf{s}_{t+1} \perp \mathbf{s}_{t-1} | \mathbf{s}_t$. It may be represented by $\mathcal{M} = \{\mathcal{S}, \mathcal{T}\}$, where $\mathcal{S}$ is the *state space* and $\mathcal{T}$ is the *transition operator*, which describes the linear operator applied to some vector μ_t , for which entry i describes the probability of being in the i th state at time t , to produce μ_{t+1} , i.e. $\mu_{t+1} = \mathcal{T}\mu_t$. Note that $\mathcal{T}_{i,j} = p(\mathbf{s}_{t+1} = i | \mathbf{s}_t = j)$. Markov chains, of course, inspired MDPs.

More formally, we extend Markov chains to produce MDPs characterized by $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$, where

- States $\mathbf{s} \in \mathcal{S}$.
- Actions $\mathbf{a} \in \mathcal{A}$.
- Transition operator $\mathcal{T}$.
- Reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.

MDP $\rightarrow$ Markov Chain?

Given a policy $\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$, it is possible to turn an MDP into a Markov chain. It suffices to define the states of the Markov chain to be the set of all $(\mathbf{s}, \mathbf{a}) \in \mathcal{S} \times \mathcal{A}$ pairs and define the transition operator using the policy, i.e. $p(\mathbf{s}_{t+1}, \mathbf{a}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) = p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)\pi_\theta(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})$.

We note that **partially observed** MDPs, i.e. $\mathbf{o}_t \neq \mathbf{s}_t$, we must extend this definition slightly, i.e. $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$, where

- States $\mathbf{s} \in \mathcal{S}$.
- Actions $\mathbf{a} \in \mathcal{A}$.
- Observations $\mathbf{o} \in \mathcal{O}$
- Transition operator $\mathcal{T}$.
- Emission operator $\mathcal{E}$.
- Reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.

This can also be written as a Markov chain; for now, though, we will continue with our assumption that $\mathbf{o}_t = \mathbf{s}_t$, i.e. **fully observed** MDPs.

Some Useful Concepts

For a **finite horizon** RL problem with policy π_θ , we can write that the probability of any sequence of states and actions is

$$p_\theta(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

by the chain rule of probability. $p_\theta(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H)$ is known as the **trajectory distribution**. Another notable distribution is the **state marginal** $p_\theta(\mathbf{s}_t)$, which describes the probability of being in state $\mathbf{s}_t$ at time step t for some policy π_θ . Critically, both probabilities, while having very long, complex formulas, are easy to compute; the trajectory distribution is derived by simply simulating the MDP (just run the Markov chain), while the state marginal is computed by just simulating the policy in the real world.

The Objective of RL

Simple—choose a policy that maximizes total expected reward over the entire horizon.

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

where τ is the sequence/trajectory of states and actions and $p_\theta(\tau) = p_\theta(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H)$, the trajectory distribution.

Viewing it as an MDP in Markov chain form, we can derive that the total expected reward is

$$\mathbb{E} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] = \left[\sum_{t=1}^H \mathcal{T}_\theta^{t-1} \mu_1 \right]^T \vec{r}$$

where $\vec{r}$ is the vector of rewards for $r(\mathbf{s}, \mathbf{a})$, $\forall(\mathbf{s}, \mathbf{a}) \in \mathcal{S} \times \mathcal{A}$, μ_1 is the first state, and $\mathcal{T}_\theta$ is the transition probabilities according to the policy π_θ . In practice, of course, this is never computed this way, but it does give a nice, concise representation of the total expected reward.

But, what if the horizon H is **infinite**? Typically, the total expected reward is represented as $\frac{1}{H} \mathbb{E}[\dots]$ so that it does not approach infinity as well. Using this expression does, in fact, behave well. In particular, provided the underlying Markov chain is

- **Ergodic**: $\forall(\mathbf{s}_1, \mathbf{s}_2) \in \mathcal{S} \times \mathcal{S}$, it's possible to reach $\mathbf{s}_2$ from $\mathbf{s}_1$.
- **Aperiodic**: the system μ does not return to a state periodically for some fixed interval, i.e. we do not have $\mu_t = \mu_{t+k} = \mu_{t+2k} = \dots$ for some k .

the terms from the infinite sum, past some sufficiently large threshold T , are selected from a **stationary distribution**. This is important because a stationary distribution has the property that

$$\bar{\mu} = \mathcal{T}_\theta \bar{\mu}$$

which allows us to solve for $\bar{\mu}$, as it implies

$$(\mathcal{T}_\theta - \mathbf{I})\bar{\mu} = 0$$

or that $\bar{\mu}$ is an eigenvector of $\mathcal{T}_\theta$ with eigenvalue 1.

Expectations and Stochasticity

One critical underlying principle of RL is that we care about *expectations on probabilities*. Even with a discontinuous problem description (e.g. $r(\text{go left}) = 1$ while $r(\text{go right}) = -1$), the expectation $\mathbb{E}_{\pi_\theta}[r(\mathbf{x})]$ remains differentiable in θ because it is being differentiated *w.r.t policy parameters* (which induces probabilities), not states. This allows us to use the same gradient-based optimization techniques widely popularized in machine learning.

Anatomy of RL Algorithms

The simple cycle is

1. Generate samples (i.e. run policy)
2. Fit a model/estimate return
3. Improve policy

and repeat.

Notably, the relative cost and complexity of these steps differ between algorithms.

1. Sample generation is cheap for problems that can be simulated, but expensive for those that must be run in real-time
2. Fast for simple gradients, expensive for large model-based methods
3. Again, fast for simple gradients, expensive for large model-based methods with backpropagation

Value Functions and Q-Functions

To calculate total expected reward for each (state, action) pair, we can define a function $Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ that is effectively equivalent to the reward $r(\mathbf{s}_t, \mathbf{a}_t)$ plus the total expected reward given that the t th (state, action) pair is $(\mathbf{s}_t, \mathbf{a}_t)$. This should be intuitive, hopefully, and helps us quantify this notion of considering *future rewards* in maximizing total expected reward into one function, the *Q-function*. Formally,

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta}[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t]$$

A *value function* is a related concept that is based only on the state, i.e. $V^\pi(\mathbf{s}_t) = \sum_{t'=t}^T \mathbb{E}_{\pi_\theta}[r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t]$ represents the total expected reward given that the t th state is $\mathbf{s}_t$. Notably, $\mathbb{E}_{\mathbf{s}_1 \sim p(\mathbf{s}_1)}[V^\pi(\mathbf{s}_1)]$ is the RL objective! Also, note that

$$V^\pi(\mathbf{s}) = \mathbb{E}[Q^\pi(\mathbf{s}, \mathbf{a})]$$

So, how can we use *Q-functions* and *value functions*? Here are some important ideas.

1. If $Q^\pi(\mathbf{s}, \mathbf{a})$ is known, we can simply set $\pi'(\mathbf{a} \mid \mathbf{s}) = 1$ for $\mathbf{a} = \arg \max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a})$.
2. Although, we can compute a *policy gradient*: if $Q^\pi(\mathbf{s}, \mathbf{a}) > V^\pi(\mathbf{s})$, then $\mathbf{a}$ is a better-than-average action, so modifying $\pi(\mathbf{a} \mid \mathbf{s})$ to improve the probability of selecting $\mathbf{a}$ should improve the policy.

Types of RL Algorithms

- **Policy gradient** methods compute the gradient of the objective with respect to the parameters. They tend to be simple, but require extremely large datasets.

- **Value-based** methods estimate the value function or Q -function of the optimal policy. (No explicit policy).
- **Actor-critic** methods estimate the value function or Q -function of the current policy to improve/train the policy. (They train a *critic* policy to improve/train the *actor* policy). They tend to be the most efficient methods.
- **Model-based** methods estimate the transition model, and are quite varied and often domain-specific.

Model-based RL

The model typically learns $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$, and has several options for how to use this to improve the policy:

- Use the model to plan (basically backprop for trajectory optimization/related algorithms).
- Backpropagate gradients into the policy.
- Learn a value function (e.g. dynamic programming).

Value-based RL

The model learns $V(\mathbf{s})$ or $Q(\mathbf{s}, \mathbf{a})$ and then sets the policy based on its estimate, i.e. $\pi(\mathbf{s}) = \arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$.

Policy gradient RL

The model evaluates the policy by summing the rewards, and then improves by computing the gradient of the objective.

Actor-Credit

Combination of value-based + policy gradient methods.

Why so many different RL algorithms?

There have tradeoffs.

- Sample efficiency
- Stability and ease of use

They make different assumptions.

- Stochastic or deterministic systems
- Continuous or discrete problem
- Episodic or infinite horizon

Different things are easy in different settings.

- Easier to represent the policy?
- Easier to represent the model?

Sample Efficiency

The most important consideration is if the algorithm is **off policy**. An off policy algorithm (e.g. model-based) is able to improve the policy *without* generating new samples from the policy, while an on policy (e.g. policy gradient) algorithm must on policy generate new samples. Typically, the tradeoff is compute cost vs sample generation cost.

Stability and Ease of Use

To consider stability/ease of use, you should ask

- Does it converge?
- If so, what does it converge to?
- Does it always converge?

Why does this matter? Because, unlike supervised learning, RL is often not gradient-descent based, and thus does not come with the same guarantees. For instance,

- Value-based algorithms frequently have no guarantees of convergence.
- Model-based algorithms will converge, but won't necessarily converge to a good policy.
- Policy gradient algorithms do converge to good policies because they compute gradients directly on the objective function. (Gradient ascent).

Lecture 5: Policy Gradients

REINFORCE: Basic Policy Gradients

Recall the objective of RL

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

with sequences τ sampled from the trajectory distribution

$$p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t) p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$$

With policy gradient methods, we evaluate the policy by computing an estimate of our average reward, i.e.

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

where $J(\theta)$ is the value being maximized in the RL objective,

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

and improve it by computing a gradient on the parameters

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

One might think that we're done with just this—we can go train a neural network now and compute gradients $\nabla_{\theta} J(\theta)$ to learn the correct policy. However, whatever machine learning library we're using doesn't know about the *external environment* that produces the feedback reward according to our policy's decisions. Therefore, we'll just receive a gradient of 0, since our program doesn't even know the variables are influenced by the policy π_{θ} ; this is evidenced most clearly by the fact that our estimate of $J(\theta)$, i.e. $\frac{1}{N} \sum_i \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$, doesn't even include θ in the expression!

Direct Policy Differentiation

So here's how **direct policy differentiation** is done to optimize with respect to θ instead of the usual methods.

Let $r(\tau) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)$. Then, we can write that

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)] = \int p_{\theta}(\tau) r(\tau) \, d\tau$$

Deriving with respect to θ produces

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) r(\tau) \, d\tau$$

where the derivative operator may be moved inside the integral due to linearity of expectation. Now, note the following identity.

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \nabla_{\theta} p_{\theta}(\tau)$$

Which allows us to substitute

$$\nabla_{\theta} J(\theta) = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) r(\tau) d\tau$$

This is important because it allows us to rewrite this as an expectation!

$$\nabla_{\theta} J(\theta) = \int (p_{\theta}(\tau)) (\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)) d\tau = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

This conclusion is sometimes called the **policy gradient theorem**, and is very important—make sure you understand this!!

There's still a little bit of work to be done to finish this off, though. It's not immediately obvious how to compute $\log p_{\theta}(\tau)$. First, note that

$$p_{\theta}(\tau) = p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

Thus,

$$\begin{aligned} \log p_{\theta}(\tau) &= \log p(\mathbf{s}_1) + \sum_{t=1}^H [\log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)] \\ \nabla_{\theta} \log p_{\theta}(\tau) &= \cancel{\log p(\mathbf{s}_1)} + \sum_{t=1}^H [\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \log \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}] \\ &= \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \\ \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p(\tau)} \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \end{aligned}$$

Consequently, to evaluate the gradient, we can ***directly** compute it from the policy evaluations*!

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) \right]$$

This algorithm, in fact, is known as **REINFORCE**:

1. Sample τ (run policy)
2. Perform direct policy differentiation
3. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Maximum Likelihood Estimation

We can compare this policy gradient with the gradient for imitation learning computed by maximum likelihood estimation.

$$\begin{aligned} \text{policy gradient:} \quad \nabla_{\theta} J(\theta) &\approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right) \right] \\ \text{maximum likelihood:} \quad \nabla_{\theta} J_{\text{ML}}(\theta) &\approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) \right) \end{aligned}$$

So, the policy gradient is identical except for the *reward term*; thus, the policy gradient doesn't just try to imitate the sample trajectories, but it also favors positive rewards and penalizes negative rewards. Essentially, you can think of policy gradient as *formalizing trial and error*!

❓ Partial Observability?

Does policy gradient still function for RL problems with partial observability, i.e. $\mathbf{o}_t \neq \mathbf{s}_t$? The answer is yes—it functions pretty much identically, just with $\log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{o}_t^{(i)})$ instead, because the Markov property was never used in the derivation.

Limitations

Policy gradient methods, however, have some limitations. For example, consider applying REINFORCE to an RL problem where a model plays from a preset chess position. The reward is +1 for winning and −1 for losing.

Naturally, the goal is for the model to favor good moves and penalize bad moves; for this to happen, we'd like $\nabla_{\theta} J(\theta)$ to produce positive multipliers for good moves and negative multipliers for bad moves. But this is not necessary the case, since other factors influence the model's winning chances!

- The quality of the starting position
- Incorrect multipliers when a good move is made, only to be followed later by a bad move (or vice versa)

Now, while these issues do average out, it takes *many, many samples*! The key problem is that this policy gradient algorithm has *high variance*. How can we reduce it?

Variance Reduction

Baselines

We can essentially *demean* the trajectories' rewards by computing

$$b = \frac{1}{N} \sum_{i=1}^N r(\tau)$$

and modifying $\nabla_{\theta} J(\theta)$ to

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau) [r(\tau) - b] = \frac{1}{N} \sum_{i=1}^N [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau) - \nabla_{\theta} \log p_{\theta}(\tau) b]$$

Notably, the estimator of $\nabla_{\theta} J(\theta)$ remains *unbiased in expectation*!

$$\begin{aligned}
\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau) b] &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b \, d\tau \\
&= \int \nabla_{\theta} p_{\theta}(\tau) b \, d\tau & (1) \\
&= b \nabla_{\theta} \int p_{\theta}(\tau) \, d\tau & (2) \\
&= b \nabla_{\theta} (1) & (3) \\
&= 0
\end{aligned}$$

Where (1) derives from the identity we showed earlier, (2) derives from linearity of expectation, and (3) derives from the properties of a probability distribution, i.e. $\int p(x) \, dx = 1$ for any probability distribution p over x .

However, for some values of b , it will *reduce variance*! Letting b be the average reward (i.e. defined above) does reduce variance, and generally performs well (though there exists better, but the optimal baseline is not used in practice due to its complexity).

The intuition here is that, even if all the trajectories' rewards are positive, we still induce the model to more strongly favor the higher rewards, rather than favor all positive rewards.

Causality

Causality means that *a policy at time t' cannot affect reward at time t when $t < t'$* . At the moment, our gradient calculation does not consider causality.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - b_t \right) \right]$$

The gradient $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})$ is affected by rewards across *all* time steps. Instead, by including causality, we produce

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) - b_{t'} \right)$$

So that the gradient at time step t is only influenced by rewards from time steps $t' \in [t, H]$.

Note that this reduces variance for a rather trivial reason actually—we're multiplying $\nabla_{\theta} \log \pi_{\theta}$ by smaller numbers. Regardless, this modification is typically effective.

Practical Implementation

Automatic Differentiation

We can compute the policy gradients $\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})$ with *automatic differentiation*, actually. The key idea is that these gradients are the same as the gradients for a supervised neural network; we just have to design the right graph such that its overall gradient is the policy gradient $\nabla_{\theta} J(\theta)$.

We have to essentially set the loss function to be a *weighted maximum likelihood* loss function (recall the *comparison* between maximum likelihood and policy gradient), weighted by reward.

$$\tilde{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) \hat{Q}_t^{(i)}$$

where $\hat{Q}_t^{(i)} = \sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})$. $\log \pi_{\theta}$ is cross-entropy for discrete problems and MSE for continuous (Gaussian) problems, like with maximum likelihood.

The pseudocode for maximum likelihood (discrete) for supervised learning would look like

```
logits = policy.predictions(states)
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(labels=actions, logits=logits)
loss = tf.reduce_mean(negative_likelihoods)
gradients = loss.gradients(loss, variables)
```

While the pseudocode for policy gradient (discrete) for reinforcement learning would look like

```
logits = policy.predictions(states)
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(labels=actions, logits=logits)
weighted_negative_likelihoods = tf.multiply(negative_likelihoods, q_values)
loss = tf.reduce_mean(weighted_negative_likelihoods)
gradients = loss.gradients(loss, variables)
```

And that's it!

Reminders

- High variance gradients
- Larger batch sizes may be better
- Tweaking learning rates is hard!

Lecture 6: Actor-Critic Algorithms

Improving Policy Gradient with Values

Previously, we discussed the application of **causality** to reduce variance in policy gradient methods. In essence, we multiply the gradient for each time step t by the "reward-to-go" $\hat{Q}_t^{(i)}$, the rewards received starting from this time step until the end of the sample.

This reward-to-go is actually a random variable, since the rest of the trajectory is random (since the policy itself is stochastic). (As in, if you were to start from the same state and action pair multiple times, the reward to go would vary across the iterations due to the stochasticity of the policy). Naturally, this random variable has some associated variance; in fact, we can produce an estimate of $\hat{Q}_t^{(i)}$ that has **lower variance** relative to our existing estimate, i.e.

$$\hat{Q}_t^{(i)} \approx \sum_{t'=t}^H \mathbb{E}_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'} \mid \mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})]$$

Ideally, we'd like our estimator to be an unbiased estimate of the **true** expected reward-to-go.

$$Q(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H \mathbb{E}_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mid \mathbf{s}_t, \mathbf{a}_t]$$

The current $\hat{Q}_t^{(i)}$ is unbiased (it is the same formula, after all, just taken over a few trajectories instead of the whole space) but experiences high variance.

Baseline?

Recall the **baseline** that replaced our reward-to-go estimate with $r(\tau) - b_t$, for some value b_t . What should this value be? Previously, we discussed how the **average reward** served as a good baseline. However, this was for policy gradient methods **without causality**—what's a good baseline for our reward-to-go, i.e. $\hat{Q}_t^{(i)}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = Q_t(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - b_t$?

As it turns out, using the mean as a metric is still good; but, instead of averaging across rewards, we average across the possible Q -values for the state at time t instead, or just the average reward-to-go.

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} [Q(\mathbf{s}_t, \mathbf{a}_t)]$$

This is known as the **value function**. As with the average reward baseline, this results in an estimator that retains unbiasedness while lowering variance. Thus,

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_\theta \log_\theta(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)}) (Q(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - V(\mathbf{s}_t^{(i)}))$$

The expression $Q - V$ is so important it's denoted the **advantage**.

$$A(\mathbf{s}_t, \mathbf{a}_t) = Q(\mathbf{s}_t, \mathbf{a}_t) - V(\mathbf{s}_t)$$

Positive advantage is assigned to actions that are **better than average** for the current state, and negative advantage is assigned to those that are worse than average.

State/State-Action Value Functions

In short,

- $Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$, the Q -function, is the total reward from taking $\mathbf{a}_t$ in $\mathbf{s}_t$.
- $V^\pi(\mathbf{s}_t)$, the value function, is the total reward from $\mathbf{s}_t$.
- $A^\pi(\mathbf{s}_t, \mathbf{a}_t)$, the advantage function, is the measure of how much better $\mathbf{a}_t$ is for $\mathbf{s}_t$.

An **actor-critic** method will contain two models: one to capture the policy (actor), and one to capture the advantage function (critic). The latter estimator is typically a Q -function or value function.

Policy Evaluation

Policy evaluation is the process of using a policy π to estimate V^π or Q^π .

Q^π has a nice, recursive expression.

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [V^\pi(\mathbf{s}_{t+1})]$$

which we can estimate with a single sample as

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1})$$

This introduces a bit of variance due to replacing an expectation with a single sample, but not too much since it's only over a single step. This results in a nice expression for the advantage, though, that is expressed *only in terms of $V^\pi(\mathbf{s})$* .

$$A^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1}) - V^\pi(\mathbf{s}_t)$$

Thus, we can just try to fit a model $\hat{V}^\pi(\mathbf{s})$ with parameters ϕ to estimate $V^\pi(\mathbf{s})$. (Note, this is possible with just Q -functions too, but this is originally how actor-critic methods were done).



Tip

This estimator of A^π remains *unbiased* because the expectation was replaced with a single sample taken from the true data generating distribution.



Info

The RL objective is just $J(\theta) = \mathbb{E}_{\mathbf{s}_1 \sim p(\mathbf{s}_1)} [V^\pi(\mathbf{s}_1)]$.

So how do we fit a model $\hat{V}^\pi(\mathbf{s})$?

Monte Carlo Method

Monte Carlo policy evaluation is the most natural method, and just involves generating some samples and averaging together (what our policy gradient method already does!). One drawback, however, is that to produce multiple samples for a single state, it requires *resetting* the simulator/environment back to the exact same state, which isn't necessarily possible; however, we can still use the single sample estimator.

Critically, this single sample estimator can still be effective! The key idea is that many states observed from our data will be similar, and a good neural network/model should be able to learn this similarity and effectively "average" their resulting trajectories/reward-to-go's together. Thus, the model we train to predict $V^\pi(\mathbf{s})$ will actually provide *better estimates* than the single sample estimates themselves.

Info

Actual Monte Carlo policy estimation, i.e. resetting the environment and measuring multiple trajectories, is better, but the model-based method works well in practice and is effectively an *approximation* of true Monte Carlo estimation.

The model is typically some sort of supervised regression, e.g. MSE regression

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^N \sum_{t=1}^H \|\hat{V}_{\phi}^{\pi}(\mathbf{s}_t^{(i)}) - y_t^{(i)}\|^2$$

where the target label $y_t^{(i)}$ is

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + V^{\pi}(\mathbf{s}_{t+1}^{(i)})$$

and V^{π} is the measured value function for that trajectory, i.e. a single sample estimate of the value function.

Bootstrapping

Instead of using the single-sample estimate of V^{π} to generate our target labels, however, we can use our *previously fitted* value function $\hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$ to estimate $V^{\pi}(\mathbf{s}_{t+1}^{(i)})$, i.e.

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$$

This is known as **bootstrapping**.

Tip

$y_t^{(i)}$, as a target value, is treated as a *constant* when calculating the gradient with respect to ϕ . This is usually denoted by the **stop-gradient** operator $[\dots]_{\times}$, e.g.

$$\mathcal{L}(\phi) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H \left(\hat{V}_{\phi}^{\pi}(\mathbf{s}_t^{(i)}) - \left[r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)}) \right]_{\times} \right)^2 \right]$$

This is used for bootstrapping because ϕ is a parameter in calculating $\hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$, but its gradient should not be included when calculating $\nabla_{\phi} \mathcal{L}(\phi)$ because the target value $y_t^{(i)}$ should be a constant, and should not be "trained" or "learned." (Wouldn't really make sense for the model to learn to adjust the target value it's trying to match, now would it?).

Warning

Use of bootstrapping creates a *biased* estimator, due to an **imperfect critic** $\hat{V}_{\phi}^{\pi}$. Despite this, it performs much better than the standard Monte Carlo estimator because it substantially reduces variance.

Info

Bootstrapping is known as **temporal difference (TD) learning**, which we will discuss in further detail in future lectures.

This has some issues, though, for **infinite horizon** RL problems. Imagine an environment where every single action (regardless of state) is 1. Regardless of what the neural network/model of $\hat{V}$ is initialized to, its predictions will grow without bound, as the cyclical nature of bootstrapping causes increases in $y_t^{(i)}$ to increase $\hat{V}$, which increase $y_t^{(i)}$, which increases $\hat{V}$, etc.

Discount Factors

We can fix this, though, with **discounting**! Discounting represents the idea that sooner rewards are better than later rewards with the same value, and is mathematically

$$y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})$$

for some constant **discount factor** $\gamma \in [0, 1]$ (in practice, $\gamma \approx 1$, e.g. 0.99).

Notably, this changes the MDP. Every state now has a $1 - \gamma$ probability of transitioning to a "death" or absorbing state, in which no more rewards may be received/there are no transitions out of the death state. Thus, intuitively, discounting represents a belief that there is a $1 - \gamma$ probability that the horizon will end now, and therefore it's more desirable to receive rewards sooner than later.

Discounting also helps **reduce variance**. By deprioritizing rewards further in the future, we reduce variance because those rewards are more uncertain (higher variance) than more immediate rewards.



Tip

With discounting, we practically always use the reward-to-go formulation, i.e. causality. Why? Because this discounts rewards relative to the current time step. Without causality, it would just discount later rewards **not** relative to the current time step, but relative to $t = 0$.

Time-Varying vs. Time-Invariant

Oftentimes, we don't actually care the time step at which data is collected! Instead, we only care about the **transition** probabilities, i.e. we consider $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i')$ instead of $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})$. This just lets us change notation around, i.e. training data becomes

$$\{(\mathbf{s}_t^{(i)}, r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)}))\} \longrightarrow \{(\mathbf{s}_i, r(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_\phi^\pi(\mathbf{s}_i'))\}$$

and similarly for other expressions. The problem itself doesn't change.

Examples

- TD-Gammon played Backgammon, using a value function that just estimated the expected outcome given board state
- AlphaGo played Go, using the same value function but a bigger, more advanced model

The Actor-Critic Algorithm

Basic Actor-Critic

1. Sample $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i'\}$ from $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ (run policy).
2. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_i')$.

3. Refit $\hat{V}$ to targets $\{y_i\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_i') - V_\phi^\pi(\mathbf{s}_i)$.
5. Compute $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$.
6. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.
7. Repeat!

Online Actor-Critic

One may realize that, because we're just using transitions as training data, rather than whole trajectories, we don't need to use a whole trajectory or set of trajectories each iteration of model training.

The most extreme example of this is using *only one transition* to train the model each iteration. This actually allows use to have a fully online actor-critic algorithm that, during the simulation/in the environment, takes one transition, trains the model on that transition, and then uses the updated model to decide its next action.

This, however, has several issues.

- It is biased, as $\hat{V}$ is essentially "out of date" by one time step.
- The data is not i.i.d. anymore! The next time step's data is dependent on the current time step's data.
- Small batch size of 1 is very volatile.

In practice, this can actually work, but only with multiple parallel workers and lots of hyperparameter tuning.

On-Policy Actor-Critic

Can we improve the basic actor-critic algorithm? Yes!

- Better ways to estimate $\hat{A}^\pi$. (Improve the *critic*)
- Better ways to estimate $\nabla_\theta J(\theta)$. (Improve the *actor*)

So far, we've seen two methods of estimating $\nabla_\theta J(\theta)$. *Actor-critic* methods lower variance, but add some bias due to an *imperfect critic*, i.e. $\hat{V}_\phi^\pi$. *Policy-gradient* methods are entirely unbiased, but has higher variance due to it being a single sample estimate.

🔗 Which actor-critic RL objectives are biased, and which are unbiased? >

The actor-critic RL objectives here all (generally) take the form

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^H \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) (\hat{Q}^\pi(\mathbf{s}_t, \mathbf{a}_t) - b) \right]$$

for an estimator $\hat{Q}^\pi$ of the true Q -function (or the advantage function) and a baseline b . The key idea is that $\hat{Q}^\pi(\mathbf{s}_t, \mathbf{a}_t)$ must remain an unbiased estimator of the Q -function to keep the estimator unbiased. However, the baseline b need only be independent of the current action $\mathbf{a}_t$ to keep the estimator unbiased.

🔗 Question

Is it possible to use $\hat{V}^\pi$ but still produce an unbiased estimate?

In fact, we can, with the following estimator. (Note that we are actually improving the *critic* here).

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\left(\sum_{t'=t}^H \gamma^{t'-t} r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t^{(i)}) \right)$$

which is essentially a compromise between the two methods, i.e. it's policy-gradient but the baseline is simply replaced by the value function. This produces a lower variance, unbiased estimator. (Sidenote: this is also a discounted, reward-to-go formulation).

? Why is this unbiased?

As long as the baseline used doesn't depend on θ , it remains unbiased. See [Lecture 5](#) for the proof of an unbiased baseline; it may be reused here.

However, it'd be nice if we could have a sort of "sliding scale" that determines the degree of mixing between the two methods, rather than only having this one combination option...

Eligibility Traces and n -step Returns

Let $\hat{A}_C(\mathbf{s}_t, \mathbf{a}_t)$ denote the standard actor-critic estimator of advantage

$$r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$$

Let $\hat{A}_{MC}$ denote the standard Monte Carlo estimator of advantage

$$\sum_{t'=t}^{\infty} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$$

In RL problems, we'd naturally expect higher variance further in the future. Thus, ideally, we'd like to use a high variance, unbiased estimator for time steps closer to the present, and have variance decrease (while bias may increase) as time moves further into the future.

One way of achieving this is to use an *n -step return estimator*.

$$\hat{A}_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t) + \gamma^n \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+n})$$

Essentially, the n -step estimator uses the Monte Carlo estimator for n steps—unbiased, but high variance—and then *switches* to the actor-critic estimator for the remaining steps until the end of the horizon (possibly ∞)—high bias, low variance. This provides a *discrete* cutoff point at which we switch between the two estimators. $n > 1$ commonly works better!

But, can we make a *continuous* averages of these two methods?

Generalized Advantage Estimation (GAE)

The *generalized advantage estimator* $\hat{A}_{GAE}^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ is a *weighted average of n -step returns*.

$$\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n \hat{A}_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$$

where, typically, $w_n \propto \lambda^{n-1}$, so that n -step returns closer to the present are weighted more, therefore reducing variance. Moreover, it leads to an elegant simplification of the advantage estimator.

$$\begin{aligned} \hat{A}_{\text{GAE}}^{\pi} &= r(\mathbf{s}_t, \mathbf{a}_t) + \gamma((1 - \lambda)\hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}) + \lambda(r(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) + \dots)) \\ &= \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'} \end{aligned}$$

where $\delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma\hat{V}_{\phi}^{\pi}(\mathbf{s}_{t'+1}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t'})$. In other words, the λ behaves similarly to the discount factor γ . This expression may then be rewritten as a vastly *more efficient recursive* formula

$$\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \delta_{t'} + \gamma\lambda\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})$$

Info

This is the most popular advantage estimator used for policy gradient methods in modern reinforcement learning.

is λ the exact same as γ ?

No. γ actually affects the RL objective; smaller γ means more short-term prioritization. λ only affects the accuracy (bias/variance) of your estimator, not your actual objective. Hence, they should be adjusted as two distinct hyperparameters.

Tip

In practice, the advantage is also centered/normalized.

Off-Policy Actor-Critic

On-Policy vs Off-Policy

On-policy actor-critic methods update the policy using data generated from the current policy, and are

- Computationally cheap
- Sample inefficient

while off-policy actor-critic methods update the policy using past data generated from old policies, and are

- Computationally expensive
- Sample efficient

Replay Buffer

The key idea behind off-policy algorithms is the *reuse of data*. On-policy algorithms throw out all previous data with every iteration. Off-policy algorithms store previous data in a *replay buffer*.

Recall the online single-sample on-policy actor-critic method from [before](#). What if, instead of using the currently sampled transition to evaluate our policy, we *store* the current transition into our replay buffer for later use, and use a *minibatch* sampled from the replay buffer to evaluate our policy? Well, this would actually break the algorithm—the data in the replay buffer will no longer be *representative* of our current value estimator, and may instead be garbage from past, bad value estimators; thus, our policy evaluation may return garbage results.

Critically, though, *if we estimate the Q -function instead of the value function*, this works! How?

First, consider why the value function method doesn't work. We have

$$\begin{aligned} y_i &= r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i), \quad (\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i) \sim \bar{\pi} \\ \mathcal{L}(\phi) &= \frac{1}{2} \sum_{i=1}^N \|\hat{V}_\phi^\pi(\mathbf{s}_i) - y_i\|^2 \\ V^\pi(\mathbf{s}_i) &\approx \mathbb{E}_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s}_i)} [r(\mathbf{s}_i, \mathbf{a}) + \gamma V^\pi(\mathbf{s}')] \end{aligned}$$

However, we want $V^\pi(\mathbf{s}_i)$ to be an expectation where $\mathbf{a}$ is sampled from $\pi_\theta(\mathbf{a} | \mathbf{s}_i)$, not $\bar{\pi}$.

Now, let's look at using a Q -function instead. We have

$$\begin{aligned} y_i &= r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')], \quad (\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i) \sim \bar{\pi} \\ \mathcal{L}(\phi) &= \frac{1}{2} \sum_{i=1}^N \|\hat{Q}_\phi^\pi(\mathbf{s}_i) - y_i\|^2 \\ Q^\pi(\mathbf{s}_i, \mathbf{a}_i) &\approx \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')] \end{aligned}$$

The key distinction is that, while the data is still sampled from an old policy, the expectation is computed over actions drawn from the current policy π_θ . Why? Because we don't have to interact with the environment to sample from the latest policy and compute our Q -function, i.e. these are both represented by neural networks/models we are training. In contrast, the value function method must sample the actions from the old policy data; it cannot instead sample $\mathbf{a}'$ from the current policy because we're estimating $V^\pi(\mathbf{s}_i)$, and estimating $V^\pi(\mathbf{s}'_i)$ would require sampling a new transition/interacting with the environment, not reusing old policy data. (Because the estimate would be based on the next state, so, without state transition probabilities, one would need to simulate taking action $\mathbf{a}'$ from state $\mathbf{s}'$ in the environment to produce the next state, and then V^π of this state may be computed).

This new algorithm needs a few more tweaks to finish it off.

1. Sample $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ using π_θ with one step and store in the replay buffer $\mathcal{R}$.
2. Load a minibatch and evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')] for each transition.$
3. Update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$, where B is the minibatch size.
4. Reuse the above trick to now *sample* $\mathbf{a}_i$ (rather than $\mathbf{a}'_i$) from the latest policy, i.e. $\mathbf{a}_i^\pi \sim \pi_\theta(\mathbf{a} | \mathbf{s}_i)$, and compute $\nabla_\theta J(\theta) = \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i^\pi | \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i^\pi)$.
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.
6. Repeat



Tip

In practice, $\hat{Q}_\phi^\pi$ is used in place of $\hat{A}^\pi$ because variance only really matters when sampling is expensive, as using more data/samples naturally lowers variance, and therefore lowering variance through techniques like baselines is not as necessary for this off-policy actor-critic Q method. (And excluding variance-lowering methods results in lower implementation complexity).



Is it an issue that the state distribution doesn't match, i.e. the state distribution is sampled from old policies?

Theoretically, yes, but in practice, with a sufficiently high capacity model, this poses no issues, as it doesn't really hurt the model.

Reparametrization Trick

Previously, with [direct policy differentiation](#), we had to use samples from our policy used with the environment to compute the gradient $\nabla_{\theta} J(\theta)$. However, with this off-policy method, to estimate $\nabla_{\theta} J(\theta)$, we are estimating, essentially,

$$\nabla_{\theta} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})]$$

Crucially, $\hat{Q}$ and π_{θ} are both neural networks/models that we are training, and thus can evaluate **without involving system dynamics/the environment at all!** Therefore, we can actually compute $\frac{d}{d\mathbf{a}} \hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})$ with backprop, which produces a better gradient estimator.

Warning

This only works for continuous, parametric action distributions and when $\hat{Q}$ is differentiable.

How can we utilize this? WLOG, assume $\pi_{\theta}(\mathbf{a} | \mathbf{s}) = \mathcal{N}(\mu_{\theta}(\mathbf{s}), \sigma_{\theta}(\mathbf{s}))$. Let $\epsilon \sim \mathcal{N}(0, 1)$. Then, one may note that $\mathbf{a} = \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon$. This allows us to approximate

$$\begin{aligned} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] &= \mathbb{E}_{\epsilon \sim \mathcal{N}(0, 1)} [Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon)] \\ &\approx Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon) \\ \nabla_{\theta} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] &\approx \nabla_{\theta} Q(\mathbf{s}, \mu_{\theta}(\mathbf{s}) + \sigma_{\theta}(\mathbf{s})\epsilon) \end{aligned}$$

and the last expression is easy to differentiate with an autograd library. This is known as the **reparametrized gradient estimator**.

Finally, we have an actual, practical algorithm for RL :)

What??

See the last part of [this section](#) of Lecture 12 that I wrote after I did some more research into the reparametrization trick. Note that it may help to know what variational inference is, i.e. [Lecture 11](#).

Summary

A collection of key formulas.

On-policy Actor/Advantage Estimation

$\hat{A}_{\text{MC}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^H \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$	Monte Carlo
$\hat{A}_{\text{C}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$	Bootstrapped
$\hat{A}_{\text{n}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma^n \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+n}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t)$	n -step
$\begin{aligned} \hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) &= \frac{1-\lambda}{1-\lambda^{H-t}} \sum_{n=1}^{H-t} \lambda^{n-1} \hat{A}_{\text{n}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \\ &= \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \hat{A}_{\text{C}}^{\pi}(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \\ &= \hat{A}_{\text{C}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) + \gamma\lambda \hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \end{aligned}$	Generalized Advantage

Note that $\frac{1-\lambda}{1-\lambda^{H-t}} = \frac{1}{1+\lambda+\dots+\lambda^{H-t-1}}$ in the GAE expression is the aforementioned *normalizing factor*.

Lecture 7: Value-Based RL

Actor-Critic without the Actor

Recall the off-policy actor-critic algorithm from [the previous lecture](#). Now, imagine we have a *small, discrete* action space. Instead of having a distinct actor model and critic model, we can just have a critic model, and define the actor's policy implicitly as based on the critic, i.e.

$$\pi_{\theta}(\mathbf{a} \mid \mathbf{s}) = \begin{cases} 1, & \mathbf{a} = \arg \max_{\mathbf{a}} \hat{Q}_{\phi}^{\pi}(\mathbf{s}, \mathbf{a}) \\ 0, & \text{otherwise} \end{cases}$$

This simplifies our algorithm to the following steps. Note that we'll rename our parameters to Q_{θ} instead of Q_{ϕ} to follow standard terminology.

1. Get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking one step, store in replay buffer
2. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}' \mid \mathbf{s}'_i)} [\hat{Q}_{\theta}(\mathbf{s}'_i, \mathbf{a}')] = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} \hat{Q}_{\theta}(\mathbf{s}'_i, \mathbf{a}')$.
3. Update ϕ using $\nabla_{\phi} \sum_{i=1}^B \|\hat{Q}_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.

This is known as an *off-policy critic* algorithm, and is known as *Q-learning*.

Policy Iteration and Dynamic Programming

We'll now discuss a different sort of thinking path that leads to the same conclusion, i.e. *Q-learning*.

Policy iteration, at a high level, is a two-step algorithm.

1. **Policy Evaluation:** Fit Q^{π} or V^{π} .
2. **Policy Improvement:** Improve the policy $\pi \leftarrow \pi'$.

Actor-critic and *Q-learning* are both algorithms within this family.

Value Function DP

First, we assume knowledge of $p(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$ and that the state and action space are small and discrete. For instance, consider a small grid of states, with an action space of moving up, down, left, or right. We can store the entirety of $V^{\pi}(\mathbf{s})$ in a table, and perform *bootstrapped* policy evaluation

$$V^{\pi}(\mathbf{s}) \leftarrow \mathbb{E}_{\mathbf{a} \sim \pi(\mathbf{a} \mid \mathbf{s})} [r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})} [V^{\pi}(\mathbf{s}')]]$$

where we calculate over the entire space, i.e. all $(\mathbf{s}', \mathbf{a}')$, which provides an exact calculation, not an estimate (no sampling), of the updated value function. Afterwards, we may update our policy, i.e.

$$\pi_{\theta}(\mathbf{a} \mid \mathbf{s}) = \begin{cases} 1, & \mathbf{a} = \arg \max_{\mathbf{a}} \hat{Q}_{\phi}^{\pi}(\mathbf{s}, \mathbf{a}) \\ 0, & \text{otherwise} \end{cases}$$

to produce a *deterministic* policy. Notably, the deterministic policy actually allows us to simplify the policy evaluation equation to

$$V^{\pi}(\mathbf{s}) \leftarrow r(\mathbf{s}, \pi(\mathbf{s})) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})} [V^{\pi}(\mathbf{s}')]]$$

Notably, since there is *no sampling*, and thus no policy running, this is simply dynamic programming, and not really any reinforcement learning! This is also known as *tabular policy learning*.

Value Iteration

We can also use Q functions for DP. In essence, we change our policy evaluation step to

$$Q^\pi(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\mathbf{s}'|\mathbf{s}, \mathbf{a})} [V^\pi(\mathbf{s}')]]$$

and change our policy improvement step to

$$V^\pi(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a})$$

where our policy is implicitly defined by *greedily* following the value function. This is known as **value iteration**, which is essentially just a one step algorithm.

Fitted Value Iteration

Now, let's imagine our state space has grown much larger, and thus it is no longer tractable to represent V^π in a tabular format. How do we represent $V(\mathbf{s})$?

Of course, the natural idea is to use a neural network that regresses on $V(\mathbf{s})$ with an MSE loss function. This produces the **fitted value iteration** algorithm.

1. Set $y_i \leftarrow \max_{\mathbf{a}_i} (r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}[V_\theta(\mathbf{s}'_i)])$.
2. Set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|V_\theta(\mathbf{s}_i) - y_i\|^2$.

Notably, we still require knowledge of the state transition probabilities and the action space must be small and discrete, due to the nature of step 1.

In practice, however, we won't know state transition probabilities, and we need to use samples rather than iterating over the entire action space...

Fitted Q Iteration

The trick is to simply use Q -functions instead of value functions!

1. Set $y_i \leftarrow r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}[V_\theta(\mathbf{s}'_i)]$, where we can approximate $\mathbb{E}[V_\theta(\mathbf{s}'_i)] \approx \max_{\mathbf{a}'_i} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)$.
2. Set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.

The key idea is that the Q function, $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, is also used to compute an expected value over actions, but does so without needing extra sampling/known the state transition probabilities; the best action is already known according to the estimated Q_θ , by taking the max value over existing samples.

? Why doesn't this work with value functions? >

With value functions, if we were to compute y_i using samples $\mathbf{a}_i$, we would be evaluating our current policy π' using samples from an old policy π ... not good! This results in an outdated policy evaluation, and therefore the target values for V_θ aren't representative of the true, current V_θ , i.e. the labels y_i are outdated. Hence, fitted value iteration is *not* an off-policy algorithm; it does not use any samples, but rather iterates over the entire action space and uses its knowledge of the state transition probabilities to compute its estimate of the value function. Fitted Q iteration *is* able to act as an off-policy algorithm, and uses samples to estimate Q .



Tip >

There are two ways to model the $\arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}_i', \mathbf{a}_i')$.

1. Make a neural network that takes in $\mathbf{s}_i, \mathbf{a}_i$ and outputs the expected Q -value. This is the intuitive method (model $Q(\mathbf{s}, \mathbf{a})$ directly), and is the implied method above. (Max taken over existing samples).
2. If the action space is small, make a neural network that takes in $\mathbf{s}_i$ and outputs the expected Q -value for all $\mathbf{a}_i$. Then, the arg max is just computed directly over the resulting outputs, without needing to use existing sample actions.

Q -learning works for off-policy samples, and only has one neural network, and therefore no high-variance policy gradient. However, it lacks convergence guarantees for nonlinear function approximations—an issue we'll discuss more later.

Our full fitted Q -iteration algorithm is thus

1. Collect B samples $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i')$ to add to the replay buffer.
2. Set $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}_i', \mathbf{a}_i')$.
3. Set $\theta \leftarrow \arg \min_{\theta} \frac{1}{2} \sum_i \|Q_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
4. Repeat steps 2-3 K times, for some hyperparameter K .
5. Go back to step 1 and repeat.

Return to Q -Learning

Q -Iteration Optimization Objective

First, what is fitted Q -iteration really optimizing?

Well, it's performing some regression, and attempting to compute parameters θ for our model Q_{θ} that minimize the error $\mathcal{E}$

$$\begin{aligned}\mathcal{E} &= \frac{1}{2} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \beta} [(Q_{\theta}(\mathbf{s}, \mathbf{a}) - y_i)^2] \\ &= \frac{1}{2} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \beta} [(Q_{\theta}(\mathbf{s}, \mathbf{a}) - [r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}')])^2]\end{aligned}$$

where β is the dataset/replay buffer we're sampling from.

If $\mathcal{E} = 0$, then $Q_{\theta}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}')$, and Q_{θ} is an **optimal** Q -function Q^* that corresponds to the optimal policy π^* .

Online Q -Learning

We can produce the **online Q -learning** algorithm by simply using fitted Q -iteration with $B, K = 1$.

1. Take some action $\mathbf{a}$, observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i')$.
2. $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}_i', \mathbf{a}_i')$.
3. $\theta \leftarrow \theta - \alpha \frac{dQ_{\theta}}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_{\theta}(\mathbf{s}_i, \mathbf{a}_i) - y_i)$.

Notably, this is still **off-policy**, unlike the **online actor-critic algorithm** we covered last lecture. Thus, we have a choice in sampling $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}_i')$; most importantly, it doesn't have to be the result of taking the best action according to our current policy. This allows us to **explore** more of the action space.

Exploratory Q-Learning

We can extend this idea of *exploration* with **ϵ -greedy methods**! In essence, we act according to the policy

$$\pi(\mathbf{a}_t \mid \mathbf{s}_t) = \begin{cases} 1 - \epsilon, & \text{if } \mathbf{a}_t = \arg \max_{\mathbf{a}_t} Q_\theta(\mathbf{s}_t, \mathbf{a}_t) \\ \epsilon / (|\mathcal{A}| - 1), & \text{otherwise} \end{cases}$$

which chooses the greedy action with probability $1 - \epsilon$ and chooses a different action uniformly at random with probability ϵ .

An alternative is to use a **soft max**, which ensures

$$\pi(\mathbf{a}_t \mid \mathbf{s}_t) \propto \exp(Q_\theta(\mathbf{s}_t, \mathbf{a}_t))$$

This is known as **Boltzmann exploration**, and basically weights the non-greedy actions according to their estimated values, rather than just choosing from them randomly.

Warning

Simply applying the Q-Learning algorithm as it is now is not sufficient; we need some tricks to make it work in practice, which we'll discuss in future lectures!

Lecture 8: Q-Learning in Practice

Target Networks

So, why doesn't Q -learning work if we just implement what we discussed in [Lecture 7](#)? Well, it's not really gradient descent. Recall that the target values are associated with a *stop-gradient* operator, i.e.

$$\theta \leftarrow \phi - \alpha \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_\theta(\mathbf{s}'_i, \mathbf{a}'_i)])$$

therefore, Q -learning is not gradient descent because it doesn't have a well-defined objective, and thus it does not necessarily converge!

Recall that Q -learning is a type of fitted Q -iteration algorithm. The primary difference, however, is that Q -learning takes only *one step* between each gradient update. In contrast, Q -iteration collects a large, static dataset of transitions, and then performs a *full regression*, not just a single gradient step, on the entire dataset. In essence, Q -learning chases a moving target during optimization, while Q -iteration "freezes" the target y_i values and performs essentially standard supervised learning each step to find the new θ .

In some sense, one can think of this aspect of Q -iteration as storing a "frozen copy" of the model for Q_θ when learning the next value of θ . We can attempt to achieve this same "static" property of Q -iteration in Q -learning with this frozen copy idea—we separately store a model $Q_{\bar{\theta}}$ where $\bar{\theta} \leftarrow \theta$ every X iterations, e.g. $X = 10^4$. Then, we use this frozen Q -function to calculate the targets y_i , i.e.

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)$$

this is known as *Q-learning with target networks*, and allows us to stabilize learning a bit and "slow the moving target."

This has some drawbacks, though. In particular, $Q_{\bar{\theta}}$ lags behind Q_θ , and may produce a fairly inaccurate representation of the true Q -function.

Deep Q-Learning Network (DQN)

Famous foundational reinforcement learning algorithm—it's just generalized Q -learning with a replay buffer and target network.

1. Save target network parameters: $\bar{\theta} \leftarrow \theta$.
2. Collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ with some policy, add to buffer $\mathcal{R}$.
3. Sample batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i) \sim \mathcal{R}$.
4. Update $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i) (Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$.

where the indentation denotes nested loops. For DQN, the innermost loop iterates 1 time and the loop starting on step 2 iterates 1 times.

Alternatives to Target Networks

Target networks, notably are *discrete* updates, in that they update only every X steps. This produces bursty changes in the learning algorithm. In practice, this isn't usually an issue; but, if necessary, one solution is to use *Polyak averaging*, in which we update $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$ (e.g. $\tau = 0.999$).

More Efficient Q-Learning

Generalized Q -learning with a replay buffer and target network can be represented as three different processes that essentially run at their own rates.

1. Data collection (also evicting old data).
2. Update target $\bar{\theta}$.

3. Q -function regression.

Here's the speeds for some different types of Q -learning.

- In online Q -learning, data is evicted immediately, and all three processes run at the same speed.
- In DQN, process 1 and process 3 run at the same speed, while process 2 is slow.
- In fitted Q -iteration, process 3 runs faster than process 2, which runs faster than process 1

Also, here's how we'll describe the generalized Q -learning algorithm with a replay buffer and target network.

1. Save target network parameters: $\bar{\theta} \leftarrow \theta$.
2. Collect dataset $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}$ with some policy, add to buffer $\mathcal{R}$. (Loop N times).
3. Sample batch $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i) \sim \mathcal{R}$. (Loop K times).
4. Update $\theta \leftarrow \theta - \alpha \sum_i \frac{dQ_\theta}{d\theta}(\mathbf{s}_i, \mathbf{a}_i)(Q_\theta(\mathbf{s}_i, \mathbf{a}_i) - [r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}'_i, \mathbf{a}'_i)])$.

So, which choices of N, K are most efficient?

K

Let's first focus on K . This is also known as the **update-to-data (UTD)** ratio. A higher K increases learning speed, as more updates are made for every data collection step, but this is more computationally expensive. Moreover, an excessively high K will cause overfitting to the current set of data. In fact, $K \geq 10$ is generally already a risky choice!

N

Now, consider N , the ratio of dataset collection steps to target network update steps. A lower N increases learning speed, as fewer steps between target network updates keeps the target network more up-to-date. Generally speaking, though, $N \sim [10^3, 10^4]$ as lower values creates a lot of instability.

n -step Returns?

In essence, we can apply n -step returns to the target values themselves!

$$y_t^{(i)} = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}^{(i)}) + \gamma^n \max_{\mathbf{a}_{t+n}^{(i)}} Q_{\bar{\theta}}(\mathbf{s}_{t+n}^{(i)}, \mathbf{a}_{t+n}^{(i)})$$

This *increases* learning speed because the target values are now calculated from a mix of the up-to-date reward values from a trajectory and the out-of-date target network. However, this has a major limitation—because we're learning with a *trajectory* that originates from older data from a worse policy, the target values are lower than they should be! Thus, the off-policy nature of Q -learning makes this n -step returns modification *biased*. Hence, people will typically use small n values if applying this technique.

Overestimation in Q -Learning

For naive DQN, the Q -functions tend to **overestimate** the actual returns. The problem is the max function in calculating the target value for target networks, i.e.

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}_i, \mathbf{a}_i)$$

Why? Because it essentially selects the Q -functions with the *largest positive error*.

To form some intuition, consider two random variables X_1, X_2 . Then, $\mathbb{E}[\max(X_1, X_2)] \geq \max(\mathbb{E}[X_1], \mathbb{E}[X_2])$. In other words, taking the max of the $Q_{\bar{\theta}}$ function **systematically overestimates** the maximum of the expected values of the actions.

Double Q-Learning

We note that

$$\max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}') = Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}'))$$

the key idea of double Q-learning is to use **two distinct models** in the RHS expression—one to choose the action (inner Q) and one to evaluate the value (outer Q). This helps eliminate the overestimation error because, if both models have *different noise patterns*, then the action chosen to maximize the inner Q (and thus possesses the largest positive error) does not necessarily correspond to the value with the largest positive error in the outer Q .

Classic double Q-learning was thus modeled as

$$\begin{aligned} Q_{\theta_A} &\leftarrow r + \gamma Q_{\theta_B}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_A}(\mathbf{s}', \mathbf{a}')) \\ Q_{\theta_B} &\leftarrow r + \gamma Q_{\theta_A}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta_B}(\mathbf{s}', \mathbf{a}')) \end{aligned}$$

In practice, though, there is an easier implementation for double Q-learning. One may note that DQN itself already has two Q -functions: the target network and the current Q -function. Thus, double Q-learning is modeled as

$$y = r + \gamma Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}'))$$

⚠ $\theta \sim \bar{\theta}??$

Aren't θ and $\bar{\theta}$ correlated? Yes, but they are distinct enough that this proves very effective in practice. And this method is just much easier to implement so :)

Generalized Double Q-Learning

Note that $\arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}')$ is essentially equivalent to taking the action according to a greedy policy, i.e.

$Q_{\bar{\theta}}(\mathbf{s}', \arg \max_{\mathbf{a}'} Q_{\theta}(\mathbf{s}', \mathbf{a}')) = \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}')]$. In other words,

$$y = r + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [Q_{\bar{\theta}}(\mathbf{s}', \mathbf{a}')]$$

Occasionally, if overestimation is a problem even after double Q-learning, some practitioners implement **clipped** double Q-learning.

$$y = r + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}')} [\min_{i \in \{1,2\}} Q_{\bar{\theta}_i}(\mathbf{s}', \mathbf{a}')]$$

where we now train an **ensemble** of two different target networks $Q_{\bar{\theta}_1}$ and $Q_{\bar{\theta}_2}$. (You may have any number of target networks, but it gets increasingly expensive). This can, however, cause *underestimates* for Q-learning, though it's often still practical for actor-critic!

Tips for Q-Learning

Practical Tips

- Q-learning takes time to stabilize
 - Test on easier tasks first to validate implementation

- Be patient when training
- Large replay buffers improve stability
- Start with high exploration (epsilon) and gradually reduce

Advanced Tips

- Bellman error gradients can be big; clip gradients or use Huber loss
- Double Q -learning is very effective in practice
- n -step returns can help a lot, but be careful
- Schedule exploration/learning rate or just use Adam
- Run multiple times with different seeds

Back to Continuous Actions

Q -learning with continuous actions

With continuous actions, choosing the value-maximizing action value is hard and inefficient. This is particularly problematic for when we're evaluating the target values, as this occurs in the innermost training loop. There are a couple of ways to solve this.

Solution 1: Stochastic Optimization

The simplest solution is to approximate $\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$ with some samples, i.e.

$$\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}) \approx \max\{Q(\mathbf{s}, \mathbf{a}_1), \dots, Q(\mathbf{s}, \mathbf{a}_N)\}$$

this is efficiently *parallelizable* and easy to implement, but unfortunately lacking in accuracy. We can do better.

- Cross-Entropy Method (CEM)
- CMA-ES

these stochastic optimizers work decently up to about 40 dimensions.



Info

Very popular improvement to offline actor-critic methods.

Such methods are also known as **sample and rank** or **rejection sampling**.

Solution 2: Learn Approximate Maximizer

The classic example is **deep deterministic policy gradient (DDPG)**, and is essentially *deterministic actor critic*. The idea is to train another model $\mu_{\theta}(\mathbf{s})$ to approximate $\arg \max_{\mathbf{a}} Q_{\phi}(\mathbf{s}, \mathbf{a})$, where we find $\theta \leftarrow \arg \max_{\theta} Q_{\phi}(\mathbf{s}, \mu_{\theta}(\mathbf{s}))$. This is found by using the following

$$\frac{dQ_{\phi}}{d\theta} = \frac{d\mathbf{a}}{d\theta} \frac{dQ_{\phi}}{d\mathbf{a}}$$

which is essentially the same as the **reparametrization trick** from Lecture 6.

The new target value is thus

$$y_j = r_j + \gamma Q_{\bar{\phi}}(\mathbf{s}'_j, \mu_{\bar{\theta}}(\mathbf{s}'_j))$$

🔍 Deterministic Actor-Critic?

$\mu_{\bar{\theta}}$ is the actor, $Q_{\bar{\phi}}$ is the critic. Deterministic because there is no expected value on $Q_{\bar{\phi}}$ over sampled actions from the policy.

Some Q-Learning Theory

Value Iteration

Let's return to value iteration.

1. $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \mathbb{E}[V(\mathbf{s}')$.
2. $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$.
3. Repeat.

Does value iteration converge?

Let $\mathcal{B}$ be an operator such that, for a vector V composed of the values of the different states,

$$\mathcal{B}V = \max_{\mathbf{a}} r_{\mathbf{a}} + \gamma \mathcal{T}_{\mathbf{a}}V$$

where r is the stacked vector of rewards at all states for action $\mathbf{a}$ and $\mathcal{T}_{\mathbf{a}}$ is the matrix of transitions for action $\mathbf{a}$, i.e. $\mathcal{T}_{\mathbf{a},i,j} = p(\mathbf{s}' = i \mid \mathbf{s} = j, \mathbf{a})$. This is essentially value iteration written out with just an operator $\mathcal{B}$ and a value vector V , i.e.

1. $V \leftarrow \mathcal{B}V$
2. Repeat

We note that V^* , the vector representing the *optimal* value function, is a **fixed point** of $\mathcal{B}$, i.e. $V^* = \mathcal{B}V^*$. We can prove that V^* always exists, is always unique, and is always optimal. The way we prove that $\mathcal{B}$ converges to V^* is by showing that $\mathcal{B}$ is a **contraction**. That is, for any vectors $V, \bar{V}$, $\|\mathcal{B}V - \mathcal{B}\bar{V}\|_{\infty} \leq \gamma \|V - \bar{V}\|_{\infty}$. In other words, the gap always gets smaller by at least a factor of γ . We will not prove this now, just take my word :)

If we choose $\bar{V} = V^*$, we have that $\|\mathcal{B}V - \mathcal{B}V^*\|_{\infty} = \|\mathcal{B}V - V^*\|_{\infty} \leq \gamma \|V - V^*\|_{\infty}$, or that the operator always contracts the gap. Thus, value iteration converges.

□

Non-Tabular Value Function Learning

Non-tabular value function learning, e.g. fitted value iteration, is a bit trickier, because we train a model that has at least *some error*. However, we can interpret the regression performed by fitted value iteration as a *projection* (in the L2 norm, due to MSE) onto the space of functions representable by the model.

$$V' \leftarrow \arg \min_{V' \in \Omega} \frac{1}{2} \sum \|V'(\mathbf{s}) - (\mathcal{B}V)(\mathbf{s})\|^2$$

where Ω represents the space of representable value functions. In essence, each time we update $V \leftarrow \mathcal{B}V$, we first project $\mathcal{B}V$ back into Ω . To simplify this, we define a new operator

$$\Pi : \Pi V = \arg \min_{V' \in \Omega} \frac{1}{2} \sum \|V'(\mathbf{s}) - V(\mathbf{s})\|^2$$

so we can write fitted value iteration as

1. $V \leftarrow \Pi \mathcal{B} V$
2. Repeat

Now, $\mathcal{B}$ is a contraction w.r.t ∞ norm, and Π is a contraction w.r.t ℓ_2 norm. Yet, because they are contractions only w.r.t different norms, $\Pi \mathcal{B}$ is *not* a contraction! Thus, while value iteration does converge, fitted value iteration **does not converge**. (Though, with deep RL, Ω is much larger in practice, and therefore works well).

□

Fitted Q -Iteration, Actor-Critic

We can do the same for Q -iteration (and online Q -learning) and actor-critic, and find that they too are not guaranteed convergence for essentially the same reason.

Conclusions

- Fitted value-based methods (Q -learning, Q -function actor-critic) are frequently *unstable*!
 - In practice, not guaranteed convergence
 - Requires more hyperparameter tuning
- Some tricks are very helpful though...
 - Replay buffers
 - Target networks

Lecture 9: Advanced Policy Gradients: Part 1

Return to Policy Gradients

So, why policy gradient? They *converge*, unlike Q -function methods, and on-policy methods allow Monte Carlo advantage estimators (GAE with $\lambda = 1$), which provides an *unbiased* estimator. Thus, policy gradients are stable, reliable methods when *sample efficiency* is *not* a concern (samples are cheap to generate).

Let's consider our usual policy gradient algorithm with GAE.

1. Sample $\{\tau^{(i)}\}$ from $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ (run policy).
2. Evaluate $y_i = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1}^{(i)})$.
3. Refit $\hat{V}_\phi^\pi$ to targets $\{y_t^{(i)}\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}^{(i)}$.
5. Compute $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$.
6. Update $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

As it is now, it is computationally intractable for practical purposes.

- Each iteration takes one gradient step
- Each iteration requires generating new samples
- It is high variance, and therefore requires large batches and smaller learning rates (in other words, more iterations)

Really, it'd be nice to take *multiple gradient steps* for every sampling. In other words, repeat steps 5-6 some K times for every execution of steps 1-4. However, this is problematic—the trajectories being used are produced by an old policy, not π_θ .

The primary goal of this lecture will be to derive an algorithm that can achieve this goal without issue.

Importance Sampling

Importance sampling is a technique that estimates an expected value under a distribution from which we did not sample on. This is exactly our problem :)

It's described mathematically as follows.

$$\begin{aligned}\mathbb{E}_{x \sim p(x)}[f(x)] &= \int p(x) f(x) \, dx \\ &= \int \frac{q(x)}{q(x)} p(x) f(x) \, dx \\ &= \int q(x) \frac{p(x)}{q(x)} f(x) \, dx \\ &= \mathbb{E}_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right]\end{aligned}$$

Thus, our objective function

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[r(\tau)]$$

may be written using samples from a different policy $\bar{p}$ as

$$J(\theta) = \mathbb{E}_{\tau \sim \bar{p}(\tau)} \left[\frac{p_\theta(\tau)}{\bar{p}(\tau)} r(\tau) \right]$$

However, there's one critical issue with this formula. We do not assume knowledge of p_θ , as $p_\theta(\tau) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$. In other words, it's dependent on state transition probabilities, which we may not know. However, consider that

$$\frac{p_\theta(\tau)}{\bar{p}(\tau)} = \frac{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^H \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}}{\cancel{p(\mathbf{s}_1)} \prod_{t=1}^H \bar{\pi}(\mathbf{a}_t | \mathbf{s}_t) \cancel{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}} = \prod_{t=1}^H \frac{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)}{\bar{\pi}(\mathbf{a}_t | \mathbf{s}_t)}$$

This ratio may be computed using only knowledge of our policies!

Let's look at the gradients for a policy gradient algorithm with importance sampling. Define

$$J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} r(\tau) \right]$$

Then,

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\frac{\nabla_{\theta'} p_{\theta'}(\tau)}{p_\theta(\tau)} \right] = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_\theta(\tau)} \nabla_{\theta'} \log p_{\theta'}(\tau) r(\tau) \right]$$

where we once again use the identity that $p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$, only for $\theta = \theta'$ here. Note that this is similar to the on-policy policy gradient expression, except with an extra term $\frac{p_{\theta'}(\tau)}{p_\theta(\tau)}$ to account for the difference in policy.

We can substitute some terms in now (recall from policy gradient) to produce

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\left(\prod_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \right) \left(\sum_{t=1}^H \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

which may be rewritten **with causality** as

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=1}^H \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \underbrace{\left(\prod_{t'=1}^t \frac{\pi_{\theta'}(\mathbf{a}_{t'} | \mathbf{s}_{t'})}{\pi_\theta(\mathbf{a}_{t'} | \mathbf{s}_{t'})} \right)}_A \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \underbrace{\left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(\mathbf{a}_{t''} | \mathbf{s}_{t''})}{\pi_\theta(\mathbf{a}_{t''} | \mathbf{s}_{t''})} \right)}_B \right) \right]$$

where term A implements causality to ensure future actions don't affect the current weight, and term B represents the probability that this reward $r(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ represents the current policy. These are essentially the "weights" assigned to each reward.

Info

Term B is typically eliminated for, intuitively, the same motivation as the ideas from the [double Q-learning](#) section, i.e. maximizing over an old policy is actually more effective. We explore this in more detail next lecture.

We'll focus more on term A for the above reason. It turns out that term A can actually prove very problematic—as a product over such long time intervals, with each term likely < 1 since the trajectory produced by the old policy is likely to have a greater probability under the old policy compared to the new policy, it will vanish at an exponential rate as t increases.

Now, let's rewrite this expectation in terms of sampling.

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\pi_{\theta'}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})}{\pi_{\theta}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)}$$

where $\pi_{\theta}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$ denotes the probability of the state-action pair $(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$ occurring under the policy π_{θ} , over the entire state-action space. This is known as a **state-action marginal distribution**, and is essentially the same expression as the previous importance sampling ratio product in expectation.

Subsequently, we can expand the state-action marginal

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\cancel{\pi_{\theta'}(\mathbf{s}_t^{(i)})}}{\cancel{\pi_{\theta}(\mathbf{s}_t^{(i)})}} \cdot \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)}$$

and then essentially ignore the relative probabilities of state $\mathbf{s}_t^{(i)}$ occurring under the different policies because we assume that θ' is very similar to θ , and therefore the state distribution will have shifted very little. In other words, we ignore the importance sampling ratios of the previous steps taken on the trajectory to arrive at this state because we assume the probability of arriving at this state, in expectation, should not change much between policies. Therefore, we only care about the importance sampling ratio of the action probabilities for the current state. We will explore this detail more precisely next lecture. Thus,

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)}$$

Now, finally, we can reconstruct our multi-step policy gradient algorithm with importance sampling.

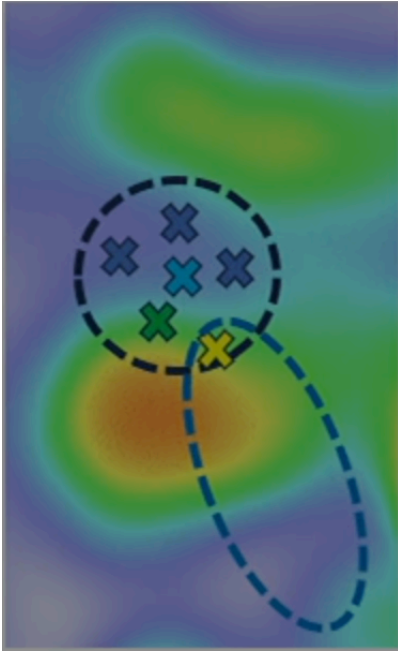
1. Sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a} | \mathbf{s})$ (run policy).
2. Evaluate $y_i = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$.
3. Refit $\hat{V}_{\phi}^{\pi}$ to targets $\{y_i^{(i)}\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}^{(i)}$.
5. $\theta' \leftarrow \theta$
6. $\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)}$.
7. $\theta' \leftarrow \theta' + \alpha \nabla_{\theta'} J(\theta')$.
6. $\theta \leftarrow \theta'$.

where the inner loop is repeated some K times.

Practical Importance Sampling

Clipped Importance Weights

One issue is that, once the importance weights $\frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}$ stray too far from 1.0, the estimator is encouraged to over-prioritize good samples and over-penalize bad samples, causing **high variance**. The below figure shows what might happen.



We can fix this by **clipping** the importance weights. Let $w(\tau) = \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)}$. We clip by defining

$$w(\tau) = \max \left\{ 1 - \epsilon, \min \left\{ 1 + \epsilon, \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \right\} \right\}$$

in other words, $\|1 - \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)}\| \leq \epsilon$, which prevents the algorithm from weighting good samples too heavily.

There's one more detail to consider though. Consider a good sample with $w(\tau_g) < 1 + \epsilon$, and a bad sample with $w(\tau_b) \geq 1 + \epsilon$, such that the two samples are similar enough that increasing $w(\tau_g)$ will increase $w(\tau_b)$, and vice versa. Notably, the estimator is only ever encouraged to increase $w(\tau)$ (and thus increase the probability of choosing τ under the new policy θ') for these samples because $w(\tau_b)$ will stay constant, but $w(\tau_g)$ will increase, based on the clipping. However, this may result in the algorithm making a change that substantially increases the probability of τ_b under the new policy for only marginal improvements in the probability of τ_g , because in the perspective of the algorithm with importance weight clipping, this is always an improvement. Thus, the clipping is modified a little bit to be

$$\begin{aligned} J(\theta') &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\min\{w(\tau)r(\tau), w_c(\tau)r(\tau)\}] \\ w(\tau) &= \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \\ w_c(\tau) &= \max \left\{ 1 - \epsilon, \min \left\{ 1 + \epsilon, \frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \right\} \right\} \end{aligned}$$

where, essentially, $w(\tau)$ is clipped for good samples, but is not clipped for bad samples. In other words, the bad samples can become arbitrarily bad, but the good samples are always bounded by how good they can be.

Proximal Policy Optimization

Subsequently, we can now implement **proximal policy optimization (PPO)**, or policy gradient with clipped importance sampling.

1. Sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a} \mid \mathbf{s})$ (run policy).
2. Evaluate $y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$.
3. Refit $\hat{V}_{\phi}^{\pi}$ to targets $\{y_t^{(i)}\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}^{(i)}$.
5. $\theta' \leftarrow \theta$.

1. $\nabla_{\theta'} J(\theta') \approx \nabla_{\theta'} \mathcal{L}_{\text{CLIP}}(\theta')$

$$2. \theta' \leftarrow \theta' + \alpha \nabla_{\theta'} J(\theta').$$

$$6. \theta \leftarrow \theta'.$$

where we define

$$\mathcal{L}_{\text{CLIP}}(\theta') = \sum_{i=1}^N \sum_{t=1}^H \min \left\{ \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})} \hat{A}_t^{(i)}, \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} \mid \mathbf{s}_t^{(i)})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)} \right\}$$

Other Practical Details

- Use GAE for $y_t^{(i)}$.
- Add $\sum_i \sum_t \mathcal{H}(\pi_{\theta'}(\cdot \mid \mathbf{s}_t^{(i)}))$ to the objective for **entropy regularization** that encourages exploration (because the agent now tries to maximize entropy as well).

Lecture 10: Advanced Policy Gradients: Part 2

Mathematical Shortcuts So Far

When rewriting the importance sampling policy gradient formula with causality, we (1) removed the B term, i.e.

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \underbrace{\left(\prod_{t'=1}^t \frac{\pi_{\theta'}(\mathbf{a}_{t'} | \mathbf{s}_{t'})}{\pi_{\theta}(\mathbf{a}_{t'} | \mathbf{s}_{t'})} \right)}_A \left(\sum_{t'=t}^H r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \underbrace{\left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(\mathbf{a}_{t''} | \mathbf{s}_{t''})}{\pi_{\theta}(\mathbf{a}_{t''} | \mathbf{s}_{t''})} \right)}_B \right) \right]$$

Which corresponds to essentially using the advantage with respect to the policy π_{θ} , $\hat{A}^{\pi_{\theta}}$, instead of the advantage with respect to the policy we're improving, $\hat{A}^{\pi_{\theta'}}$. That is, leaving in the B term uses importance sampling to correct the advantage according to the importance sampling ratio between the two distributions; removing it means using the advantage without adjusting for the difference in distribution.

We also replaced term A with a *state-action marginal*, which is equivalent under expectation,

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \frac{\cancel{\pi_{\theta'}(\mathbf{s}_t^{(i)})}}{\pi_{\theta}(\mathbf{s}_t^{(i)})} \cdot \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{A}_t^{(i)}$$

and (2) ignored the mismatch between the state distributions of the two policies.

In short, there are two shortcuts:

1. Incorrect advantage
2. Incorrect state distribution

Policy Iteration → Policy Gradient (Incorrect Advantage)

Policy gradient, notably, may be viewed as policy iteration.

1. Estimate $\hat{A}^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for current policy π .
2. Use $\hat{A}^{\pi}$ to produce an improved policy π' .

We can formalize this by using an alternate objective

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right]$$

where we define

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

we will attempt to prove this claim of equality.

🔍 Why do this?

Note that calculating the advantage A^{π_θ} is easy—we use the estimator $\hat{A}$ we've constructed—while calculating the advantage $A^{\pi_{\theta'}}$ is hard, since we lack an estimator for the new policy.

Proof.

$$J(\theta') - J(\theta) = J(\theta') - \mathbb{E}_{\mathbf{s}_0 \sim p(\mathbf{s}_0)} [V^{\pi_\theta}(\mathbf{s}_0)] \quad (1)$$

$$= J(\theta') - \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [V^{\pi_\theta}(\mathbf{s}_0)] \quad (2)$$

$$= J(\theta') - \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_\theta}(\mathbf{s}_t) \right] \quad (3)$$

$$= J(\theta') + \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)) \right] \quad (4)$$

$$= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right] + \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)) \right] \quad (5)$$

$$= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_\theta}(\mathbf{s}_{t+1}) - V^{\pi_\theta}(\mathbf{s}_t)) \right] \quad (6)$$

$$= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \quad (7)$$

Reasoning for each step:

1. Substitution of given expression.
2. The initial state distribution is the same for any policy.
3. Equivalent expression.
4. Equivalent expression (flips sign of second term, exploits ∞).
5. Definition of $J(\theta')$.
6. Combining expectations (τ sampled from same distribution).
7. Definition of advantage function A^{π_θ} .

So what does this really mean in the context of policy iteration? With the objective $J(\theta') - J(\theta)$, we are maximizing the expectation of our returns under policy $\pi_{\theta'}$, formulated via the estimated advantage, according to the advantage function of π_θ , of the discounted trajectories sampled from $\pi_{\theta'}$.

Now, we note the following expansion.

$$\begin{aligned} \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] &= \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} [\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)} [\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t)]] \\ &= \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \end{aligned}$$

where the inner expectation of sampling actions was changed to sample from the $\pi_{\theta'}$ distribution to the π_θ distribution, with the added correction of the importance sampling factor $\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)}$. Thus, we've proved that using the advantage function of θ , not θ' , is okay!

Incorrect State Distribution

We'd like for

$$\sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \approx \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

because we'd like to sample from our θ distribution, and not have to generate new samples from our θ' distribution. We will denote the RHS as $\bar{A}(\theta')$, and we will note that *equality* is generally not true unless $p_\theta = p_{\theta'}$; hence we want to show when exactly we may allow this approximation.

Claim: $p_\theta(\mathbf{s}_t) \approx p_{\theta'}(\mathbf{s}_t)$ when $\pi_\theta \approx \pi_{\theta'}$, for some notion of "closeness" or $\approx$.

Naturally, such a claim seems obvious. We will proceed to formalize this notion, though

Proof.

Case 1: π_θ is deterministic.

Let $\mathbf{a}_t = \pi_\theta(\mathbf{s}_t)$. Let $\pi_{\theta'} \approx \pi_\theta$ if $\pi_{\theta'}(\mathbf{a}_t \neq \pi_\theta(\mathbf{s}_t) \mid \mathbf{s}_t) \leq \epsilon$ for some small ϵ . Actually, we can apply techniques from [Lecture 2](#) to this proof; in particular, the proof about distributional shift for behavioral cloning. In fact, the rest of the proof is precisely identical! This allows us to conclude that $D_{\text{TV}}(p_{\theta'}, p_\theta) = \frac{1}{2} \sum_{\mathbf{s}_t} |p_{\theta'}(\mathbf{s}_t) - p_\theta(\mathbf{s}_t)| \leq \epsilon t$.

This isn't quite a great bound yet, though, because the total variation divergence between the two distributions is linear in the horizon length.

Case 2: π_θ is an arbitrary distribution (may be stochastic).

This is the more general case. Let $\pi_{\theta'} \approx \pi_\theta$ if $D_{\text{TV}}(\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t), \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)) \leq \epsilon$. We first consider a useful lemma:

■ If $D_{\text{TV}}(p_X(x), p_Y(y)) \leq \epsilon$, there exists $p(x, y)$ such that $p(x) = p_X(x)$, $p(y) = p_Y(y)$, and $p(x = y) = 1 - \epsilon$.

In other words, $p_X(x)$ "agrees" with $p_Y(y)$ with probability ϵ . In the context of our proof, $\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)$ takes a different action than $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ with probability at most ϵ . (In practice, we enforce this constraint).

This lemma actually allows us to do the exact same thing as before, i.e. show that $D_{\text{TV}}(p_{\theta'}, p_\theta) \leq \epsilon t$.

Finally, we'd like to relate the closeness between π 's to the (bounds on) closeness between p 's. We note that

$$\begin{aligned} \mathbb{E}_{p_{\theta'}(\mathbf{s}_t)}[f(\mathbf{s}_t)] &= \sum_{\mathbf{s}_t} p_{\theta'}(\mathbf{s}_t) f(\mathbf{s}_t) \\ &= \sum_{\mathbf{s}_t} p_{\theta'}(\mathbf{s}_t) f(\mathbf{s}_t) + (p_\theta(\mathbf{s}_t) - p_{\theta'}(\mathbf{s}_t)) f(\mathbf{s}_t) \\ &= \sum_{\mathbf{s}_t} p_\theta(\mathbf{s}_t) f(\mathbf{s}_t) - (p_\theta(\mathbf{s}_t) - p_{\theta'}(\mathbf{s}_t)) f(\mathbf{s}_t) \\ &\geq \sum_{\mathbf{s}_t} p_\theta(\mathbf{s}_t) f(\mathbf{s}_t) - |p_\theta(\mathbf{s}_t) - p_{\theta'}(\mathbf{s}_t)| f(\mathbf{s}_t) \\ &\geq \sum_{\mathbf{s}_t} p_\theta(\mathbf{s}_t) f(\mathbf{s}_t) - |p_\theta(\mathbf{s}_t) - p_{\theta'}(\mathbf{s}_t)| \max_{\mathbf{s}_t} f(\mathbf{s}_t) \\ &\geq \mathbb{E}_{p_\theta(\mathbf{s}_t)}[f(\mathbf{s}_t)] - 2\epsilon t \max_{\mathbf{s}_t} f(\mathbf{s}_t) \end{aligned}$$

pay close attention to the θ and θ' ... they switch around a few times in the above sequence of equalities/inequalities. Note also that the last step is simply the application of the bound on the total variation divergence D_{TV} .

Now, consider defining f as

$$f = \mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right]$$

or simply what's inside the brackets of the outer expected value for the expression we derived for $J(\theta') - J(\theta)$,

$$\sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

using our recently derived bound, we may note that

$$\sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \geq \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] - \sum_t 2\epsilon t C$$

where $C = \max_{\mathbf{s}_t} f(\mathbf{s}_t)$. Note the change from $p_{\theta'}$ to p_{θ} in the outer expected value, i.e. sampling states from the distribution produced by π_{θ} instead.

We'd like to find an upper bound, however, for the additional error $\sum 2\epsilon t C$. We can note that $A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_{\theta}}(\mathbf{s}_{t+1}) - V^{\pi_{\theta}}(\mathbf{s}_t)$. We can infer that $\gamma V^{\pi_{\theta}}(\mathbf{s}_{t+1}) - V^{\pi_{\theta}}(\mathbf{s}_t)$ is likely pretty small! Therefore, $r(\mathbf{s}_t, \mathbf{a}_t)$ is the largest contributor to $A^{\pi_{\theta}}$, and we can therefore conclude that we can reasonably bound $A^{\pi_{\theta}} \in O(r_{\max})$. Subsequently, we can bound $C \in O(H r_{\max})$, where H is the horizon, due to the need to sum across the entire horizon of time steps. Furthermore, with discounting, we produce a geometric series that implies a bound of $O\left(\frac{r_{\max}}{1-\gamma}\right)$. Thus, we can bound $\sum_t 2\epsilon t C \in O\left(\epsilon H^2 \frac{r_{\max}}{1-\gamma}\right)$. (Note the similarity to the bound derived in [Lecture 2](#)).

Notably, this is not a great bound at all! However, it does imply that, as ϵ shrinks, the error will become sufficiently small. In conclusion, it's okay to ignore the state distribution divergence provided we do not change the policy too much :)

Policy Gradient with Constraints

We'd like to limit the divergence between the two policies, π_{θ} and $\pi_{\theta'}$, in our policy gradient algorithm.

We have a TVD upper bound on our state marginal difference. However, TVD, unfortunately, is inconvenient to work with due to its use of absolute values (which don't work well with logs and gradients). However, we can actually develop a bound on TVD in terms of KL divergence.

$$D_{\text{TV}}(\pi_{\theta'}, \pi_{\theta}) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \parallel \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t))}$$

where $D_{\text{KL}}(p_1(x) \parallel p_2(x)) = \mathbb{E}_{x \sim p_1(x)} \left[\log \frac{p_1(x)}{p_2(x)} \right] \approx \sum_i \log p_1(x_i) - \log p_2(x_i)$. KL divergence has some very convenient properties that makes is easier to approximate. For our purposes,

$$D_{\text{KL}} \approx \sum_i \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) - \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$$

and minimizing D_{KL} w.r.t θ' gives

$$\min_{\theta'} D_{\text{KL}} = \sum_i \log \cancel{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} - \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$$

where we can ignore the first term because it is not dependent on θ' . Thus, minimizing D_{KL} is the exact same thing as maximizing the likelihood of samples from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (precisely like supervised learning).

Therefore, we can enforce this constraint by turning our constrained optimization problem into an unconstrained optimization problem. We write the Lagrangian

$$\mathcal{L}(\theta', \beta) = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] - \beta (D_{\text{KL}}(\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \parallel \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)) - \epsilon)$$

and repeat the following steps.

1. Maximize $\mathcal{L}(\theta', \beta)$ with respect to θ' .
2. $\beta \leftarrow \beta + \alpha (D_{\text{KL}}(\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \parallel \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)) - \epsilon)$. (Intuitively, just adjust β depending on if the loss function is over- or under-constrained).

This is actually a form of **dual gradient descent**, which describes a general class of techniques to solve constrained optimization problems by alternating between (1) minimizing the Lagrangian over primal variables (θ') via gradient descent and (2) taking a gradient ascent step on the dual variables (β). Also, for practical purposes, it's not necessary to wait for the first step to converge.

Rewriting in terms of our samples,

$$\mathcal{L}_{\text{KL}}(\theta', \beta) \approx \underbrace{\sum_{i=1}^N \sum_{t=1}^H \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}{\pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} A_t^{(i)}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})}_{\text{importance sampled surrogate objective}} + \underbrace{\beta \log \pi_{\theta'}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})}_{\text{penalty for moving away from samples of } \pi_{\theta}} + \text{const}$$

And here's how the algorithm for PPO with KL divergence would look like.

1. Sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a} | \mathbf{s})$ (run policy).
2. Evaluate $y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$.
3. Refit $\hat{V}_{\phi}^{\pi}$ to targets $\{y_t^{(i)}\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma \lambda)^{t'-t} \delta_{t'}^{(i)}$.
5. $\theta' \leftarrow \theta$.
 1. $\nabla_{\theta'} J(\theta') \approx \nabla_{\theta'} \mathcal{L}_{\text{KL}}(\theta')$
 2. $\theta' \leftarrow \theta' + \alpha \nabla_{\theta'} J(\theta')$.
6. $\beta \leftarrow \beta + \alpha (D_{\text{KL}} - \varepsilon)$
7. $\theta \leftarrow \theta'$.

Note the extra step to perform gradient ascent on β .

Natural Policy Gradient

Note: we will let

$$\bar{A}(\theta') = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

Let's try replacing our objective $\bar{A}(\theta')$ with a *first-order Taylor approximation*, i.e.

$$\theta' \leftarrow \arg \max_{\theta'} \nabla_{\theta} \bar{A}(\theta)^{\top} (\theta' - \theta)$$

subject to the same KL divergence constraint. In essence, we are *linearizing* our objective. We note that

$$\nabla_{\theta'} \bar{A}(\theta') = \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

but with θ ,

$$\begin{aligned} \nabla_{\theta} \bar{A}(\theta) &= \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\cancel{\frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}} A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \\ &= \sum_t \mathbb{E}_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[\mathbb{E}_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} [A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t)] \right] \end{aligned}$$

since the importance sampling factor doesn't matter anymore when we're calculating w.r.t θ instead of θ' . This is just normal policy-gradient; so, to perform our first-order approximation, we can just substitute this expression in and then multiply by $\theta' - \theta$ and then maximize over θ' , i.e.

$$\theta' \leftarrow \arg \max_{\theta'} \nabla_{\theta} J(\theta)^{\top} (\theta' - \theta)$$

where $J(\theta)$ represents the normal policy gradient objective.

Can we just set $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$, then, for some constant α ? Sadly, no. Even if we choose $\alpha = \sqrt{\frac{\varepsilon}{\|\nabla_{\theta} J(\theta)\|^2}}$, which guarantees $\|\theta - \theta'\|^2 \leq \varepsilon$, this is not the same as our KL divergence constraint! This is because some parameters changes probabilities a lot more than others.

However, we can instead use a *second-order Taylor approximation* of the KL divergence, which comes out to

$$D_{\text{KL}}(\pi_{\theta'} \parallel \pi_{\theta}) \approx \frac{1}{2} (\theta' - \theta)^{\top} \mathbf{F} (\theta' - \theta)$$

where $\mathbf{F}$ is a [Fisher information matrix](#) and is defined

$$\mathbf{F} = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(\mathbf{a} \mid \mathbf{s}) \nabla_{\theta} \log \pi_{\theta}(\mathbf{a} \mid \mathbf{s})^{\top}]$$

And now, with a first-order approximation of our objective, and a second-order approximation of our constraints, this becomes a (tractable) quadratic programming problem.



Tip

The comparison between the constraint $\|\theta - \theta'\|^2 \leq \varepsilon$ and the second-order approximation of the KL divergence may be likened to a sphere vs an ellipsoid in hyperdimensional space, where the latter essentially adjusts according to the "strength" or "impact" of a parameter on the divergence.

With this, we can define our gradient ascent to be

$$\begin{aligned} \theta' &\leftarrow \theta + \alpha \mathbf{F}^{-1} \nabla_{\theta} J(\theta) \\ \alpha &= \sqrt{\frac{2\varepsilon}{\nabla_{\theta} J(\theta)^{\top} \mathbf{F} \nabla_{\theta} J(\theta)}} \end{aligned}$$

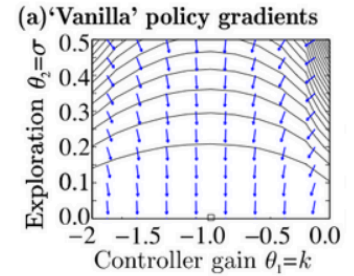
This is known as the [natural gradient](#).

But what's the point of the natural gradient? The idea is that naive gradient ascent experiences problems due to poorly-conditioned functions (see [this section from Goodfellow's Deep Learning](#)). See the slide below.

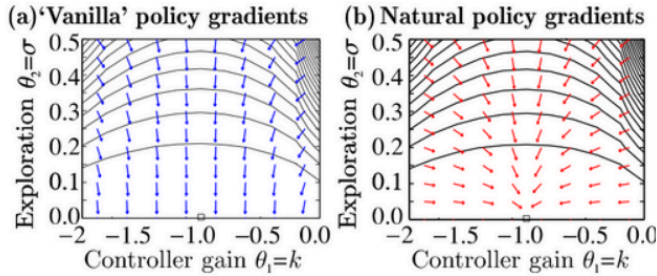


$$r(\mathbf{s}_t, \mathbf{a}_t) = -\mathbf{s}_t^2 - \mathbf{a}_t^2$$

$$\log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) = -\frac{1}{2\sigma^2} (k\mathbf{s}_t - \mathbf{a}_t)^2 + \text{const} \quad \theta = (k, \sigma)$$

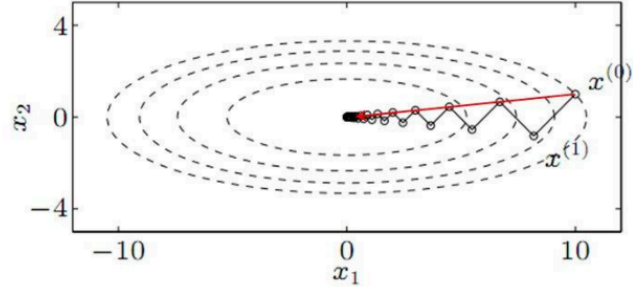


(image from Peters & Schaal 2008)



(figure from Peters & Schaal 2008)

Essentially the same problem as this:



Trust Region Policy Optimization

TRPO is essentially just natural policy gradient.

1. Sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a} | \mathbf{s})$ (run policy).
2. Evaluate $y_t^{(i)} = r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+1}^{(i)})$.
3. Refit $\hat{V}_{\phi}^{\pi}$ to targets $\{y_t^{(i)}\}$, minimizing $\mathcal{L}(\phi)$.
4. Evaluate $\hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}^{(i)}$.
5. $\theta' \leftarrow \theta + \alpha \mathbf{F}^{-1} \nabla_{\theta} J(\theta)$.

where $\mathbf{F} \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \nabla_{\theta} \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})^{\top}$.

Warning

In practice, building the full matrix $\mathbf{F}$ and calculating its inverse is far too expensive (quadratic in the number of parameters). Instead, **conjugate gradient** is typically used. (See [here](#)). The TL;DR idea is that you'll only ever use $\mathbf{F}^{-1}$ to perform matrix-vector multiplication.

Conclusion

- Policy gradient is more productively interpreted as a form of policy iteration.
- PPO with clipping is a heuristic approximation of PPO with KL.
- There are other ways to enforce the constraints
 - Natural gradient!
- TRPO vs PPO? Usually PPO today due to ease of implementation.

Note that TRPO/natural gradient was the *precursor* to PPO with KL divergence, i.e. PPO is a refinement that offers lower implementation complexity and lower computational complexity.

Lecture 11: Variational Inference

"Variational inference is an optimization-based framework for training models with latent variables."

Latent Variable Models

Latent variables are learned as **mixture elements** that turn probabilistic models into latent variable models whose distribution is represented as a mixture of multiple distributions, i.e.

$$p(x) = \sum_z p(x | z)p(z)$$
$$p(y | x) = \sum_z p(y | x, z)p(z)$$

One possible implementation of a continuous latent variable model $p(x) = \int p(x | z)p(z) dz$ is the nonlinear composition of two "easy" distributions, e.g.

$$p(x | z) = \mathcal{N}(\mu_{nn(z)}, \alpha_{nn(z)})$$
$$p(z) = \mathcal{N}(0, 1)$$

where the nn subscript implies a neural network. This is nice because it's easy to use our "easy" distributions.

Sidenote: this is applicable in RL for

- Conditional latent variable models for multi-modal policies
- Latent variable models for model-based RL (where you learn the MDP of the environment itself, more on this later in the course)

So how do we train latent variable models? Well, using maximum likelihood gives us

$$p(x) = \int p(x | z)p(z) dz$$
$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \log \left(\int p(x | z)p(z) dz \right)$$

This is intractable though—the maximum likelihood solution is difficult to compute because of the integral! Moreover, the integral can't be approximated with sampling because it lies inside a log, and therefore **linearity of expectation no longer applies** D:

(Had we tried to approximate with Monte Carlo sampling, the estimator would be **biased**).

Instead, we can use **expected log-likelihood**.

$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \mathbb{E}_{z \sim p(z|x_i)} [\log p_{\theta}(x_i, z)]$$

in essence, we "guess" the most likely value of the latent variable z for x_i by sampling from a **posterior** $p(z | x_i)$ that we construct, and then pretend our guess is correct. But, we still need to compute this posterior.

This problem of calculating $p(z | x_i)$ is known as **probabilistic inference**.

Sidenote: probabilistic inference is also seen in RL for

- Modeling human behavior

- Generative models + variational inference for exploration
- Reinforcement learning from human feedback (RLHF) (predict reward based on model output)

Variational Inference

Performing probabilistic inference exactly is extremely hard; however, for practical purposes, there are *approximate* methods that work well.

First, let's pretend our posterior is extremely simple. This assumption is *incorrect*, but very convenient! (We'll see soon how we may quantify the extent to which this assumption is wrong). For instance, we can approximate $p(z | x_i) \approx q_i = \mathcal{N}(\mu_i, \sigma_i)$. Critically, this approximation allows us to tractably produce a *lower bound* on $\log p(x_i)$.

We note that

$$\begin{aligned}\log p(x_i) &= \log \int p(x_i | z)p(z) dz \\ &= \log \int p(x_i | z)p(z) \frac{q_i(z)}{q_i(z)} dz \\ &= \log \mathbb{E}_{z \sim q_i(z)} \left[\frac{p(x_i | z)p(z)}{q_i(z)} \right]\end{aligned}$$

(Really, this is just an application of Bayes' rule).

We can then apply what is known as *Jensen's inequality*, which states that

$$f(\mathbb{E}[y]) \geq \mathbb{E}[f(y)]$$

for any *concave function* f . Notably, $\log$ is a concave function. Using this, we may conclude

$$\begin{aligned}\log p(x_i) &= \log \mathbb{E}_{z \sim q_i(z)} \left[\frac{p(x_i | z)p(z)}{q_i(z)} \right] \\ &\geq \mathbb{E}_{z \sim q_i(z)} \left[\log \frac{p(x_i | z)p(z)}{q_i(z)} \right] \\ &\geq \mathbb{E}_{z \sim q_i(z)} [\log p(x_i | z) + \log p(z)] - \mathbb{E}_{z \sim q_i(z)} [\log q_i(z)] \\ &\geq \mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] + \mathcal{H}(q_i)\end{aligned}$$

where $\mathcal{H}$ is the *entropy* of the distribution. Notably, $\mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)]$ is precisely our expected log-likelihood term!

Note that the definition of entropy is

$$\mathcal{H}(p) = -\mathbb{E}_{x \sim p(x)} [\log p(x)] = -\int p(x) \log p(x) dx$$

Intuitively, entropy describes...

- how random the random variable is?
- how large is the log probability in expectation under itself?

So what exactly do the two terms in the expression $\mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] + \mathcal{H}(q_i)$ do?

1. The first term, our expected log-likelihood, induces the model to maximize probability at the *peaks* of the distribution.
2. The second term, entropy, induces the model to distribute its probability over a greater area, i.e. it encourages increased entropy in the model.

Now, note the following representation of *KL divergence*.

$$\begin{aligned}
D_{\text{KL}}(q \parallel p) &= \mathbb{E}_{x \sim q(x)} \left[\log \frac{q(x)}{p(x)} \right] \\
&= -\mathbb{E}_{x \sim q(x)} [\log p(x)] - \mathcal{H}(q)
\end{aligned}$$

that looks suspiciously similar to our expression above...

Intuitively, KL divergence describes...

- how different are the two distributions?
- how small is the expected log probability of one distribution under another, minus entropy?

Thus, our expression $\mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] + \mathcal{H}(q_i)$ is really very similar to KL divergence, and is a measure of the divergence between the distributions $\log p(x_i, z)$ and $q_i(x)$. Notably, maximizing this expression, and therefore $\log p(x)$, is equivalent to minimizing the KL divergence, or minimizing the distance between the two distributions.

Let's now define $\mathcal{L}_i(p, q_i) = \mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] + \mathcal{H}(q_i)$. We previously alluded to how we may quantify how incorrect our initial assumption was. So, how do we measure how good a choice of q_i is? The KL divergence $D_{\text{KL}}(q_i(z) \parallel p(z \mid x_i))$. Consider that

$$\begin{aligned}
D_{\text{KL}}(q_i(z) \parallel p(z \mid x_i)) &= \mathbb{E}_{z \sim q_i(z)} \left[\log \frac{q_i(z)}{p(z \mid x_i)} \right] \\
&= \mathbb{E}_{z \sim q_i(z)} \left[\log \frac{q_i(z)p(x_i)}{p(x_i, z)} \right] \\
&= -\mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] + \mathbb{E}_{z \sim q_i(z)} [\log q_i(z)] + \mathbb{E}_{z \sim q_i(z)} [\log p(x_i)] \\
&= -\mathbb{E}_{z \sim q_i(z)} [\log p(x_i, z)] - \mathcal{H}(q_i) + \log p(x_i) \\
&= -\mathcal{L}_i(p, q_i) + \log p(x_i)
\end{aligned}$$

and therefore

$$\log p(x_i) = D_{\text{KL}}(q_i(z) \parallel p(z \mid x_i)) + \mathcal{L}_i(p, q_i)$$

In other words, q_i produces a lower bound $\mathcal{L}_i(p, q_i) \leq \log p(x_i)$ where the error (distance between lower bound and the actual value of $\log p(x_i)$) is precisely the KL divergence between $q_i(z)$ and the distribution it attempts to approximate, $p(z \mid x_i)$.

Now, we may note that, in the expression for the KL divergence,

$$D_{\text{KL}}(q_i(z) \parallel p(z \mid x_i)) = -\mathcal{L}_i(p, q_i) + \log p(x_i)$$

$\log p(x_i)$ is independent of q_i . Therefore, if we maximize $\mathcal{L}_i(p, q_i)$ with respect to q_i , we minimize KL divergence! Thus,

- Maximizing $\mathcal{L}$ with respect to p increases the log-likelihood
- Maximizing $\mathcal{L}$ with respect to q_i tightens the lower bound

Thus, we can actually select q_i intelligently by iteratively maximizing $\mathcal{L}_i$ w.r.t q_i .

Here's the final idea we derive from the above deductions.

1. For each x_i or mini-batch:
 1. Sample $z \sim q_i(z)$ and approximating $\nabla_{\theta} \mathcal{L}_i(p, q_i) \approx \nabla_{\theta} \log p_{\theta}(x_i \mid z)$.
 2. $\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}_i(p, q_i)$.
3. Update q_i to maximize $\mathcal{L}_i(p, q_i)$.

Where the optimization step w.r.t q_i may be done with gradient ascent or another optimization method. For instance, assuming $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$, we use the gradients $\nabla_{\mu_i} \mathcal{L}_i(p, q_i)$ and $\nabla_{\sigma_i} \mathcal{L}_i(p, q_i)$ to perform gradient ascent on μ_i and σ_i .

There's just one problem... **there are way too many parameters!** In fact, it scales with the size of the dataset, i.e. $|\theta| + (|\mu_i| + |\sigma_i|) \times N$. We'll address this issue next lecture; in particular, how deep learning makes this tractable :)



Tip

The Expectation-Maximization algorithm is actually a special case of variational inference! The E step corresponds to updating q_i , and the M step corresponds to updating θ .



Tip

The lower bound we derived is known as the **Evidence Lower Bound** or **ELBO**.

Lecture 12: Variational Inference in RL

Amortized Variational Inference

The problem with variational inference, as mentioned at the end of last lecture, is that there are far too many parameters to tractably train. So, how can we resolve this?

Recall that $q_i(z) \approx p(z | x_i)$. A naive solution might use only one unified q function for all data points, rather than a q_i function for each individual data point. This produces a valid lower bound, but... this is probably a *terrible* lower bound since it's highly unlikely every data point has the same posterior (and if so, your dataset probably isn't good anyways!).

A better solution is to represent all q_i with a *function* q (neural network) such that $q_i(z) = q(z | x_i) \approx p(z | x_i)$, i.e. it produces a distribution over z for each distinct x_i . For instance, we could use a function $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, where the neural network's parameters are ϕ and μ_ϕ, σ_ϕ are functions that output the posterior's mean and variance given a data point x as input.

Notably, our update step for q doesn't really change; now, we simply update ϕ to maximize $\mathcal{L}_i(p, q)$ when training on data point x_i .

This is known as *amortized* variational inference, because the difficulty of computing the posterior distribution is "amortized" or distributed across all the data points (rather than determining a posterior for each data point). Our algorithm thus becomes

1. For each x_i /mini-batch

1. Sample $z \sim q_\phi(z | x_i)$ and calculate $\nabla_\theta \mathcal{L}(p_\theta(x_i | z), q_\phi(x_i | z)) \approx \nabla_\theta \log p_\theta(x_i | z)$.
2. $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$.
3. $\phi \leftarrow \phi + \alpha \nabla_\phi \mathcal{L}$.

where the loss function for data point x_i is

$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z | x_i)} [\log p_\theta(x_i | z) + \log p(z)] + \mathcal{H}(q_\phi(z | x_i))$$

Now let's define $r(x_i, z) = \log p_\theta(x_i | z) + \log p(z)$. Then this becomes

$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z | x_i)} [r(x_i, z)] + \mathcal{H}(q_\phi(z | x_i))$$

...hang on, doesn't this look like the expected reward function for an RL problem now? That is, $J(\phi) = \mathbb{E}_{z \sim q_\phi(z | x_i)} [r(x_i, z)]$ is precisely an expected reward function for a "policy" q_ϕ . This means we can apply RL methods for optimization—in particular, we can just use policy gradient!

Applying policy gradient, we can derive a way to determine $\nabla_\phi J(\phi)$ with samples.

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi \log q_\phi(z_j | x_i) r(x_i, z_j)$$

and this leads to

$$\nabla_\phi \mathcal{L}_i = \nabla_\phi J(\phi) + \nabla_\phi \mathcal{H}(q_\phi(z | x_i))$$

so we can now compute our gradient on $\mathcal{L}_i$ w.r.t ϕ via sampling! (Note that computing $\nabla_\phi \mathcal{H}$ is well-known).

But, this gradient has one problem—it's a very *high variance* gradient. Can we do better?

Reparametrization Trick

Consider that, if $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$, we can express samples of z as simply $z = \mu_\phi(x) + \epsilon\sigma_\phi(x)$ where $\epsilon \sim \mathcal{N}(0, 1)$. Notably, this expression is independent of ϕ . This allows us to then rewrite

$$J(\phi) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0,1)}[r(x_i, \mu_\phi(x_i) + \epsilon\sigma_\phi(x_i))]$$

In other words, we have reparametrized the expectation under ϕ into an expectation under a constant distribution ϵ . Then, to estimate $\nabla_\phi J(\phi)$, we can sample $\epsilon \sim \mathcal{N}(0, 1)$ instead (usually just one ϵ), and then just perform normal backpropagation because ϵ doesn't depend on ϕ .

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi r(x_i, \mu_\phi(x_i) + \epsilon_j \sigma_\phi(x_i))$$

This results in a much lower variance estimator.

Why is this possible here, but not for normal RL? In our normal policy-gradient, r is unknown; it's a function of environment dynamics that we do not assume knowledge of. However, here, we defined and know the expression for r ; therefore, we can compute the gradient of r , and therefore backpropagate gradients through r .

? Huh? ▾

Me too. I read [this blog](#) (and asked Google/Gemini) to try and understand why. From my limited understanding, it's because we cannot backpropagate through the actual sampling of z from $q_\phi(z)$ (due to the stochasticity of sampling). This is problematic because, in standard variational inference, we are trying to compute a gradient w.r.t ϕ over an expectation over a distribution parameterized by ϕ . We'd like to estimate the expression $\nabla_\phi \mathbb{E}_{q_\phi(z)}[f(z)]$ via Monte Carlo sampling; to do so, we need to push the gradient operator inside the expectation. However, this is mathematically incorrect—the distribution we are taking the expectation over is parametrized by ϕ itself. Thus, a single sample $z \sim q_\phi(z)$ does not actually convey any information about the gradient w.r.t ϕ , because it carries no information about the distribution $q_\phi(z)$. Without reparametrization, we can use the policy gradient/REINFORCE trick to rewrite the expectation $\nabla_\phi \mathbb{E}_{z \sim q_\phi(z)}[f(z)]$ as $\mathbb{E}_{z \sim q_\phi(z)}[f(z) \nabla_\phi \log q_\phi(z)]$; this is what we did previously. However, this suffers from high variance because it ignores the gradient of $f(z)$ itself. The reparametrization trick helps us rewrite the expression, turning a gradient of an expectation into an expectation of a gradient. By defining $z = g_\phi(\epsilon)$, we can write $\nabla_\phi \mathbb{E}_{p(\epsilon)}[f(g_\phi(\epsilon))]$, which allows us to push the gradient operator inside the expectation (since $p(\epsilon)$ is unrelated to ϕ), turning the expression into $\mathbb{E}_{p(\epsilon)}[\nabla_\phi f(g_\phi(\epsilon))]$. This gradient now actually uses the gradient of our relevant loss function f , yielding lower variance and consequently faster convergence.

Implementation

We can turn $\mathcal{L}_i$ into an equivalent, but more useful, expression for implementation.

$$\begin{aligned} \mathcal{L}_i &= \mathbb{E}_{z \sim q_\phi(z|x_i)}[\log p_\theta(x_i | z) + \log p(z)] + \mathcal{H}(q_\phi(z | x_i)) \\ &= \mathbb{E}_{z \sim q_\phi(z|x_i)}[\log p_\theta(x_i | z)] + \mathbb{E}_{z \sim q_\phi(z|x_i)}[\log p(z)] + \mathcal{H}(q_\phi(z | x_i)) \\ &= \mathbb{E}_{z \sim q_\phi(z|x_i)}[\log p_\theta(x_i | z)] - D_{\text{KL}}(q_\phi(z | x_i) \parallel p(z)) \\ &= \mathbb{E}_{\epsilon \sim \mathcal{N}(0,1)}[\log p_\theta(x_i | \mu_\theta(x_i) + \epsilon\sigma_\phi(x_i))] - D_{\text{KL}}(q_\phi(z | x_i) \parallel p(z)) \\ &\approx \log p_\theta(x_i | \mu_\theta(x_i) + \epsilon\sigma_\phi(x_i)) - D_{\text{KL}}(q_\phi(z | x_i) \parallel p(z)) \end{aligned}$$

As $q_\phi(z | x_i)$ and $p(z)$ are both Gaussians, the KL divergence between the two has a nice analytical form. Meanwhile, the gradient of the term on the left may be estimated via Monte Carlo methods, as explained above.

Notably, the resulting architecture is precisely a [variational autoencoder \(VAE\)](#). The encoder network, parametrized by ϕ , takes an input x_i and produces $\mu_\phi(x_i)$ and $\sigma_\phi(x_i)$. With the random sampling of some noise $\epsilon \sim \mathcal{N}(0, 1)$, we produce $z = \mu_\phi + \epsilon\sigma_\phi$. This z is then fed into the decoder network, parameterized by θ , to produce $p_\theta(x_i | z)$. Training is done by backpropagating through the entire computation graph just described for each minibatch of samples. (With the addition of the KL divergence, of course, to the loss function; this may be represented analytically, though, and thus its gradient may be computed without backpropagation).

So what's the *intuition* behind this? This KL divergence term tries to keep the q_ϕ closer to the prior $p(z)$, while the other term tries to ensure the q_ϕ is good enough such that p_θ may reconstruct (decode) the original x_i from the encoded $z = \mu_\phi + \epsilon\sigma_\phi$. With the selection of a simple prior, this usually means that the VAE is looking for the simplest q_ϕ that still does a good job of reconstructing x_i (i.e. encodes a simple representation with only the most important information critical to reconstructing x_i).

Reparametrization Trick vs Policy Gradient

Policy gradient

- Can handle both discrete and continuous latent variables
- High variance

Reparameterization trick

- Only continuous latent variables
- Simple to implement
- Low variance

Generative Models

Variational Autoencoder

First up, the VAE. As aforementioned, it's precisely what we defined above. We have an encoder $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$ and a decoder $p_\theta(x | z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta(z))$. Refer to above for a more detailed explanation on its training.

Note that we may sample (generate x) from the VAE by first sampling $z \sim p(z)$ and then sampling $x \sim p(x | z)$. Notably, we sample z from the *prior* $p(z)$, not the learned posterior $q_\phi(z | x)$, as we are generating x and therefore do not have access to x . However, this is okay because our KL divergence term ensured that $q_\phi(z | x) \approx p(z)$, and therefore the distribution of z 's seen during training are approximately similar to the distribution of z 's seen at test/inference time.

Representation Learning

We can train a VAE to learn a latent variable z that is a *representation* of a state s . Then, during training, rather than train our policy to learn off of states as inputs, we use the latent space of the VAE instead of our state space! That is,

1. Train VAE on states in replay buffer $\mathcal{R}$.
2. Run RL using z as the state instead of s .

This is nice because the latent variables are much simpler than the possible states, and the subspaces of the latent space may represent the underlying factors of variation within the state space.



Training this VAE is also a great way to use already existing data, and will likely induce faster convergence for your RL algorithm!

Conditional Models

We can modify our loss function to be

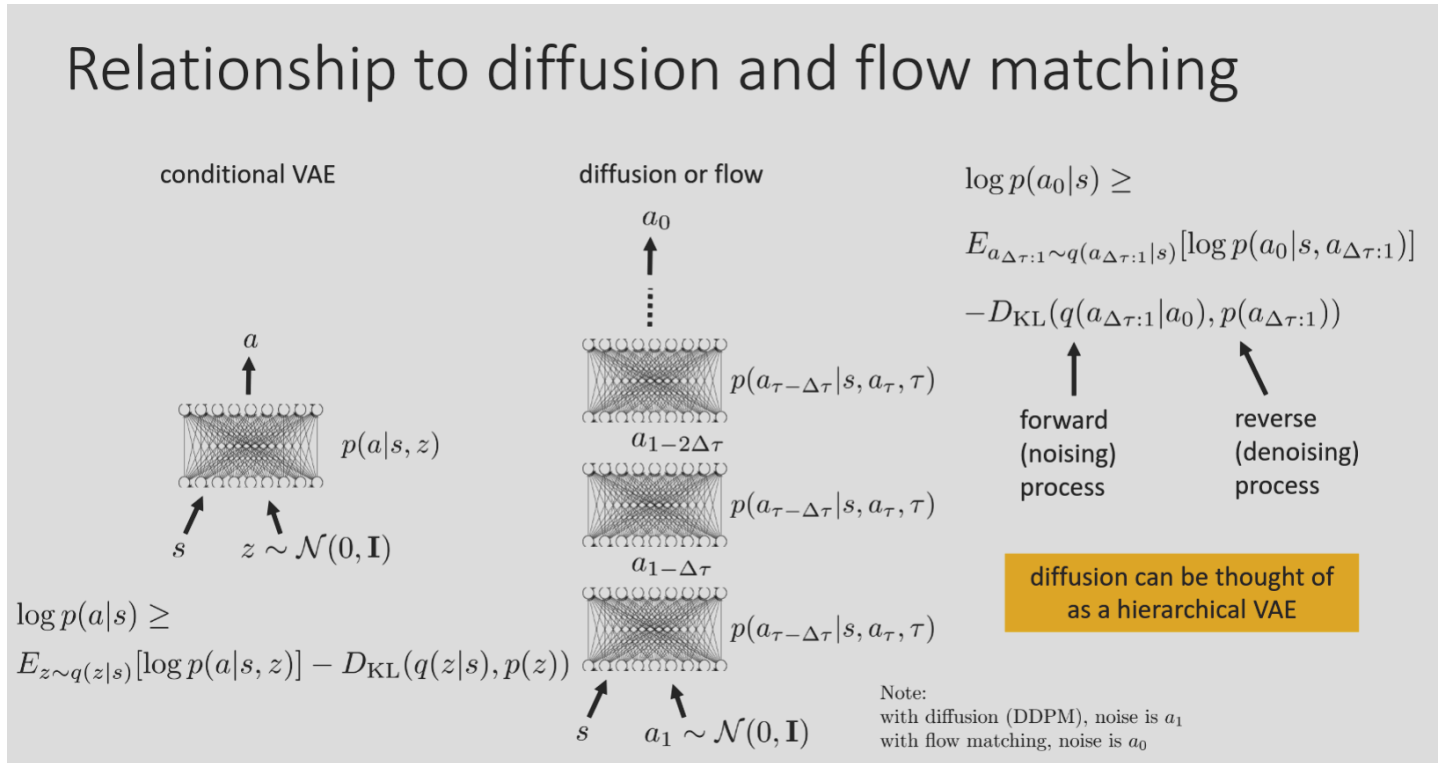
$$\mathcal{L}_i = \mathbb{E}_{z \sim q_\phi(z | x_i, y_i)} [\log p_\theta(y_i | x_i, z) + \log p(z | x_i)] + \mathcal{H}(q_\phi(z | x_i, y_i))$$

this is useful for, say, modeling our policy, i.e. we predict our actions y_i given the state x_i and the latent variable z , which really represents the different modes of the action distribution (peaks).

This may be used for multimodal imitation learning, since z represents the different modes of the action distribution, i.e. z can be interpreted as a variable of a "latent action suggestion" space.

Relationship to Diffusion and Flow Matching

The below slide summarizes the idea.



where diffusion is a step-by-step VAE that uses the noising process as its encoder and the denoising process as its prior.

State Space Models

Again, we'll discuss this more when we talk about model-based RL. In essence, though, with a partially-observed RL problem, we use a VAE that almost acts like a recurrent neural network through time steps, i.e. we learn $p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$. The prior is much more complicated, i.e. $p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$, the decoder is $p_\theta(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$, and the encoder is $q_\phi(\mathbf{z} | \mathbf{o}) = \prod_t q_\phi(\mathbf{z}_t | \mathbf{o}_{1:t})$. Notably, we have much more freedom of choice with regards to the model for the encoder.

Control as Inference

Imagine trying to train optimal control by modeling human behavior. If we assume humans are perfectly rational, and take actions to maximize some implicit reward function, then this works fine! Unfortunately, humans, and organisms in general, do not exhibit perfect rationality, and therefore do not perform optimal actions.

For example, consider a monkey trying to move an object from one place to another. Typically, the monkey won't follow a perfect, straight-line path; there will be some imperfections. But, crucially, it will allow some certain degree of mistakes or approximation of optimality such that it will still achieve the objective. Thus, the monkey exhibits approximate rationality and includes some stochastic behavior, but in places where it doesn't really matter—the key lesson here is that **some mistakes are more important than others!**

Thus, when we're modeling, say, human behavior, we shouldn't model some perfect reward maximization algorithm to produce optimal behavior; instead, we should model how humans act approximately optimal, or model the human "intention" of acting optimal.

We can add a binary variable $\mathcal{O}_t$ at each time step of a trajectory, where $\mathcal{O}_t = 1$ denotes a monkey exhibiting the intent of optimal behavior and $\mathcal{O}_t = 0$ represents the opposite. (In other problems, $\mathcal{O}_t$ may take on a wider range of values). And, we will define $p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$, where $r(\mathbf{s}_t, \mathbf{a}_t) < 0$ so that $p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t)$ is a valid probability distribution. We will see why this is helpful soon.

Now, consider $p(\tau | \mathcal{O}_{1:T})$, which represents the probability of trajectory τ given that the monkey is trying to be optimal throughout the entire trajectory.

$$\begin{aligned} p(\tau | \mathcal{O}_{1:T}) &= \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})} \\ &\propto p(\tau) \prod \exp(r(\mathbf{s}_t, \mathbf{a}_t)) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right) \end{aligned}$$

this seems pretty intuitive! As the reward of a trajectory increases, the probability of it occurring under optimal monkey behavior increases too. And, critically, we can see how this probabilistic model can assign nonzero probabilities to *potentially suboptimal* trajectories that are "close enough" to optimal.

This is important because such a model can help us ignore noise in samples of human or otherwise approximately optimal behavior. It's very applicable to control and planning problems, and, in some sense, can provide an intuitive explanation for the benefits of stochasticity.

Now, what sort of relevant inference problems do we care about?

- Computing **backward messages** $\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$, i.e. "how likely will the monkey continue to be optimal for the rest of the episode given the current state and action?"
- Computing the policy $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$, i.e. "how likely is an action given a state and the assumption that the monkey is optimal over the entire trajectory?"
- Computing **forward messages** $\alpha_t(\mathbf{s}_t) = p(\mathbf{s}_t | \mathcal{O}_{1:t-1})$, i.e. "how likely is state $\mathbf{s}_t$ given the assumption that the monkey was optimal over all previous steps?"

The key idea here is that backward messages will help us compute a policy, while forward messages will help us figure out the state the monkey ends up in.

Backward Messages

Note first that we know $p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) \propto \exp(r(\mathbf{s}_t, \mathbf{a}_t))$. So, we'd like to compute a recursive formula *backwards through time*, using the last time step as our base case.

$$\begin{aligned} \beta_t(\mathbf{s}_t, \mathbf{a}_t) &= p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t) \\ &= \int p(\mathcal{O}_{t:T}, \mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) d\mathbf{s}_{t+1} \\ &= \int p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) d\mathbf{s}_{t+1} \\ p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) &= \int \underbrace{p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}, \mathbf{a}_{t+1}) p(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})}_{\beta_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})} d\mathbf{a}_{t+1} \longrightarrow \beta_t(\mathbf{s}_t) \end{aligned}$$

Note the use of the Markov property in the second step. Also, note that $p(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})$ is a *prior* for our actions, and represents the monkey's actions under zero optimality conditions—this can essentially be assumed to be a uniform distribution and thus ignored, and we will formalize why soon. The above derivation allows us to write

$$\begin{aligned} \beta_t(\mathbf{s}_t, \mathbf{a}_t) &= p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) \mathbb{E}_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})] \\ \beta_t(\mathbf{s}_t) &= \mathbb{E}_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)] \end{aligned}$$

and we can iterate from $t = T - 1$ to $t = 1$ to compute all β_t values.

Now, let's define

$$\begin{aligned} V_t(\mathbf{s}_t) &= \log \beta_t(\mathbf{s}_t) \\ Q_t(\mathbf{s}_t, \mathbf{a}_t) &= \log \beta_t(\mathbf{s}_t, \mathbf{a}_t) \end{aligned}$$

V_t has recursive definition

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

We will call V_t a **soft max**.

Warning

This is *not* the same thing as a *softmax*!!

Why is V_t called a soft max? Because $\int \exp$ is *dominated* by large $Q_t(\mathbf{s}_t, \mathbf{a}_t)$, and thus this is a continuous function that approximates the maximum value of Q_t . Formally,

$$\lim_{Q_t \rightarrow \infty} V_t(\mathbf{s}_t) = \max_{\mathbf{a}_t} Q_t(\mathbf{s}_t, \mathbf{a}_t)$$

Meanwhile, Q_t has the following recursive definition

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log \mathbb{E}[\exp(V_{t+1}(\mathbf{s}_{t+1}))]$$

Deterministic Transitions

If state transitions are deterministic, the recursive formula has the form

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + V_{t+1}(\mathbf{s}_{t+1})$$

which looks exactly like our usual Q -function Bellman update equation, in fact!

Now, with deterministic transitions, if we just throw these functions into *value iteration*, we get essentially the same thing, except now with our "soft max" function replacing the usual "hard max." But, without deterministic transitions, there's now also a "soft max" function over the possible state transitions; in other words, this encourages *excessive optimism*, in which the algorithm favors states that may lead to potentially higher-value states, but may also have fairly high probability of low-value states. (Since it takes the expected value of the soft max over the possible next states!). This is pretty bad... but we'll come back to this in future lectures.

Action Prior

Let's now return to the **action prior** $p(\mathbf{a}_t | \mathbf{s}_t)$. Previously, we just assumed that it was a uniform distribution, but this may not always be the case. (In our monkey example, the monkey might prefer small movement actions because it's lazy). If it isn't uniform, then our recursive definitions change a bit.

$$\begin{aligned} V(\mathbf{s}_t) &= \log \int \exp(Q(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t | \mathbf{s}_t)) d\mathbf{a}_t \\ Q(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \log \mathbb{E}[\exp(V(\mathbf{s}_{t+1}))] \end{aligned}$$

Or, we could simply define a new Q function

$$\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + p(\mathbf{a}_t | \mathbf{s}_t) + \log \mathbb{E}[\exp(V(\mathbf{s}_{t+1}))]$$

and define $V(\mathbf{s}_t)$ as

$$V(\mathbf{s}_t) = \log \int \exp(\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

In other words, we can treat the action prior as simply modifying the reward function to be $\tilde{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + p(\mathbf{a}_t | \mathbf{s}_t)$! And, hopefully, this should intuitively make sense; a monkey that is lazy in his action prior would, in its perspective, be receiving higher rewards for lazier actions. Now, this mathematical basis actually gives us reason to assume the action prior is simply uniform henceforth; even if it isn't, we can simply modify the reward function to express it.

Policy Computation

Now, we can finally compute our policy $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$ using our backward messages.

$$p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}) = \pi(\mathbf{a}_t | \mathbf{s}_t) \quad (1)$$

$$= p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{t:T}) \quad (2)$$

$$= \frac{p(\mathbf{a}_t, \mathbf{s}_t | \mathcal{O}_{t:T})}{p(\mathbf{s}_t | \mathcal{O}_{t:T})} \quad (3)$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t) p(\mathbf{a}_t, \mathbf{s}_t) / p(\mathcal{O}_{t:T})}{p(\mathbf{s}_t, \mathcal{O}_{t:T}) / p(\mathcal{O}_{t:T})} \quad (4)$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t)}{p(\mathcal{O}_{t:T} | \mathbf{s}_t)} \cdot \frac{p(\mathbf{a}_t, \mathbf{s}_t)}{p(\mathbf{s}_t)} \quad (5)$$

where the steps are

1. Definition
2. Due to the Markov property, $\mathbf{s}_t$ captures all that is needed for the previous states; therefore, only the future optimality variables are needed.
3. Law of Conditional Probability
4. Bayes' Rule + Law of Conditional Probability
5. Law of Conditional Probability

The final formula is notably the same as

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \cancel{p(\mathbf{a}_t | \mathbf{s}_t)}$$

where the action prior doesn't matter for the reasons given above. Thus, we have a formula for our policy in terms of our backward messages! But what does this mean? Well, if we replace β_t here with V_t and Q_t , we find that

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)) = \exp(A_t(\mathbf{s}_t, \mathbf{a}_t))$$

Intuitively, this formula says that our policy is just more likely to take the actions that provide higher advantages. (This is effectively analogous to *Boltzmann exploration*, which was briefly mentioned towards the end of [Lecture 7](#)).



Temperature

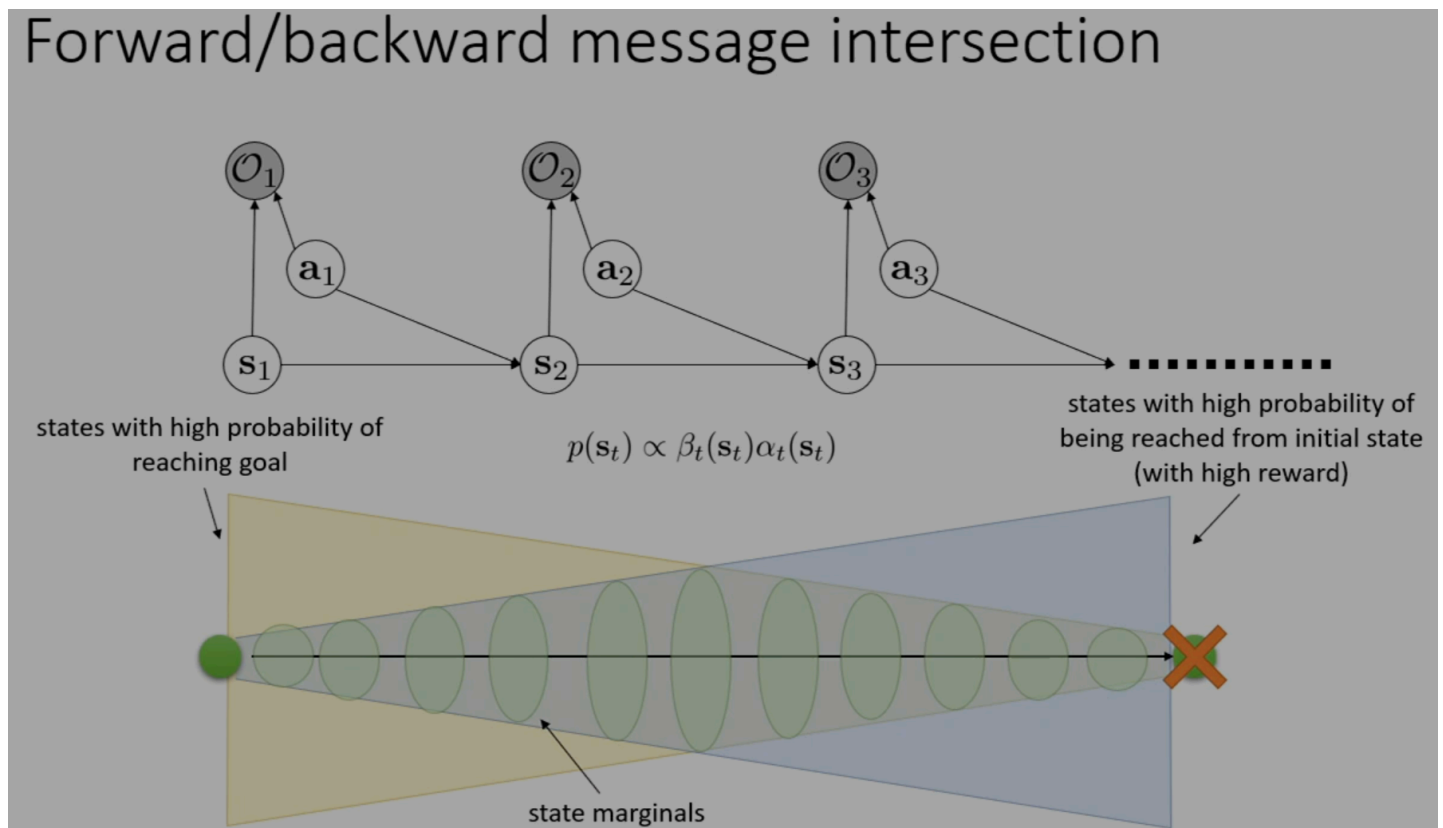
Larger reward functions result in sharper policies, so oftentimes a temperature constant α is applied, $\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp(\frac{1}{\alpha} A_t(\mathbf{s}_t, \mathbf{a}_t))$. As temperature decreases, the soft max approaches the hard max function, and the policy approaches a greedy policy.

Forward Messages

We won't discuss the math in detail, but the whole idea and simplification is very similar to the derivation of backward messages. The key point is, though, is that once we have derived the forward messages recursive formula (allowing us to compute the forward messages across an entire trajectory), we can then compute the probability of any state along an *optimal trajectory*. In fact,

$$p(\mathbf{s}_t | \mathcal{O}_{1:T}) \propto \alpha_t(\mathbf{s}_t) \beta_t(\mathbf{s}_t)$$

and, one may note



In other words, there is more variability in the middle of the trajectory, and less towards the start and the end of the trajectory. Perhaps intuitive, but now we've formalized this idea :)

Lecture 13: Control as Variational Inference

Theory

Recall that our "control as inference" derivation from [Lecture 12](#) used the following value functions.

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log \mathbb{E}[\exp(V_{t+1}(\mathbf{s}_{t+1}))]$$
$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

recall that, with stochastic environment dynamics, the second term of the Q -function expression encourages *excessive optimism* regarding states with very large rewards/values, even if they occur with very small state transition probabilities. We'd like to eliminate this excessive optimism.

Let's first take a step back and consider: why does this occur in the first place?

Our inference problem is $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T} \mid \mathcal{O}_{1:T})$, i.e. what is the probability of the trajectory τ given that the monkey is being optimal? Through marginalization over the time steps t and conditioning on the state $\mathbf{s}_t$, we produce the policy $p(\mathbf{a}_t \mid \mathbf{s}_t, \mathcal{O}_{1:T})$, i.e. what is the probability of the action $\mathbf{a}_t$ being chosen in state $\mathbf{s}_t$ given the optimality of the monkey? However, we may note that $\mathcal{O}_t$ is really just an indicator (*evidence*) of whether or not $\mathbf{a}_t$ was high reward or not; thus, this is really asking, "given that you obtained high reward, what was your action probability?" This, intuitively, is a good question to ask—after understanding that we obtained a high reward, we'd like to maximize our action probability for this high reward.

However, when performing inference over the computational graph, the action probabilities are not the only factors; critically, the *state transition probabilities* are involved too. In fact, we can similarly marginalize over t and condition to produce $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t, \mathcal{O}_{1:T}) \neq p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$. That is, "given that you obtained high reward, what was your transition probability?" And this is a bad question to ask! This is like, say, training our model to make a decision on whether or not to buy a lottery ticket after they already observed winning the lottery ticket in the future. However, just because it won this time does not mean that buying a lottery ticket is a good action in general—in essence, the model is learning action optimality from the fact that it "lucked out" from hitting rare state transition probabilities, and believes it can optimize this state transition probability to always happen.

🔗 Still confused? >

Courtesy of Gemini, who clarified the lecture's idea for me. As you noted, standard inference asks: "*Given that I succeeded, what must have happened?*" If you condition a probabilistic graphical model on achieving high reward ($\mathcal{O}_{1:T} = 1$), the model uses Bayes' rule to update the probability of **all** latent variables that caused that reward. Because both the actions $\mathbf{a}_t$ and the state transitions $\mathbf{s}_{t+1}$ are latent variables leading to the reward, the model updates both.

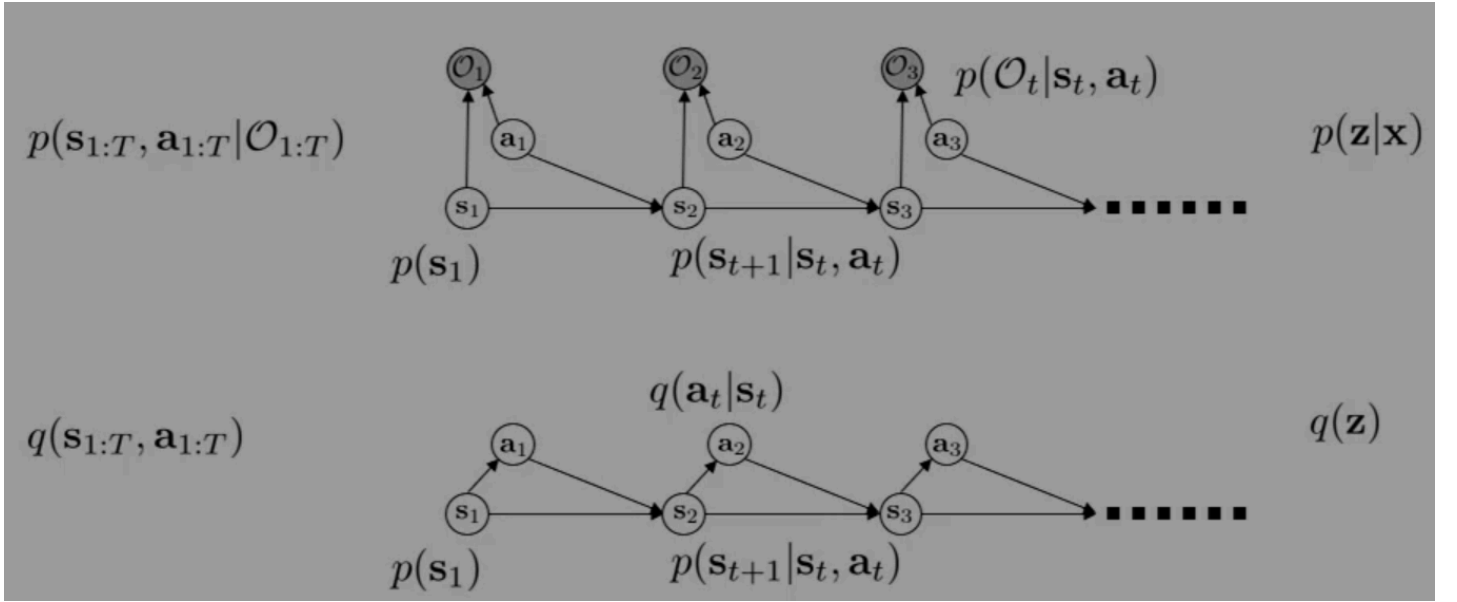
This leads to the model becoming "delusional." If it is in a stochastic environment where taking a risky action usually results in death but has a 1% chance of hitting a jackpot, standard inference looks at the jackpot and assumes the 1% chance is actually a certainty. It essentially assumes it can control the environment's dice rolls.

In short, we'd like to phrase the question as "given that you obtained high reward, what was your action probability, given that your transition probability did not change?" More generally, we'd like to find a way to tell the model to maximize the probability of success without changing the environment dynamics. Or, mathematically, we'd like to find another distribution $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$ that is close to the posterior $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T} \mid \mathcal{O}_{1:T})$ but has static dynamics $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$ (i.e. the model should not believe it is capable of maximizing state transition probabilities like action probabilities).

Well, we can actually apply **variational inference** to this problem! Let our observed variables be $\mathbf{x} = \mathcal{O}_{1:T}$ and our latent variables be $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$. We'd like to find a $q(\mathbf{z})$ that approximates $p(\mathbf{z} \mid \mathbf{x})$; in particular, we will choose to define

$$q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) q(\mathbf{a}_t | \mathbf{s}_t)$$

Notably, it's very unusual to include $p(\mathbf{s}_1)$ and $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ in the definition of q , as, in variational inference, it's typically desirable to make q a simple distribution (for a tractable approximation), and we have no guarantees about the simplicity of those distributions derived from the computational graph. We will immediately see why this is useful, though (induces a lot of cancellation below).



Then, according to the **ELBO** (evidence lower bound)

$$\begin{aligned} \log p(\mathbf{x}) &\geq \mathbb{E}_{\mathbf{z} \sim q(\mathbf{z})} [\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z})] \\ \log p(\mathcal{O}_{1:T}) &\geq \mathbb{E}_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} \left[\left(\log p(\mathbf{s}_1) + \sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) + \sum_{t=1}^T \log p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) \right) \right. \\ &\quad \left. - \left(\log p(\mathbf{s}_1) + \sum_{t=1}^T \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) + \sum_{t=1}^T \log q(\mathbf{a}_t | \mathbf{s}_t) \right) \right] \\ &= \mathbb{E}_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} \left[\sum_{t=1}^T \log p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) - \log q(\mathbf{a}_t | \mathbf{s}_t) \right] \\ &= \mathbb{E}_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) \sim q} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) - \log q(\mathbf{a}_t | \mathbf{s}_t) \right] \end{aligned}$$

For the last step, recall that we defined $p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$ last lecture. This allows us to subsequently conclude that

$$\log p(\mathcal{O}_{1:T}) \geq \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim q} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t | \mathbf{s}_t))]$$

This looks pretty similar to what we've seen before; in fact, it's just our regular RL policy gradient objective plus the entropy of the action distribution! Crucially, though, the only thing within the expectation that is not fixed relative to q (since the expectation is computed over q) is the action probabilities! Our choice to hardcode the environment dynamics $p(\mathbf{s}_t)$ and $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ into $q(\tau)$ cancels with the existing probabilities in $p(\tau, \mathcal{O}_{1:T})$. Thus, our model won't try and modify our state transition probabilities to maximize $p(\mathcal{O}_{1:T})$; it's forced to optimize only $q(\mathbf{a}_t | \mathbf{s}_t)$, which is the only aspect the model actually controls.

Just one final note to close this off: we still need to solve for $q(\mathbf{a}_t | \mathbf{s}_t)$ for all t . We can recurse backwards, like we did for backwards messages. We first compute the base case

$$\begin{aligned}
q(\mathbf{a}_T | \mathbf{s}_T) &= \arg \max_q \mathbb{E}_{\mathbf{s}_T \sim q(\mathbf{s}_T)} [\mathbb{E}_{\mathbf{a}_T \sim q(\mathbf{a}_T | \mathbf{s}_T)} [r(\mathbf{s}_T, \mathbf{a}_T)] + \mathcal{H}(q(\mathbf{a}_T | \mathbf{s}_T))] \\
&= \frac{\exp(r(\mathbf{s}_T, \mathbf{a}_T))}{\int \exp(r(\mathbf{s}_T, \mathbf{a})) d\mathbf{a}} = \exp(Q(\mathbf{s}_T, \mathbf{a}_T) - V(\mathbf{s}_T))
\end{aligned}$$

and then, with many steps of mathematical derivation, we can derive

$$q(\mathbf{a}_t | \mathbf{s}_t) = \exp(Q(\mathbf{s}_t, \mathbf{a}_t) - V(\mathbf{s}_t))$$

where $Q(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}[V_{t+1}(\mathbf{s}_{t+1})]$. Notably, it is now a regular Bellman backup equation; it's no longer optimistic.

And thus, we can compute our value functions backwards through time with our recursive expressions. For $t = T - 1$ to 1,

$$\begin{aligned}
Q_t(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}[V_{t+1}(\mathbf{s}_{t+1})] \\
V_t(\mathbf{s}_t) &= \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t
\end{aligned}$$

and we now have a **soft value iteration** algorithm:

1. Set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \mathbb{E}[V(\mathbf{s}')].$
2. Set $V(\mathbf{s}) \leftarrow \text{soft max}_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}).$

Various modifications like *discounting* and *temperature* may also be added.

Maximum Entropy RL Algorithms

🔗 Why use soft max?

The easy answer is that it encourages more exploration, though this does not capture the entire impact of the soft max. The true answer is that it encourages robustness to inaccurately specified MDPs, i.e. the addition of the entropy term $\mathcal{H}(q)$ encourages the model to seek high rewards while acting as randomly as possible, eventually placing the learned policy in an action space that is robust to perturbations. In reality, though, the real answer is that it just works well in practice ;)

Q-Learning with soft optimality

Standard Q-learning uses the *hard max* over the Q values to determine the value function, i.e. $V(\mathbf{s}') = \max_{\mathbf{a}'} Q_{\phi}(\mathbf{s}', \mathbf{a}')$. For **soft Q-learning**, we just change the hard max to soft max. Ultimately, this is a very popular algorithm because, for small action spaces, the hard max works just fine.

Policy Gradient with soft optimality

Recall that the control with variational inference strategy ended up with a policy gradient objective with an added action entropy term. The modification to policy gradient is precisely the same, just define $J(\theta) = \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \pi_{\theta}} [r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t))]$. This is very commonly applied, because it's simple to add (entropy usually has a closed form expression for most policy classes) and policy gradient often collapses to a deterministic, non-exploratory policy too soon during training.

Soft Actor-Critic

This is the most well-known application of soft optimality. We modify two things:

$$y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{s}_i')} [\hat{Q}_\phi^\pi(\mathbf{s}_i', \mathbf{a}') + \underbrace{\mathcal{H}(\pi_\theta(\mathbf{a}' | \mathbf{s}_i'))}_{\text{new}}]$$

$$J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}) + \underbrace{\mathcal{H}(\pi_\theta(\mathbf{a} | \mathbf{s}_i))}_{\text{new}}]$$

where you may recall that the y_i are the target values used to train the critic $\hat{Q}_\phi^\pi$. Note that, conventionally, the entropy for the Q -function is usually estimated, i.e.

$$y_i \approx r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{s}_i')} [\hat{Q}_\phi^\pi(\mathbf{s}_i', \mathbf{a}') - \log \pi_\theta(\mathbf{a}' | \mathbf{s}_i')]$$

Also, it's very common to add a "temperature" β as a coefficient to the entropy terms.

Inverse Reinforcement Learning

In standard imitation learning, e.g. behavioral cloning, the model attempts to copy the **actions** the expert takes, without any reasoning about the action outcomes. In contrast, humans, when learning, attempt to copy the **intent** of the expert, and consequently may take very different actions due to their reasoning about the expert's intent. Thus, it's potentially very useful to **learn reward functions from demonstrations**, rather than have the developer define a reward function for the model to optimize.

Our previous soft optimality or control as inference framework is very helpful for this. Let's use the same probabilistic model with the same optimality variable; but now, instead of learning a policy based on a reward function, we will attempt to learn a reward function given trajectories sampled from a **soft-optimal** policy. In other words, we have

$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t, \psi) = \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$$

and consequently

$$p(\tau | \mathcal{O}_{1:T}, \psi) \propto p(\tau) \exp\left(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\right)$$

Maximum likelihood learning tells us that the optimal ψ is simply

$$\hat{\psi} = \arg \max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i | \mathcal{O}_{1:T}, \psi)$$

Notably, this is a **latent variable** model, as the states and actions are unobserved (note their exclusion from the maximum likelihood equation for $\hat{\psi}$), and thus must be inferred using **variational inference**.

Also, we may note that, when taking the gradient of $p(\tau | \mathcal{O}_{1:T}, \psi)$ w.r.t ψ , $p(\tau)$ is irrelevant. Therefore, we're really considering

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \underbrace{\log Z}_{\text{normalizer}}$$

where Z is the partition function $Z = \int p(\tau) \exp(r_\psi(\tau)) d\tau$. Note that the normalizer is applied because $p(\tau | \mathcal{O}_{1:T}, \psi)$ is only proportional to $p(\tau) \exp(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t))$; there's a hidden normalizer term involved with the expression as well that's not shown in the proportionality equation. Taking the gradient,

$$\begin{aligned} \nabla_\psi \mathcal{L} &= \frac{1}{N} \sum_{i=1}^N \nabla_\psi r_\psi(\tau_i) - \underbrace{\frac{1}{Z} \int p(\tau) \exp(r_\psi(\tau)) \nabla_\psi r_\psi(\tau) d\tau}_{p(\tau | \mathcal{O}_{1:T}, \psi)} \\ &= \mathbb{E}_{\tau \sim \pi^*(\tau)} [\nabla_\psi r_\psi(\tau_i)] - \mathbb{E}_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)} [\nabla_\psi r_\psi(\tau)] \end{aligned}$$

Notably, the gradient is zero when $\pi^*(\tau)$, the policy of our expert that we estimate with samples, is equivalent to $p(\tau \mid \mathcal{O}_{1:T}, \psi)$, the soft optimal policy produced by following our current estimated reward function.

Lecture 14: RL with Sequences and LLMs

Reward Learning and Inverse RL

Recall the idea of **inverse reinforcement learning** (IRL): learning a model of the reward function demonstrated through some samples from an "expert" actor. This is in contrast to behavioral cloning in imitation learning, which simply seeks to mimic the behavior of the expert, and therefore expresses limited generalization to unseen states that require similar reasoning. (There are also other advantages of IRL, such as the compounding error from distributional shift that behavioral cloning typically encounters).

When we attempt to apply inverse RL to our previous soft optimality framework, we attempt to learn the reward model, provided we have sample trajectories from a soft-optimal policy. We use the equation

$$p(\mathcal{O}_t \mid \mathbf{s}_t, \mathbf{a}_t, \psi) = \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$$

and use maximum likelihood estimation to find that

$$\hat{\psi} = \arg \max_{\psi} \frac{1}{N} \sum_{i=1}^N \log p(\tau_i \mid \mathcal{O}_{1:T}, \psi) = \arg \max_{\psi} \frac{1}{N} \sum_{i=1}^N r_\psi(\tau_i) - \log Z$$

where Z is the inverse RL **partition function**, and is equivalent to

$$Z = \int p(\tau) \exp r_\psi(\tau) d\tau$$

Note that Z functions as a "normalizer" for ψ in order to produce a probability distribution, considering that the rewards themselves are unnormalized. Taking the gradient of our loss function, we find that

$$\begin{aligned} \nabla_{\psi} \mathcal{L} &= \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \underbrace{\frac{1}{Z} \int p(\tau) \exp(r_{\psi}(\tau)) \nabla_{\psi} r_{\psi}(\tau) d\tau}_{p(\tau \mid \mathcal{O}_{1:T}, \psi)} \\ &= \mathbb{E}_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)] - \mathbb{E}_{\tau \sim p(\tau \mid \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)] \end{aligned}$$

In words, these expectations are over trajectories produced by the...

$$\nabla_{\psi} \mathcal{L} = \underbrace{\mathbb{E}_{\tau \sim \pi^*(\tau)} [\nabla_{\psi} r_{\psi}(\tau_i)]}_{\text{expert policy}} - \underbrace{\mathbb{E}_{\tau \sim p(\tau \mid \mathcal{O}_{1:T}, \psi)} [\nabla_{\psi} r_{\psi}(\tau)]}_{\text{soft optimal policy w/ current reward}}$$

For estimating that second expectation, i.e. the expectation over trajectories from our soft optimal policy, we can apply any maximum-entropy RL algorithm (see [Lecture 13](#)). Thus, we can just do

$$\nabla_{\psi} \mathcal{L} \approx \underbrace{\frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i)}_{\text{expert samples}} - \underbrace{\frac{1}{M} \sum_{j=1}^M \nabla_{\psi} r_{\psi}(\tau_j)}_{\text{policy samples}}$$

Unfortunately, this method is **computationally expensive**. (Why? We effectively have two nested loops of gradient ascent; for every step we take on ψ , we have to run an entire maximum-entropy RL algorithm to relearn our soft optimal policy). So, can we do better?

Taking ideas from **generalized policy iteration**, we can be "lazy" with our relearning of our soft optimal policy; instead of learning an estimate of $p(\mathbf{a}_t \mid \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ until convergence every time we update r_ψ , we only make a marginal update (take a few policy gradient/max-entropy algorithm steps on our policy p) that brings us closer to the soft optimal policy for our new reward function. Sadly, this is **not valid** and produces a biased estimate—our second expectation is now being performed over samples taken from a non-optimal policy for our reward function...

...wait. Training on samples taken from a trajectory distribution produced by another policy? Isn't that just **off-policy learning**? So we can just use **importance sampling** to correct our algorithm!

$$\nabla_{\psi} \mathcal{L} \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\psi} r_{\psi}(\tau_i) - \frac{1}{\sum_j w_j} \sum_{j=1}^M w_j \nabla_{\psi} r_{\psi}(\tau_j)$$

where we define

$$\begin{aligned} w_j &= \frac{p(\tau) \exp(r_{\psi}(\tau_j))}{\pi(\tau_j)} = \frac{p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \exp(r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \pi(\mathbf{a}_t | \mathbf{s}_t)} \\ &= \frac{\exp(\sum_t r_{\psi}(\mathbf{s}_t, \mathbf{a}_t))}{\prod_t \pi(\mathbf{a}_t | \mathbf{s}_t)} \end{aligned}$$

intuitively, this just weights samples according to their reward, computed by the current reward function, relative to the probability of the sample occurring under the non-optimal policy. In other words, reweighting the samples to more heavily favor those that the optimal policy of the current reward function would have been more likely to take.

One may interpret this training procedure as a two-player **adversarial** game. There is a discriminator, ψ , that attempts to increase the likelihood of samples from the expert demonstrations (positive first expectation $\mathbb{E}_{\tau \sim \pi^*(\tau)}$ *encourages* gradient ascent on/maximization of rewards from the expert trajectories), and simultaneously decreases the likelihood of samples from the policy θ (negative second expectation $-\mathbb{E}_{\tau \sim p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)}$ encourages gradient *descent* on/minimization of rewards from policy trajectories). Meanwhile, the generator θ is continuously improved to make it harder for ψ to distinguish between samples from the expert and samples from θ . In this way, we continually improve our reward function and policy in a competing manner, until convergence at an equilibrium where the $\pi^* = p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$, i.e. the policy is encouraged to imitate the expert.

Such game-like models are known as **generative adversarial networks** (GANs). A **discriminator** model is trained to distinguish between samples from a dataset and samples from a **generator** model, while the generator model is optimized to fool the discriminator.

So, let's go back and approach this question from the beginning, but with the intent of designing a GAN for our problem. We still have a policy as our generator model, but instead of a reward model, we'll use a binary discriminator.

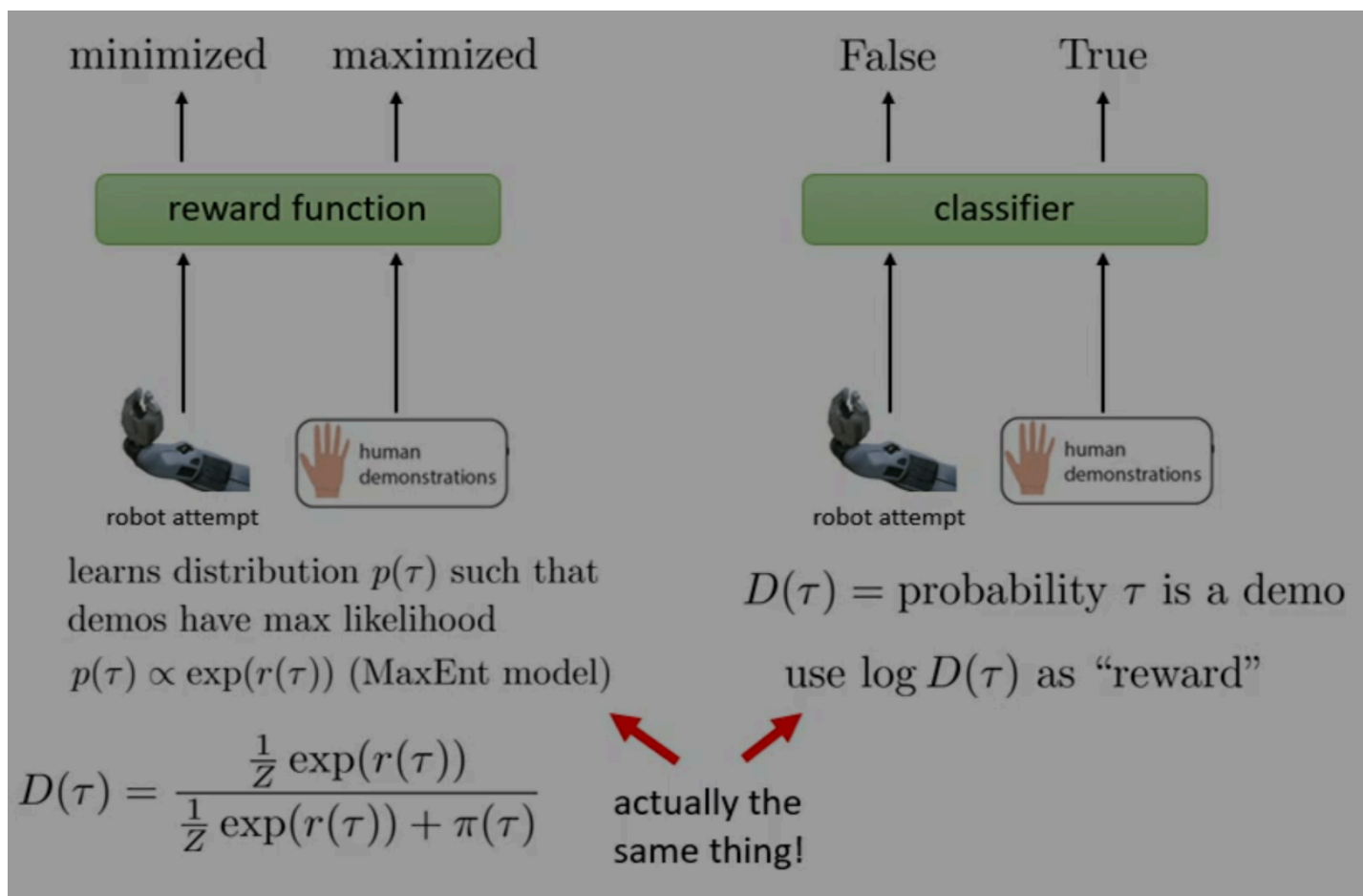
$$\psi \leftarrow \arg \max_{\psi} \mathbb{E}_{\tau \sim p^*} [\log D_{\psi}(\tau)] + \mathbb{E}_{\tau \sim \pi_{\theta}} [\log(1 - D_{\psi}(\tau))]$$

and our policy gradient is

$$\nabla_{\theta} \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^M \nabla_{\theta} \log \pi_{\theta}(\tau_j) \log D_{\psi}(\tau_j)$$

This is much simpler to implement than our inverse RL algorithm. However, at the end of training, since the policy is essentially identical to the expert demonstrations, the discriminator will learn to simply output 0.5 probability on every input; that is, the discriminator knows nothing at convergence! Moreover, it's generally not possible to *reoptimize* the reward function D .

In short, though, we have the following comparison.



RL and LLMs: The Basics

Important questions for applying RL to LLMs:

- What is the (PO)MDP, i.e. (partially observed) Markov decision process?
- What is the reward function?
- What algorithm do we use?

First, we discuss a short anatomy of LLM training.

1. Pre-training: Supervised training on a large corpus of data, gives LLM knowledge.
2. Post-training: Tells LLM how to use its knowledge.
 - Supervised fine-tuning (SFT) (instruction tuning, basically imitation learning) on a smaller, high-quality dataset that demonstrates what you want the LLM to do (e.g. be a helpful model).
 - RL fine-tuning (e.g. [reinforcement learning from human feedback/RLHF](#)).

Instruction tuning was the initial LLM post-training methodology. However, it's labor-intensive to collect an SFT dataset, especially for skilled tasks like programming. Reinforcement learning offers a more tractable solution.

There are two main approaches to RL post-training.

- Training an LLM to appeal to human preferences. This can be dangerous, though, if preferences do not actually reflect what's desirable (e.g. LLM removes all test cases to pass all unit tests).

- Training an LLM to be correct. (a.k.a. RL from verifiers). Typically, "thinking mode" is a product of this. This is difficult to check objectively, however.

So how do we define our MDP for an LLM? There are a couple methods.

- Define $p(\mathbf{a} \mid \mathbf{s})$ where $\mathbf{a}$ is the response and $\mathbf{s}$ is the prompt, i.e. creating a one-step MDP.
- Define $\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)$ for all tokens t in the response, where $\mathbf{a}_t$ is the next token and $\mathbf{s}_t$ is the prompt and all the tokens of the response so far, i.e. creating a multi-step MDP.

The second perspective is more sensible, as it allows for intermediate rewards (rewards during response generation) and value function baselines (which tokens are better than others?).

We'll now try and apply *policy gradient* to LLM post-training. We have, as usual,

$$\begin{aligned}\nabla_\theta \mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] &= \mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[\nabla_\theta \log \pi_\theta(\mathbf{a} \mid \mathbf{s}) r(\mathbf{s}, \mathbf{a})] \\ &\approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i) \\ &\approx \frac{1}{N} \sum_i \frac{\pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i \mid \mathbf{s}_i)} \nabla_\theta \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}_i) r(\mathbf{s}_i, \mathbf{a}_i)\end{aligned}$$

where the first approximation is a REINFORCE-style estimator and the second approximation is an importance-weighted estimator like PPO. In practice, the second approximation is used for efficiency reasons (multiple gradient steps for each data sampling step).

Algorithms for RL with LLMs

There are a couple issues to address. We have to choose a good...

- Baseline
- Regularizer
- Reward function

Baseline

Recall value function baselines and [GAE](#). LLMs typically use a GAE baseline for the objective function. For PPO, we use a loss function for our value function estimator $\hat{V}_\phi^\pi$ defined

$$\begin{aligned}\mathcal{L}(\phi) &= \frac{1}{2} \sum_{i=1}^N \|\hat{V}_\phi^\pi(\mathbf{s}_i) - y_i\|^2 \\ &= \frac{1}{2} \sum_{i=1}^N \|\hat{V}_\phi^\pi(\mathbf{s}_i) - (\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_{\text{old}}(\mathbf{s}_i))\|^2\end{aligned}$$

Now, we do need to use our language model to produce estimates of our value function. We can either (1) copy the LLM and train the copy to estimate the value function or (2) add a second head to each output token that estimates the value of that token.

Also, the value function is typically not trained on the prompt, only on the response (the suffix of the prompt, typically, e.g. the last word of "What is the capital of France? Paris").

There's another way to compute baselines with LLMs that does not require an extra copy or head, which is in fact not applicable to general RL. We can estimate our baseline by using *decision-time planning*: we can "checkpoint" the current state of the LLM, and average across several trajectory samples branching out from the current state. This is possible because we can easily simulate multiple completions from the same LLM state, something that may not be possible for an autonomous driving RL problem. This is known as **group relative policy optimization (GRPO)**, and is much simpler to implement :)

Regularization

Typically, this is done by simply adding a KL divergence term to the reward function with respect to some reference model, commonly the model produced by SFT.

$$\bar{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) - \beta D_{\text{KL}}(\pi_\theta(\mathbf{a}_t | \mathbf{s}_t), \pi_{\text{ref}}(\mathbf{a}_t | \mathbf{s}_t))$$

Reward Function: Bradley-Terry Model (Preference)

Optometrist Algorithm

Colloquially known as the "optometrist algorithm," since an optometrist asks a patient to compare two sets of lenses until they narrow it down to a good lens.

The algorithm proceeds as follows.

1. Sample 2 or more trajectories $\tau_i \sim \pi_\theta(\tau)$.
2. Ask a person to choose preference(s) in $\{\tau_i\}_{i=1}^N$ (i.e. pairwise ranking).
3. Train reward r_ψ using preferences.
4. Train policy π_θ using r_ψ .

But, how do we train r_ψ ? We should use a probabilistic model to account for the stochasticity and noise of human decisions. In particular, we apply

$$p(\tau_i \succ \tau_j) = \frac{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right)}{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right) + \exp\left(\sum_t r_\psi(\mathbf{s}_t^{(j)}, \mathbf{a}_t^{(j)})\right)}$$

which is actually just

$$p(\tau_i \succ \tau_j) = \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$$

Chess ELO

This is precisely how ELO scores are calculated in chess!

So, to train r_ψ , we simply do maximum likelihood estimation

$$\psi \leftarrow \arg \max_{\psi} \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$$

Full RLHF Algorithm

1. Sample 2 or more trajectories $\tau_i \sim \pi_\theta(\tau)$.
2. Ask for human preferences $\{\tau_i \succ \tau_j\}_{i,j}$.
3. Train reward model $\psi \leftarrow \arg \max_{\psi} \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$.
4. Train policy π_θ with reward r_ψ .

Other Reward Sources

- Verifier reward: other model evaluates validity of answer
- Process reward: other model evaluates validity of chain-of-thought generation

RL with Partial Observability

The key difference between full observability MDPs and POMDPs is that POMDPs do not obey the Markov property! In particular, knowledge of $\mathbf{s}_t$ makes knowledge of $\mathbf{s}_{t-1}$ useless, but knowledge of $\mathbf{o}_t$ does not make knowledge of $\mathbf{o}_{t-1}$ useless. Therefore, knowing the history of observations can be useful.

POMDPs have some special properties when compared to normal MDPs.

- An optimal policy may engage in information-gathering actions.
- Some POMDPs do not have a deterministic optimal policy, only a stochastic optimal policy.

So, which methods can be applied to POMDPs?

- Policy gradients? Okay, provided advantage estimation does not rely on a value function (requires Markov property).
- Value-based methods? (e.g. Q -learning). Not okay, they always produce a deterministic policy and depend on producing the value as a function of the state (and you can't just replace the state with observations, sadly).
- Model-based methods? More on this later...

One critical idea in RL with partial observability is to use **history states**, where $\mathbf{s}_t = (\mathbf{o}_1, \dots, \mathbf{o}_t)$. History states, luckily, do obey the Markov property! This allows us to just use history states with our existing full observability RL methods. We'll discuss this in more detail next lecture :>

Lecture 15: Model-Based RL

Learning a Simulator

Model-based RL, in contrast to the model-free RL methods discussed thus far, attempt to learn a model of the problem environment in order to produce an optimal policy. Here's a sketch of a simple simulator-learning algorithm.

1. Sample data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using base policy $\pi_\beta(\mathbf{a} \mid \mathbf{s})$.
2. Learn simulator $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$ with some loss function, e.g. MSE or MLE.
3. Run favorite RL method inside simulator f . This could simply be a model-free algorithm, or it could be a planning/trajectory optimization algorithm that takes advantage of our simulator f .

There can be some complications, however, with each step.

1. The dataset must be somewhat comprehensive with regards to the state space, thus the choice of π_β is extremely important to the consequent dataset quality. (Statistics/algorithms problem).
2. The design of the simulator model is *domain-dependent* since the model complexity and structure depend heavily on the problem complexity and structure. (Deep learning/models problem).
3. In general, model-free RL in the learned simulator may be difficult; for instance, the simulator may be very computationally inefficient, making sampling expensive. (Controls/RL problem).

Today, we'll primarily discuss the first step/problem.

Distributional Shift and Uncertainty

Let π_f be the optimal policy learned under the simulator f . The key issue with problem 1 is that the state distribution observed in the dataset produced by π_β is not the same as the state distribution observed by π_f ; in other words, we have a **distributional shift** problem, just like in [Lecture 2](#) with imitation learning!

Well, what if we add the following step to our algorithm?

1. Sample data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$ using base policy $\pi_\beta(\mathbf{a} \mid \mathbf{s})$.
2. Learn simulator $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$.
3. Learn optimal policy π_f under f .
4. $\pi_\beta \leftarrow \pi_f$.



Tip

Note the similarity between this modification and [DAgger](#).

Does this fix the problem? Unfortunately, no; while this is an improvement, this is not sufficient. In particular, step 3, which learns the optimal policy π_f , will actually end up *exploiting the mistakes* or misrepresentations in our simulator f . This encourages extreme policies, which massively slows down training speed not only because it produces policies with bad generalization but also because those policies will produce wildly varying data.

So, what if we replace step 3 with just a small improvement to our policy, like in [PPO](#)? This is a good idea, but what does "small" mean here? Ideally, we'd like larger improvements to learn faster, while also being sufficiently small such that our model does not overfit to our erroneous simulator f . Generally speaking, answering this question is a matter of hyperparameter tuning during training.

Also, how exactly do we enforce this "small" improvement? We have a few options.

- Add a trust region ([TRPO](#)) or KL divergence constraint $D_{\text{KL}}(\pi_f \parallel \pi_\beta) \leq \epsilon$.
- Stick to regions where the simulator model is "confident." This may be implemented with a [probabilistic](#), uncertainty-aware model or a pessimism penalty when presented with any model uncertainty. For the first, in step 3, we additionally extract the best policy *in expectation* under our probabilistic model.

For the probabilistic model, there are notably a few caveats to consider.

- We'd like to promote exploration when collecting our dataset to better improve our simulator model.
- Extracting the best policy in expectation is not the same as a pessimistic/optimistic value.

Uncertainty-Aware Neural Networks

Consider first a neural network that outputs a distribution over the state space. What if we use the [entropy](#) of the output distribution to measure confidence? Unfortunately, this is problematic because the simulator model often becomes erroneously confident due to excessive overfitting.

In particular, there are really two types of uncertainty.

- [Aleatoric](#) or statistical uncertainty: "how random does the model think the data is?"
- [Epistemic](#) or model uncertainty: "the model is certain about the data, but we are uncertain about the model itself!"

Epistemic uncertainty is what we are trying to reduce; let's try and estimate it!

In typical MLE, we estimate $\hat{\theta} = \arg \max_{\theta} \log p(\theta \mid \mathcal{D})$. However, what if, instead of learning just a single, likelihood-maximizing parameter $\hat{\theta}$, we learned the [conditional distribution](#) of the parameters, i.e. we learned $p(\theta \mid \mathcal{D})$. This distribution's entropy captures our epistemic uncertainty—if we're certain about the model, the distribution will have extremely low entropy, with most probability mass being concentrated on a single point. In contrast, if we're uncertain, the distribution will have high entropy.

Moreover, with $p(\theta \mid \mathcal{D})$, we can make predictions based on the entire distribution of our parameters, i.e.

$$\int p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t) p(\theta \mid \mathcal{D}) d\theta$$

Learning an entire distribution is generally intractable/much harder than learning a single parameter estimate—in practice, we approximate the distribution.

One such approximation is a [Bayesian neural network](#). A Bayesian neural network replaces weight parameters with distributions over each weight parameter. This corresponds to a [fully factorized posterior](#), as we are approximating our distribution $p(\theta \mid \mathcal{D})$ with $\prod_i p(\theta_i \mid \mathcal{D})$, which includes a posterior distribution for every individual parameter. Notably, this does not allow representing covariances between parameters; but it is simple. Also, we typically represent $p(\theta_i \mid \mathcal{D}) = \mathcal{N}(\mu_i, \sigma_i^2)$, and thus the number of parameters really only doubles. And, how are they trained? Variational inference. We won't discuss the derivation of this in detail.

Another, simpler approximation is [bootstrap ensembles](#), in which we learn multiple models, and synthesize them to produce a prediction $p(\theta \mid \mathcal{D}) \approx \frac{1}{N} \sum_i \delta(\theta_i)$. (See [chapter 7 from Goodfellow et al.](#)). But how do we train these models so that they don't simply learn the same parameters? We could partition our dataset into k disjoint sets for k models; however, this may be a very inefficient use of our dataset. Instead, we can choose to produce $\mathcal{D}_i$ by sampling with replacement from $\mathcal{D}$ for the same size as the number of data points in $\mathcal{D}$. The "with replacement" sampling produces sufficiently independent datasets that translate to sufficiently independent models.

In practice, bootstrap ensembles operate with a couple caveats/considerations: (1) the number of models is typically small for efficiency reasons, which does lower approximation accuracy, and (2) SGD with random parameter initialization typically produces sufficiently independent models without resampling with replacement.

Lecture 16: Model-Based RL Algorithms

Planning with Models

Planning in RL describes, very generally, the process of using a model of our environment to make decisions **without a policy**.

Let's consider a **deterministic dynamics, open-loop** world model. Given an initial state s_1 , we use our model to compute the reward-maximizing sequence of actions, i.e.

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)$$

where $\mathbf{s}_{t+1}$ is given by our model f at each time step, i.e. $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$. Doing this directly works just fine, and produces an optimal solution.

But, what about a **stochastic dynamics, open-loop** world model? Well, we can do essentially the same thing, but compute the maximum in expectation, i.e.

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \mathbb{E} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \mid \mathbf{a}_1, \dots, \mathbf{a}_T \right]$$

where we must consider the distribution of possible states

$$p_\theta(\mathbf{s}_1, \dots, \mathbf{s}_T \mid \mathbf{a}_1, \dots, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$$

In practice, however, with a model possessing stochastic dynamics, there are some complications. For one, the expectation must be **approximated** due to the typically large size of state and action spaces (e.g. via sampling). And, critically, this method is actually **suboptimal**!

Why is it suboptimal? It's primarily because the model is open-loop: it takes in the initial state, and then outputs a full sequence of actions to maximize expected reward. However, this produces suboptimal actions because, for instance, after transitioning to state s_2 , the optimal action may not follow the previously inferred sequence! This is because the uncertainty regarding the state transition has been resolved, allowing for a more optimal decision to be made at this point than what was previously inferred without this additional information.

Key Idea: **new information** helps us make better decisions!

Thus, for RL, we always used a **stochastic dynamics, closed-loop** model (unless the environment is truly deterministic, which doesn't really happen in the real world), in which our actor receives **feedback** from the environment about the actions its taken. Notably, the actor now takes some "policy" that reacts to the feedback its given—this diverges from our notion of planning, and in fact is really just standard reinforcement learning.

So, how can planning methods be useful? Actually, we can employ open-loop planning methods **within a closed-loop RL method** to great effect.

Example

For instance, a simple idea is to, at each step of the closed-loop RL method, use an open-loop planning method to decide a **short** sequence of actions that does not induce execution until the end of the entire horizon, but merely from time step t to $t + k$, for some small k . This is effectively the same as **action chunking**, which we discussed briefly in Lecture 3.

But first, let's describe some simple open-loop planning methods, before we discussing embedding them into a closed-loop method.

Stochastic Optimization / Random Shooting

Stochastic optimization is the simplest open-loop planning method.

First, let's briefly clarify some notation. The objective of optimal control/planning is to compute

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} J(\mathbf{a}_1, \dots, \mathbf{a}_T)$$

for some objective function J (e.g. expectation of the sum of rewards). We will define $\mathbf{A} = \mathbf{a}_1, \dots, \mathbf{a}_T$ so we may write this concisely as

$$\mathbf{A} = \arg \max_{\mathbf{A}} J(\mathbf{A})$$

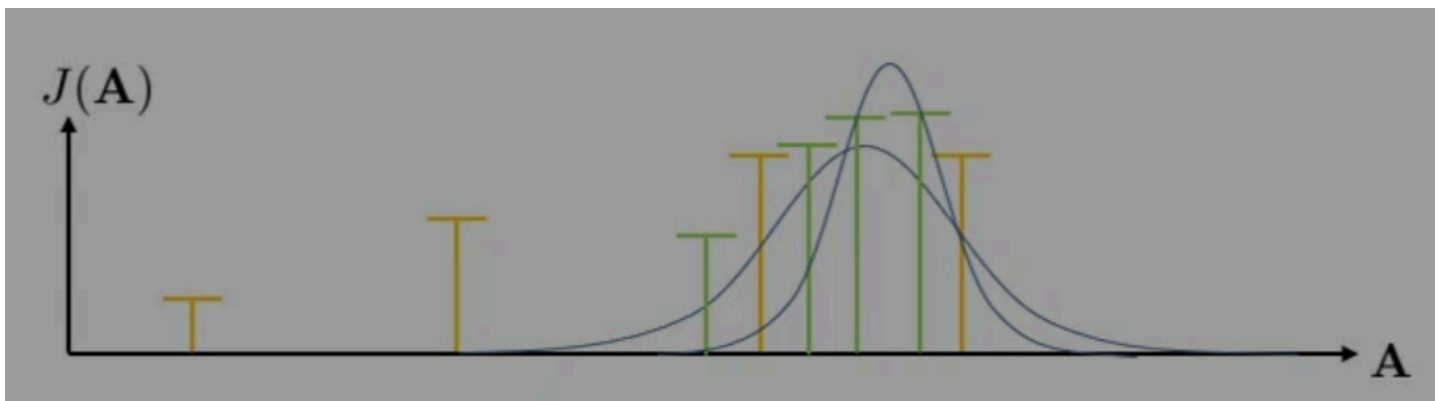
Stochastic optimization is really just "guess and check," i.e. it

1. Picks $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g. uniform).
2. Choose $\mathbf{A}_i$ based on $\arg \max_i J(\mathbf{A}_i)$.

This is computationally cheap (provided parallelization is possible, which is likely). However, the performance of this method depends heavily on dimensionality (high dimensionality is very bad) and the landscape of the objective function (low entropy distribution is bad).

Cross-Entropy Method (CEM)

The CEM method is a small modification to stochastic optimization that produces significant performance improvements. In essence, rather than simply sampling $\mathbf{A}_1, \dots, \mathbf{A}_N$ all from some distribution, we sample a set $\mathbf{A}_1, \dots, \mathbf{A}_N$ multiple times. Between iterations, we *change our sampling distribution* to place more probability mass on the existing "good" samples seen in past iterations. This idea is illustrated below, where the yellow samples are the first iteration, the green samples are the second iteration of samples, and the blue curves represent the changing sample distributions (uniform $\rightarrow$ wide curve $\rightarrow$ narrow curve).



Here's the algorithm steps.

1. Sample $\mathbf{A}_1, \dots, \mathbf{A}_N \sim p(\mathbf{A})$.
2. Evaluate $J(\mathbf{A}_1), \dots, J(\mathbf{A}_N)$.
3. Pick high-value *elites* $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$ where $M < N$.
4. Refit $p(\mathbf{A})$ to elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$.

This performs much better than stochastic optimization, while being almost just as simple and still very efficient. Unfortunately, it does still suffer from the *curse of dimensionality*.

Other Methods

- Monte Carlo tree search (MCTS)
- Continuous trajectory optimization (LQR, etc.)
- Tree-based motion planning (RRT)

How to Plan with Uncertainty

With a deterministic setting, we simply have

$$J(\mathbf{A}) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t), \quad \mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$

In the stochastic setting,

$$J(\mathbf{A}) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(\mathbf{s}_{t,i}, \mathbf{a}_t), \quad \mathbf{s}_{t+1,i} = f_i(\mathbf{s}_{t,i}, \mathbf{a}_t)$$

where we are averaging over N samples.

Given the uncertainty of the model, we may use a bootstrap ensemble (see [Lecture 15](#)) for f_i to reduce distributional shift, which is typically some distribution over deterministic models. Notably, there are two choices for our bootstrap ensemble of when we "sample" a model from the distribution:

1. Sample a single model f_i for each i , and use it over the whole trajectory
2. Sample a new model f_i for every (i, t) (each time step in each trajectory).

However, it typically is more sensible to keep a model constant across a trajectory (1). Otherwise, we're effectively changing our simulated environment every single step—this is bad, because while each environment model in the ensemble is somewhat probable, though perhaps uncertain, a combination of the models, each used at different time steps, may be a very unlikely representation of the environment!

In general, for a candidate action sequence $\mathbf{A}$,

1. Sample $\theta \sim p(\theta \mid \mathcal{D})$ (sample environment model parameters from bootstrap ensemble).
2. At each time step t , sample $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t, \theta)$ (sample from environment model).
3. Calculate $R = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$.
4. Repeat and accumulate average reward.

This, consequently, computes the value of $J(\mathbf{A})$ for some candidate $\mathbf{A}$ while considering both our *epistemic* uncertainty about the model's accuracy and the *aleatoric* uncertainty of the model with regards to the state transitions.

Policy Learning with Models

Let's now return to the stochastic, closed-loop case, or standard RL augmented with an environment model.

Model-free Optimization with a Model

First, we'll discuss how we may simply augment our existing, model-free methods with the use of a model. The simplest approach is to just use policy gradient, but use our model to generate samples.

1. Gather samples from environment model

2. Optimize policy gradient for a few steps
3. Run policy in the real world to generate data to improve our environment model

Notably, though, policy gradient was previously inspired by a lack of knowledge of environment dynamics, and thus relied on samples to estimate $\nabla_{\theta} J(\theta)$ —this is known as a **likelihood gradient**. But, one might notice that, with our environment model's estimate of the dynamics, we can actually perform backpropagation through (the trajectory in) our model to estimate $\nabla_{\theta} J(\theta)$ —this is known as a **pathwise gradient**. Is this better?

The pathwise gradient does reduce variance compared to the likelihood gradient since it is no longer Monte Carlo; however, the pathwise gradient also suffers from **severe ill-conditioning** due to the multiplication of many (about H , the horizon length) Jacobians! Thus, in practice, the standard policy gradient is often more stable than the pathwise gradient, provided sufficient samples.

Key Idea: Backpropagation through learned dynamics generally doesn't perform well due to ill-conditioning.

So, here's our new policy gradient model-based RL algorithm.

1. Run base policy $\pi_0(\mathbf{a}_t | \mathbf{s}_t)$ to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$.
2. Learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$.
3. Use $f(\mathbf{s}, \mathbf{a})$ to generate trajectories $\{\tau_i\}$ with policy $\pi_{\theta}(\mathbf{a} | \mathbf{s})$.
4. Use $\{\tau_i\}$ to improve π_{θ} via policy gradient.
5. $\pi_0 \leftarrow \pi_{\theta}$.

Note that steps 3-4 are often repeated several times before collecting more data from the real world, to improve sample efficiency.

This method still has some problems, though. In particular, it suffers from the **curse of model-based rollouts**. In essence, the error caused by distributional shift accumulates **quadratically** in the length of the horizon. Thus, if we sample long trajectories produced by policy π_{θ} from our dynamics model f , the trajectories may become extremely inaccurate!

Thus, we'd like to use our model only for **short rollouts**. How? We could try simply shortening the task horizon, but for tasks where certain states may only appear in later time steps, this may result in the model never fully learning what it needs. (E.g. model performing a task that is physically impossible to complete within the horizon).

Instead, a more effective method is to generate long rollouts from the real world environment, and then generate short rollouts from f by starting, not from the initial state, but from **randomly selected states** observed from the real world trajectories (our replay buffer). This reduces our distributional shift error while also allowing the model to see all time steps. However, our state distribution isn't right anymore—our state distribution is now sampled from the states in the replay buffer, which are sampled from older policies. Luckily, our state distribution generally does not diverge too much, and thus this is an acceptable compromise (primarily for off-policy methods, which are more tolerant of this divergence).

So now, here's the modified algorithm.

1. Run base policy $\pi_0(\mathbf{a}_t | \mathbf{s}_t)$ to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$.
2. Learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$.
3. Pick states $\mathbf{s}_i$ from $\mathcal{D}$, use $f(\mathbf{s}, \mathbf{a})$ to generate **short** rollouts with policy π_{θ} .
4. Use **both** real and model data to improve π_{θ} via **off-policy** RL.
5. $\pi_0 \leftarrow \pi_{\theta}$.

This algorithm is very similar to **Dyna**, which was an early algorithm that performed model-free RL with a model. It's essentially online Q -learning augmented with a model.

1. Given state s , pick action a using exploratory policy
2. Observe s', r to produce transition (s, a, s', r) .

3. Update model $\hat{p}(s' | s, a)$ and $\hat{r}(s, a)$ using (s, a, s') .
4. Perform Q -update (usual Q-learning update)
5. Repeat K times:
 1. Sample $(s, a) \sim \mathcal{B}$, buffer of past states and actions
 2. Perform Q -update

This inspired the general "Dyna-style" class of methods.

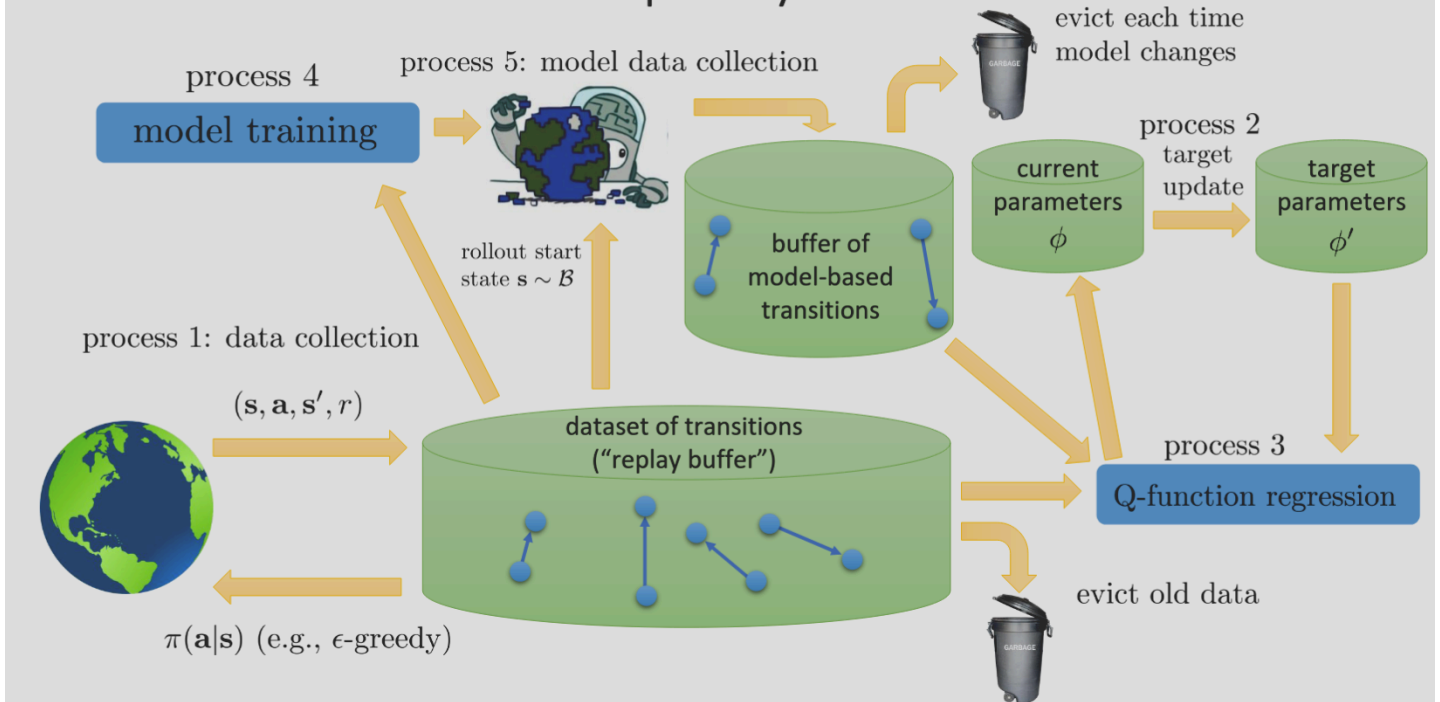
1. Collect transition data of (s, a, s', r') .
2. Learn model $\hat{p}(s' | s, a)$, possibly $\hat{r}(s, a)$ too.
3. Repeat K times:
 1. Sample $s \sim \mathcal{B}$, the buffer
 2. Choose action a (from $\mathcal{B}$, π , or at random)
 3. Simulate $s' \sim p(s' | s, a)$, $r = \hat{r}(s, a)$ too if relevant.
 4. Train on (s, a, s', r') with model-free RL.
 5. (Optional) take N more model-based steps to produce short rollout

Note how our algorithm falls into this Dyna-style class.

We can generalize these algorithms further as **model-accelerated off-policy RL**. We can interpret our methods as a composition of several independent processes, similar to Q -learning (see [Lecture 8](#)). Note also that we may tune the relative speeds of these processes to produce algorithms with different levels of sample efficiency, learning speed, etc.

1. Data collection from real world
2. Target update
3. Q -function regression
4. Dynamics model training
5. Data collection from model

Model-accelerated off-policy RL



Some actual algorithms that follow this framework:

- Model-Based Acceleration (MBA, Gu et al.)
- Model-Based Value Expansion (MVE, Feinberg et al.)
- Model-Based Policy Optimization (MBPO, Janner et al.)

Representing the Model

Now, let's discuss the *architecture* of the dynamics model itself.

Latent State Space Models

Let's consider a partially observed MDP. There is, naturally, some latent state space underlying this MDP. Oftentimes, we'd like to learn this underlying state space, primarily because it may be low-dimensional (relative to the observation space) or it may otherwise simply be easier to run RL on. In particular, we'll be using **VAEs** (see [Lecture 12](#)), which, if you'll recall, attempt to model a latent space $p(\mathbf{z})$ underlying a distribution $p(\mathbf{x})$.

In our case, $\mathbf{x} = (\mathbf{o}_1, \dots, \mathbf{o}_H)$ and $\mathbf{z} = (\mathbf{s}_1, \dots, \mathbf{s}_H)$. In order to use a VAE, we need to determine our *prior* $p(\mathbf{z})$. Well, we know that the true distribution $p(\mathbf{z})$ is defined as

$$p(\mathbf{z}) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

We can choose $p(\mathbf{s}_1)$ to be any (typically simple) distribution, e.g. $\mathcal{N}(0, I)$. However, the probabilities $p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ are the transition probabilities, and thus must be learned—this is the job of our dynamics model.

We also need to determine our *decoder* $p_\theta(\mathbf{x} | \mathbf{z})$. Well, based on the structure of the POMDP,

$$p_\theta(\mathbf{x} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{s}_t)$$

as the observation at time step t is independent of all other states besides the state at time step t (due to the Markovian nature of the states). The separation of the different time steps usually means our decoder is some *neural network*.

Finally, we need to determine our *encoder* $q_\phi(\mathbf{z} \mid \mathbf{x})$. There are actually many different choices, but one choice is to factor the encoder over time steps (known as a *filtering posterior*):

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \prod_t q_\phi(\mathbf{s}_t \mid \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$$

Since the encoder takes in all previous observations and actions, and outputs a distribution over states, it's usually some *sequence model*, e.g. a transformer.

Let's now put these together. We have the following models, which represent our POMDP.

- $p(\mathbf{o}_t \mid \mathbf{s}_t)$: observation model (decoder)
- $p(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$: dynamics model
- $p(r_t \mid \mathbf{s}_t, \mathbf{a}_t)$: reward model

And we train our latent space model with

$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \mathbb{E}_q [\log p_\phi(\mathbf{s}_{t+1,i} \mid \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_\phi(\mathbf{o}_{t,i} \mid \mathbf{s}_{t,i})] + \mathcal{H}(q_\psi(\mathbf{s}_t, \mathbf{s}_{t+1} \mid \mathbf{o}_{1:H,i}, \mathbf{a}_{1:H,i}))$$

which is just produced by substituting into our previous VAE equations. Note that the expectation is performed w.r.t. $(\mathbf{s}_t, \mathbf{s}_{t+1}) \sim q_\psi$.

Choice of Encoder

There's some interesting analysis to be done, though, with regards to our choice of encoder. Recall that we previously chose a filtering posterior

$$q_\psi(\mathbf{z} \mid \mathbf{x}) = \prod_t q_\psi(\mathbf{s}_t \mid \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$$

which is a good, simple posterior, but has a major limitation: it treats the distributions of $\mathbf{s}_t$ and $\mathbf{s}_{t+1}$ as independent, and since the expectation is taken w.r.t. $(\mathbf{s}_t, \mathbf{s}_{t+1}) \sim q_\psi$, we are essentially computing the expectation with respect to *all pairs* of states $(\mathbf{s}_t, \mathbf{s}_{t+1})$ sampled from our model! In reality, though, each $\mathbf{s}_t$ is only likely to transition to a much smaller subset of possible $\mathbf{s}_{t+1}$'s.

This gives rise to a common problem in variational inference known as *posterior collapse*, in which insufficient dependencies between the latent variables induces erroneously *low variance*. The idea is that, in order to reconcile the increased independence of the latent variables (which requires more auxiliary "state" to represent) and the "simple" prior, the model becomes overconfident about its latent variable distributions, narrowing the distributions and thus lowering variance. Mathematically, the result is $q_\psi(\mathbf{z} \mid \mathbf{x}) \approx p(\mathbf{z})$.

While this can be problematic, in instances where the observations already capture much of the state, and what's really desired is the use of VAE to *disentangle* the underlying factors of variation of the state and learn a *nicer representation* of the state, this works fine!

However, if this is a limitation for the problem, there are other posteriors we can use. In particular, the *full smoothing posterior*

$$q_\psi(\mathbf{s}_t, \mathbf{s}_{t+1} \mid \mathbf{o}_{1:H}, \mathbf{a}_{1:H})$$

which takes as input the actions and observations over the entire horizon. This is, of course, a more complex model.

Alternatively, we can go to the opposite extreme, if we desire an even simpler model, and use a *single-step encoder*:

$$q_\psi(\mathbf{s}_t \mid \mathbf{o}_t)$$

This is okay if we are in an almost fully observed MDP, i.e. each observation almost entirely capture its corresponding state.

Generally speaking, the filtering posterior is the most *balanced* choice, but the choice of encoder can change depending on the problem. (The single-step encoder and filtering posterior are pretty common).

Simple Latent Space Model Analysis

What if we wanted to make the simplest latent space model? We'll use a single-step encoder, and we will even define $q_\psi(\mathbf{s}_t | \mathbf{o}_t)$ to be deterministic. In other words,

$$q_\psi(\mathbf{s}_t | \mathbf{o}_t) = \delta(\mathbf{s}_t = g_\psi(\mathbf{o}_t)) \implies \mathbf{s}_t = g_\psi(\mathbf{o}_t)$$

for some function g_ψ . (Note that δ is the Dirac delta function, and represents the concentration of all probability mass at a single point).

Now, let's analyze our training equation. First, we can remove our expectation and substitute in g_ψ everywhere $\mathbf{s}$ was used, since q_ψ is now deterministic. Additionally, we may delete the entropy term from the expression since it is effectively useless now; g is a Dirac delta function and thus its entropy (which is technically $-\infty$, FYI) cannot change.

$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log p_\phi(g_\psi(\mathbf{o}_{t+1}, i) | g_\psi(\mathbf{o}_{t,i}, \mathbf{a}_{t,i})) + \log p_\phi(\mathbf{o}_{t,i} | g_\psi(\mathbf{o}_{t,i}))$$

Intuitively, this is saying that, w.r.t ψ and ϕ , we maximize the probability of $g_\psi(\mathbf{o}_{t,i})$ decoding back to $\mathbf{o}_{t,i}$ (second term) and the probability that g_ψ obeys the learned dynamics (first term), over all trajectories and time steps. This should make sense!

Of course, without any uncertainty or handling of partial observability, this only really works when you just desire a compact representation/encoding of your state.

Info

This is known as an **autoencoder**, without the "variational" because all stochasticity has been removed from the model.

Tip

This is just an autoencoder. To make it into an actual model-based RL objective, we just add a reward model $\log p_\phi(r_{t,i} | g_\psi(\mathbf{o}_{t,i}))$ term to the end of our objective.

Actor-Critic with Learned Representations

We can add these latent state space models to our existing model-free methods, like actor-critic, to produce an actual algorithm.

1. Get $(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a} | \mathbf{o})$, store in replay buffer $\mathcal{R}$.
2. Update the model: $p_\phi(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t | \mathbf{s}_t)$, $p(\mathbf{o}_t | \mathbf{s}_t)$, $q_\psi(\mathbf{s}_t | \mathbf{o}_t)$ using a batch $\sim \mathcal{R}$.
(dynamics, reward, decoder, and encoder models, respectively)
3. Infer $\mathbf{s}_i \sim q_\psi(\mathbf{s}_i | \mathbf{o}_i)$ and $\mathbf{s}'_i \sim q_\psi(\mathbf{s}'_i | \mathbf{o}'_i)$. (Note that $\mathbf{s}_i, \mathbf{s}'_i$ may be encoded jointly if using a full smoothing posterior).
4. Evaluate target value $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' | \mathbf{o}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')].$
5. Approximate $\nabla_\theta J(\theta)$ with e.g. reparameterized policy gradient.
6. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta).$

Actor-Critic with Model-Based RL

Returning to the ideas in [Policy Learning with Models](#), we can produce an actor-critic algorithm that uses the dynamics model to simulate more data.

1. Get $(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{o})$, store in replay buffer $\mathcal{R}$.
2. Update the model: $p_\phi(\mathbf{s}_{t+1} \mid \mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t \mid \mathbf{s}_t)$, $p(\mathbf{o}_t \mid \mathbf{s}_t)$, $q_\psi(\mathbf{s}_t \mid \mathbf{o}_t)$ using a batch $\sim \mathcal{R}$.
(dynamics, reward, decoder, and encoder models, respectively)
3. Infer $\mathbf{s}_i \sim q_\psi(\mathbf{s}_i \mid \mathbf{o}_i)$ and $\mathbf{s}'_i \sim q_\psi(\mathbf{s}'_i \mid \mathbf{o}'_i)$. (Note that $\mathbf{s}_i, \mathbf{s}'_i$ may be encoded jointly if using a full smoothing posterior).
 1. Simulate additional data with p_ϕ .
 2. Evaluate target value $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}' \mid \mathbf{o}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]$.
 3. Update ϕ with $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
 4. Approximate $\nabla_\theta J(\theta)$ with e.g. reparameterized policy gradient.
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

Lecture 17: Offline Reinforcement Learning

Introduction

Recall on-policy and off-policy RL. In both classes of algorithms, data is collected by running a policy in the real-world/simulator (off-policy RL collects more data to add to the replay buffer). **Offline RL** is somewhat similar to off-policy RL, except the replay buffer is *static*; no additional data is collected during training.

First off, how is this even possible? RL is seemingly centered around this "trial and error" approach that allows a model to test policies in the real-world and subsequently optimize. Intuitively, it's because

- Offline RL finds the "good behaviors" in a static dataset
- Good behavior in one region may suggest good behavior in another region
- Components of good behavior can be "stitched" together to produce more complex, good behavior

Notably, offline RL also differs from *imitation learning* in that the dataset may not consist of optimal trajectories! Offline RL makes no assumption about the quality of the dataset.

Distributional Shift

First, we will assume our dataset $\mathcal{D}$ is generated by some policy π_β , and that we are maximizing the usual, discounted RL objective w.r.t. policy π_θ

$$\max_{\theta} \sum_{t=0}^H \mathbb{E}_{\mathbf{s}_t, \mathbf{a}_t \sim \pi_\theta(\mathbf{a}|\mathbf{s})} [\gamma^t r(\mathbf{s}_t, \mathbf{a}_t)]$$

Unfortunately, running any of our usual model-based or model-free RL algorithms will run into serious **distributional shift** problems between our behavior policy and our target policy. This is actually a "surface symptom" of a deeper, more fundamental problem: **counterfactual queries**. Counterfactual questions ask about "what if" scenarios; in RL, this means determining how good some action is when it's never been seen before in training data. With online RL, this is resolved by simply trying out the action; in offline RL, this is not possible. So, how do we effectively *generalize* in offline RL to unseen data?

Intuitively, distributional shift occurs because RL methods inherently *maximize* when producing some policy evaluation or policy prediction function; this encourages maximization even over the *errors* of the model, like what we saw in [Lecture 8](#) with Q -learning. (If minimizing a function, e.g. loss, still maximizing error; just in the other direction). Moreover, techniques like *importance sampling* in off-policy RL methods that correct for the divergence between distributions typically incur *high variance* (for importance sampling, this is due to the multiplication of many importance ratios).

However, off-policy RL methods like **PPO** typically include some term in the objective function that *constrains distribution divergence*, i.e. a KL divergence term. This is known as a **policy constraint**, and, as we'll soon see, some type of policy constraint exists in practically all offline RL methods.

| In short, the existing challenges with sampling error and function approximation in standard RL become more severe in offline RL.

Policy Constraints

First, there are a few principles for offline RL that, empirically, have proved effective.

- Use value-based/model-based methods.
- Somehow fix distributional shift

- Policy constraints
- Pessimism
- Avoid out-of-distribution actions in updates

A simple policy-constrained offline RL algorithm can be derived just from a standard off-policy policy gradient.

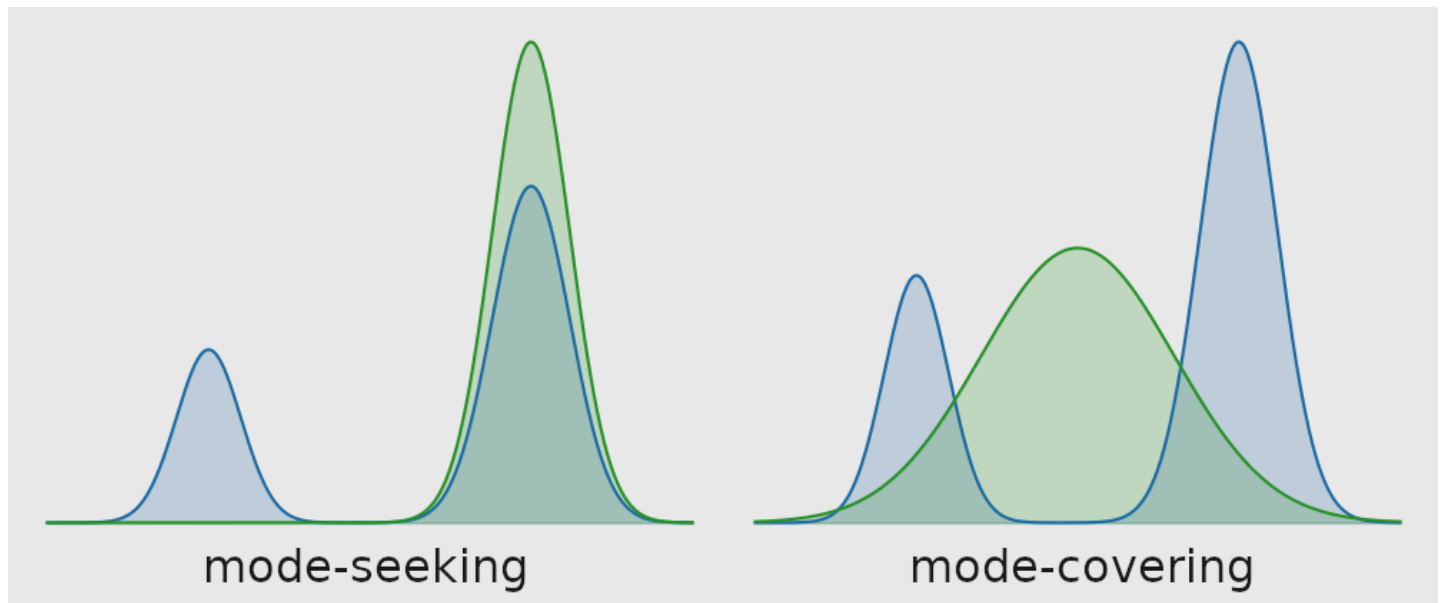
$$J(\theta) = \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_t^{(i)})} [Q^{\pi}(\mathbf{s}_t^{(i)}, \mathbf{a}_t)] + \alpha D_{\text{KL}}(\pi_{\beta}(\mathbf{a}_t | \mathbf{s}_t^{(i)}) \parallel \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t^{(i)}))$$

Actually, we can also use the reverse KL divergence, which swaps π_{β} and π_{θ} , instead of the forward KL divergence. Is there a difference? Well, the two are, mathematically,

$$\begin{aligned} D_{\text{KL}}(\pi_{\beta} \parallel \pi_{\theta}) &= -\mathbb{E}_{\mathbf{a} \sim \pi_{\beta}(\mathbf{a} | \mathbf{s})} [\log \pi_{\theta}(\mathbf{a} | \mathbf{s})] - \mathcal{H}(\pi_{\beta}(\mathbf{a} | \mathbf{s})) \\ D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\beta}) &= -\mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s})} [\log \pi_{\beta}(\mathbf{a} | \mathbf{s})] - \mathcal{H}(\pi_{\theta}(\mathbf{a} | \mathbf{s})) \end{aligned}$$

Consider the first equation, i.e. forward KL divergence. First, note that the entropy term is essentially just an additive constant, since changing θ does not change the entropy of the distribution π_{β} . For any sample that has a probability of 0 under π_{θ} and a nonzero probability under π_{β} , the overall expression diverges to $-\infty$; in other words, an extremely harsh penalty for assigning a probability of 0 to any sample in π_{β} . Meanwhile, it doesn't significantly encourage the assignment of higher probabilities to any samples; thus, this induces the assignment of moderate probabilities to most actions. This leads to a phenomenon known as **mode covering**, in which π_{θ} generates a broader distribution to cover all the data.

In contrast, reverse KL divergence encourages π_{θ} to choose actions that are high probability in π_{β} , and doesn't strongly penalize for perhaps missing out on other actions. And, for any sample that has a probability of 0 under π_{β} , the overall expression diverges to $-\infty$; in other words, an extremely harsh penalty for assigning a nonzero probability to any sample with 0 probability under π_{β} . This encourages **mode seeking**, in which π_{θ} generates a narrower distribution to cover a specific, high-probability mode of the distribution.. An illustration of mode seeking vs. mode covering is shown below.



Why choose one over the other?

- Forward KL
 - Avoids reward hacking
 - Good if we don't know π_{β} (reuse samples)
 - Good if we don't want to lose some modes (e.g. LLM)
- Reverse KL

- Avoids unobserved actions
- Good if you want the "best mode inside data"
- Good with max-entropy RL
- Requires estimating π_β

Typically, though, **forward KL** is chosen because it's simply much easier to implement!

As a sidenote, one may use non-KL-based policy constraints, e.g.

$$D_{\text{support}}(\pi_\theta(\mathbf{a} \mid \mathbf{s}), \pi_\beta(\mathbf{a} \mid \mathbf{s})) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s})} [\delta(\pi_\beta(\mathbf{a} \mid \mathbf{s}) = 0)]$$

which essentially allows assigning probability in π_θ to only states seen in π_β , and has no additional term to encourage entropy in the distribution. Due to practical challenges (difficult to approximate tractably), though, such a constraint has seen limited use.

Lecture 18: Offline RL Algorithms

Policy Constraint Methods

Explicit Policy Constraint Methods

To add policy constraints to, e.g., Q -function actor-critic, we can modify the **actor objective**

$$\begin{aligned}\theta &\leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a}_i)] + \lambda \log \pi_{\theta}(\mathbf{a}_i | \mathbf{s}_i) \\ \theta &\leftarrow \arg \max_{\theta} \sum_{\mathbf{s}_i \in \mathcal{D}} \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)} [Q(\mathbf{s}_i, \mathbf{a}_i) + \lambda \log \pi_{\beta}(\mathbf{a} | \mathbf{s}_i)] + \lambda \mathcal{H}(\pi_{\theta}(\mathbf{a} | \mathbf{s}_i))\end{aligned}$$

for the forward and reverse KL divergence, respectively. Generally, this is not the best way to add policy constraints, but it can work well if done right.

We can also instead modify the **reward function**. Typically, this is done with reverse KL + MaxEnt RL.

$$\begin{aligned}\hat{r}(\mathbf{s}, \mathbf{a}) &= r(\mathbf{s}, \mathbf{a}) - \lambda D_{\text{KL}} \\ &= r(\mathbf{s}, \mathbf{a}) + \lambda \log \pi_{\beta}(\mathbf{a} | \mathbf{s})\end{aligned}$$

This is particularly interesting because it also accounts for *future divergence*.

Now, we can construct some real offline actor-critic algorithms.

Here's a **BRAC-like**, behavior regularized actor critic, algorithm. (Behavior Regularized Offline Reinforcement Learning, Wu et al.)

1. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}' | \mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}'_i, \mathbf{a}_i) - \lambda D(\pi_{\theta}(\cdot | \mathbf{s}'_i), \pi_{\beta}(\cdot | \mathbf{s}'_i))]$. (modifying reward)
2. Update ϕ using $\nabla_{\phi} \sum_{i=1}^B \|\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
3. $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})] - \lambda D(\pi_{\theta}(\cdot | \mathbf{s}_i), \pi_{\beta}(\cdot | \mathbf{s}_i))$. (modifying actor objective)
4. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$.

And here's an **AC+BC-like**, actor critic + behavioral cloning?, algorithm.

1. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}' | \mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}'_i, \mathbf{a}_i)]$.
2. Update ϕ using $\nabla_{\phi} \sum_{i=1}^B \|\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
3. $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})] + \lambda \log \pi_{\theta}(\mathbf{a}_i | \mathbf{s}_i)$. (modifying actor objective with forward KL)
4. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$.

Implicit Policy Constraint Methods



Info

From papers like Advantage-Weighted Regression (Peng et al.) and Accelerating Online Reinforcement Learning with Offline Datasets (Nair et al.). (Both advised by Levine orz).

Implicit policy constraint methods do not add an explicit regularizer term.

Consider, for instance, finding the optimal policy π_{new} over expectation of a Q -function, subject to a reverse KL divergence constraint.

$$\pi_{\text{new}} = \arg \max_{\pi} \mathbb{E}_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})], D_{\text{KL}}(\pi \parallel \pi_{\beta}) \leq \epsilon$$

Via classical convex optimization techniques, we can derive an exact, closed-form answer. (Note: this is usually not practical due to the size of the action space).

$$\pi^*(\mathbf{a} | \mathbf{s}) = \frac{1}{Z(\mathbf{s})} \pi_{\beta}(\mathbf{a} | \mathbf{s}) \exp \left(\frac{1}{\lambda} A^{\pi}(\mathbf{s}, \mathbf{a}) \right)$$

Now, the idea is that we can use **importance sampling** to effectively approximate samples from π^* using samples from π_{β} , and then use **behavioral cloning** (supervised learning) to learn from the samples of π^* . In particular, we approximate via *importance-weighted maximum likelihood*, i.e.

$$\pi_{\text{new}}(\mathbf{a} | \mathbf{s}) = \arg \max_{\pi} \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim \pi_{\beta}} \left[\log \pi(\mathbf{a} | \mathbf{s}) \underbrace{\frac{1}{Z(\mathbf{s})} \exp \left(\frac{1}{\lambda} A^{\pi_{\text{old}}}(\mathbf{s}, \mathbf{a}) \right)}_{w(\mathbf{s}, \mathbf{a})} \right]$$

This may translate to an **AWAC-like**, advantage-weighted actor-critic, algorithm. (Nair et al.)

1. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_{\theta}(\mathbf{a}'|\mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}'_i, \mathbf{a}_i)]$.
2. Update ϕ using $\nabla_{\phi} \sum_{i=1}^B \|\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
3. Update ψ using $\nabla_{\psi} \sum_{i=1}^B \|\hat{V}_{\psi}^{\pi}(\mathbf{s}_i) - \mathbb{E}_{\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s}_i)} [\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})]\|^2$. (train $\hat{V}_{\psi}^{\pi}$)
4. $J(\theta) = \sum_i \log \pi_{\theta}(\mathbf{a}_i | \mathbf{s}_i) \exp(\hat{Q}_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_{\psi}^{\pi}(\mathbf{s}_i))$.
5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$.

Note that you can also approximate $\hat{V}^{\pi}(\mathbf{s}_i) \approx Q_{\phi}^{\pi}(\mathbf{s}_i, \mathbf{a})$ where $\mathbf{a} \sim \pi_{\theta}(\mathbf{a} | \mathbf{s}_i)$ if you don't want to train a separate neural network to learn $\hat{V}^{\pi}$.

This approach is nice for its simplicity; however, it is limited primarily by the fact that, in copying the samples generated by the behavioral policy, it is slow to learn to avoid bad actions.

Implicit Q -Learning (IQL)

Info

Paper: Offline Reinforcement Learning with Implicit Q-Learning (Kostrikov et al.). (Also Levine).

Here's a thought: what if we simply **avoided all out-of-distribution actions** when performing our Q -update? (Q -update needs to maximize over all actions). The intuition is that neural networks can often generalize very well to unseen actions, without explicit consideration of such actions.

Consider performing policy evaluation of the behavior policy. With a typical MSE loss, we might have

$$Q = \min_Q \sum_{(s, a, s') \in \mathcal{D}} \|Q(s, a) - [r(s, a) + \gamma V(s')]\|^2$$

$$V = \min_V \sum_{(s, a) \in \mathcal{D}} \|V(s) - Q(s, a)\|^2$$

which simply regresses onto the *mean* of the value of each state/action under the behavior policy. However, we can actually regress onto an *upper quantile* of these values instead, by changing our loss function.

$$V = \min_V \sum_{(s,a) \in \mathcal{D}} \ell_\tau(V(s) - Q(s,a))$$

where ℓ_τ is a function that heavily penalizes V underestimating Q , and softly penalizes V overestimating Q . It's essentially just MSE with a multiplier on all negative values in the domain. This essentially induces our function estimator to perform implicit maximization over the actions in estimating the value function.

⚠️ Doesn't this cause erroneous overestimation? Wasn't this an issue in soft actor-critic?

Actually, no. In SAC, overestimation was caused by optimism towards the state transitions. Here, the Q -function estimator still uses standard MSE to effectively regress onto the state transitions. Only the V estimator now uses the modified loss function to implicitly maximize over the actions—which is precisely what we want for an RL algorithm.

So, implicit Q -learning, in practice, uses

$$\begin{aligned} Q(s, a) &\leftarrow r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_{\mathbf{a}' \sim \pi_{\text{new}}} [Q(\mathbf{s}', \mathbf{a}')] \\ V &\leftarrow \arg \min_V \frac{1}{N} \sum_{i=1}^N \ell(V(\mathbf{s}_i) - Q(\mathbf{s}_i, \mathbf{a}_i)) \\ \ell = \ell_2^\tau &= \begin{cases} (1 - \tau)x^2, & x > 0 \\ \tau x^2, & x \leq 0 \end{cases} \end{aligned}$$

where τ is usually near 1.

Note that it is actually possible to show that this is essentially equivalent to Q -learning with Q -updates that maximize only over actions seen in the sample data of the behavioral policy.

Thus, here's an offline-actor critic algorithm with implicit Q -learning.

1. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\psi(\mathbf{s}_i)$.
2. Update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$.
3. Update ψ using $\nabla_\psi \sum_{i=1}^B \ell_2^\tau(\hat{V}_\psi(\mathbf{s}_i) - \hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i))$.
4. $J(\theta) = \sum_i \log \pi_\theta \exp(\hat{Q}_\phi(\mathbf{s}_i, \mathbf{a}_i) - \hat{V}_\psi(\mathbf{s}_i))$.
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

Some interesting things to note:

- No expectations under π_θ .
- Actor and critic are completely independent.

Conservative Q -Learning (CQL)

Info

Paper: Conservative Q-Learning for Offline Reinforcement Learning (Kumar et al.). (Yes, also Levine).

Let's now return to the idea of **pessimism** to solve distributional shift.

Our inspiration is to apply some ideas from *adversarial training*. Let's redefine our Q function as

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \alpha \mathbb{E}_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] + \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} [(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2]$$

where the first term "pushes down" on large Q -values, and the second term is just our regular Q -learning objective. (μ is our discriminator, the policy π is our generator, in GAN terms). Note that one may show that $\hat{Q}^\pi \leq Q^\pi$ for large enough α , or that our $\hat{Q}^\pi$ is a lower bound on the actual Q function.

There's one issue with this solution, though—this will produce a systematic underestimate on actions close to the data. A better estimate is

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \alpha \mathbb{E}_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim D} [Q(\mathbf{s}, \mathbf{a})] + \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} [(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2]$$

This new term essentially just pushes back up on $(\mathbf{s}, \mathbf{a})$ samples in our data, and effectively cancels out with the first term. Notably, we no longer have the guarantee that $\hat{Q}^\pi \leq Q^\pi$ for *all* $(\mathbf{s}, \mathbf{a})$, but we are guaranteed that $\mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})} [\hat{Q}^\pi(\mathbf{s}, \mathbf{a})] \leq \mathbb{E}_{\pi(\mathbf{a}|\mathbf{s})} [Q^\pi(\mathbf{s}, \mathbf{a})]$ for all $\mathbf{s} \in D$.

Thus, we have a basic conservative Q -learning algorithm.

1. Update $\hat{Q}^\pi$ w.r.t. $\mathcal{L}_{\text{CQL}}(\hat{Q}^\pi)$ using $\mathcal{D}$.
2. Update policy π , dependent on discrete/continuous action space.

We've left something out though in our discussion so far—what is this μ term? Typically, we'll actually add a regularization term $\mathcal{R}(\mu)$ to the loss function, i.e.

$$\hat{Q}^\pi = \arg \min_Q \max_\mu \alpha \mathbb{E}_{\mathbf{s} \sim D, \mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] - \alpha \mathbb{E}_{(\mathbf{s}, \mathbf{a}) \sim D} [Q(\mathbf{s}, \mathbf{a})] - \mathcal{R}(\mu) + \mathbb{E}_{(\mathbf{s}, \mathbf{a}, \mathbf{s}') \sim D} [(Q(\mathbf{s}, \mathbf{a}) - (r(\mathbf{s}, \mathbf{a}) + \mathbb{E}_\pi [Q(\mathbf{s}', \mathbf{a}')]))^2]$$

A common choice for $\mathcal{R}(\mu)$ is $\mathbb{E}_{\mathbf{s} \sim D} [\mathcal{H}(\mu(\cdot | \mathbf{s}))]$, or maximum entropy regularization. In this case, after learning, μ becomes proportional to $\exp(Q(\mathbf{s}, \mathbf{a}))$. Notably, we can either represent μ as an actual, explicit learned policy, or we can note that

$$\mathbb{E}_{\mathbf{a} \sim \mu(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] = \log \sum_{\mathbf{a}} \exp(Q(\mathbf{s}, \mathbf{a}))$$

For discrete actions, we can calculate this quantity directly, while for continuous actions, we can use importance sampling to estimate this quantity.

? What if you didn't use a regularizer?

If you didn't add a regularizer $\mathcal{R}(\mu)$, the maximizing μ is a distribution that assigns probability 1 to the highest Q -value. Because this may change between every iteration, it is potentially unstable, and therefore slows learning. With maximum entropy regularization, the distribution is encouraged to spread out a bit more over the large Q -values.

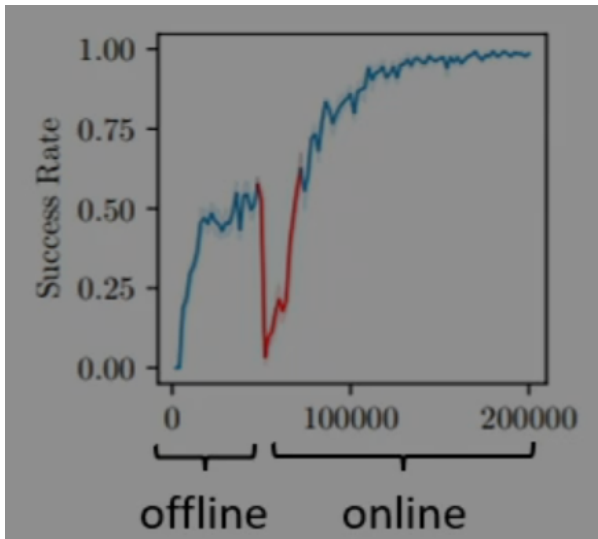
Offline-to-online RL

Now, let's turn to the problem of offline-to-online RL: using offline RL to pretrain a model, and then using online RL to fine-tune the model.

⚠ At the time of writing (March 2026), this is an active area of research. Take all following discussions with a grain of salt.

Okay, so what if we just run an offline RL algorithm, e.g. CQL, and then run a standard online RL algorithm starting from the same point (same policy, same Q -function)? Does that just work?

Unfortunately, not quite. See below.



For CQL, in particular, which this graph was generated from, during the first phase of online training, the model will realize that it has drastically underestimated the Q -function for some previously unobserved actions, due to the pessimism of CQL. Other offline RL algorithms have similar, slightly different versions of this same problem.

The problem is the differences between the training phases.

Offline:

- Stay close to π_β .
- Conservative values (e.g. $\hat{Q}^\pi \leq Q^\pi$).
- Lots of gradient steps on the same data.

Online:

- Improve as much as possible over π_β .
- Optimistic values (e.g. $\hat{Q}^\pi \geq Q^\pi$). (important for exploration)
- Learn as quickly as possible.

There is, notably, an "embarrassingly effective" method that is not really offline RL at all, yet outperformed, for a long time, many offline RL methods in using offline data for online methods. (Efficient Online Reinforcement Learning with Offline Data, Ball et al.).

1. Initialize two buffers: online replay buffer and offline data buffer
2. Initialize value function and actor from scratch.
3. Run online RL; for every batch, sample half from offline buffer and half from replay buffer.

Obviously, this method is unsatisfying because it involves absolutely zero pretraining. Unfortunately, there isn't a clear solution to this; however, in recent years, algorithms that use *diffusion* models or *flow matching* to represent the actor have empirically produced methods with pretraining that do improve over the "embarrassingly effective" method.

It's unknown why exactly they work well, but here is one theory. In the online case, optimal $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ is deterministic; thus, there's no need to capture some multimodal distribution for the policy. In the offline case, capturing only the best mode in the data is fine... but when capturing multiple modes, it may be easier to handle policy constraints. So, for offline-to-online, perhaps it helps to just track all the modes in the offline phase, and then focus on the best mode in the online phase.

However, it's **hard to use diffusion as the actor** in RL. Optimizing the objective requires either $\nabla_\theta \log \pi_\theta(\mathbf{a} \mid \mathbf{s})$ (policy gradient) or backpropagation through the diffusion process (reparameterization). Policy gradient is inaccessible for diffusion/flow matching, and backprop is often computationally expensive and unstable (backprop through time, **BPTT**).

Let's discuss some methods that have used diffusion models, and what they did to solve the above issue.

IDQL

Simple offline-to-online RL with diffusion model. (IDQL: Implicit Q-Learning as an actor-critic method with diffusion policies, Hansen-Estruch et al.)

1. Train $\hat{Q}_\phi(\mathbf{s}, \mathbf{a})$ without any actor (e.g. IQL)
2. Train $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ as a diffusion/flow model with behavioral cloning, leads to $\pi_\theta \approx \pi_\beta$.
3. At test time, sample $\{\mathbf{a}_1, \dots, \mathbf{a}_K\}$ from $\pi_\theta(\mathbf{a} \mid \mathbf{s})$, and pick $\arg \max_{\mathbf{a}_k} \hat{Q}_\phi(\mathbf{s}, \mathbf{a}_k)$.

The point is that no out-of-distribution actions are chosen because $\pi_\theta \approx \pi_\beta$. This is somewhat reminiscent of stochastic optimization/random shooting from [Lecture 16](#). However, this works surprisingly well, particularly if the data produced by π_β is somewhat decent.

FQL

An actor that stays "close" to diffusion model. (Flow Q-Learning, Park et al.)

1. Train $\pi_{\text{flow}}(\mathbf{a} \mid \mathbf{s}, \mathbf{z})$ as diffusion/flow model with behavioral cloning, where $\mathbf{z}$ is the input noise in flow matching. This leads to $\pi_{\text{flow}} \approx \pi_\beta$.
2. Run offline actor-critic with a special behavioral cloning regularizer, training the actor $\pi_\theta(\mathbf{a} \mid \mathbf{s}, \mathbf{z})$ to stay close to $\pi_{\text{flow}}(\mathbf{a} \mid \mathbf{s}, \mathbf{z})$, given both $\mathbf{s}$ and $\mathbf{z}$ as input. Notably, the actor is just a regular neural network, not a flow model, that is essentially **distilling** the behavior of the flow model. In particular, the objective for the actor is

$$J(\theta) = \sum_i \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a} \mid \mathbf{s}_i, \mathbf{z})} [\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})] + \mathbb{E}_{\mathbf{a} \sim \pi_{\text{flow}}(\mathbf{a} \mid \mathbf{s}, \mathbf{z})} [\lambda \log \pi_\theta(\mathbf{a}_i \mid \mathbf{s}, \mathbf{z})]]$$

DSRL

Diffusion Steering via Reinforcement Learning. (Steering Your Diffusion Policy with Latent Space Reinforcement Learning, Wagenmaker et al.)

Intuitively, a diffusion model actually produces an action space of **only in-distribution** actions. So, what if, instead of our actor model producing an action from the general action space, which may be out-of-distribution, we use our actor model to produce a value in the **latent space** of the diffusion model, i.e. the noise distribution, and then feed that into the diffusion model to produce an in-distribution action. That is, just run an efficient online RL algorithm in the latent space of a diffusion model!

Model-based Offline RL

The critical concern in model-based offline RL is that we'd like to limit the impact of the distributional shift of the environment model itself, because we are unable to collect more data to correct the model's errors. (**counterfactual** questions for the model, rather than for the Q -function like previously).

MOPO

MOPO: Model-Based Offline Policy Optimization (Yu et al.)

Also, MOREL: Model-Based Offline Reinforcement Learning (Kidambi et al.)

One solution is to simply adjust the reward function to be pessimistic about OOD states, i.e. to "*punish*" the policy for exploiting OOD states. The reward is adjusted as follows

$$\tilde{r}(s, a) = r(s, a) - \lambda u(s, a)$$

where $u(s, a)$ represents an **uncertainty penalty**. This can be computed by e.g. measuring disagreement between ensemble models. Subsequently, simply run any existing model-based RL method.

COMBO

COMBO: Conservative Offline Model-Based Policy Optimization (Yu et al.)

Alternatively, we can leverage the same ideas as in **CQL**. Similar to how CQL "pushes down" on large Q -values, model-based RL can "push down" on the Q -values of model state-action tuples. In essence, for a model p , we use a Q -function update rule of

$$\hat{Q}^{k+1} \leftarrow \arg \min_Q \beta (\mathbb{E}_{\mathbf{s}, \mathbf{a} \sim p(\mathbf{s}, \mathbf{a})} [Q(\mathbf{s}, \mathbf{a})] - \mathbb{E}_{\mathbf{s}, \mathbf{a} \sim \mathcal{D}} [Q(\mathbf{s}, \mathbf{a})]) + \frac{1}{2} \mathbb{E}_{\mathbf{s}, \mathbf{a}, \mathbf{s}' \sim d_T} [(Q(\mathbf{s}, \mathbf{a}) - \hat{\beta}^\pi \hat{Q}^k(\mathbf{s}, \mathbf{a}))^2]$$

Again, the intuition is the same. It's like a GAN: if the model, the generator, produces something that looks clearly different from real data, the Q -function, the discriminator, will assign low values to it.

Lecture 19: Exploration

The Exploration Problem

The problem lies in the fact that the intended objective may *correlate weakly* with rewarding events, depending on the reward function design. This issue is particularly exacerbated for *temporally extended* tasks, in which the model, before discovering any trajectory that achieves the objective, may be stuck stumbling through seemingly random or odd trajectories for a while.

Ultimately, the exploration problem poses the following (essentially equivalent) questions:

- How can an agent discover high-reward strategies that require a temporally extended sequence of complex behaviors that are individually not rewarding?
- How can an agent decide whether to attempt new behaviors (**exploration**) or continue doing the best thing it knows so far (**exploitation**)?

That begs the question—can we derive an optimal exploration strategy? And, for that purpose, what does "optimal exploration" even mean?

The Tractability of Exploration

RL problems lie on a spectrum of *theoretical tractability* with regards to the derivation of an optimal exploration strategy. Thus, in practice, an optimal strategy is derived on a much simpler task (e.g. multi-arm bandits) and heuristically applied to a more complex task (e.g. modern deep RL problems).

For the easiest situations, multi-arm bandits and contextual bandits, we can frame exploration as *POMDP identification*. For small, tabular MDPs, we can frame exploration as Bayesian model identification. For large or infinite MDPs, optimal methods are essentially impossible to find/apply, so we take inspiration from simpler problems.

Bandits

See [these notes](#) from [Reinforcement Learning: An Introduction](#) by Barton and Sutton.

Note that, because the bandits problems forms a POMDP with the observations as the rewards and the states as the latent reward distribution of each bandit, it's possible to simply run policy gradient to find our optimal policy. This is a bit overkill, though :)

Measurement of Exploration Optimality

We can measure how good an exploration algorithm with **regret**: the difference in cumulative reward from the optimal policy at time step H .

$$\text{Reg}(H) = H\mathbb{E}[r(a^*)] - \sum_{t=1}^H r(a_t)$$

Optimistic Exploration with Bandits

Let $\hat{\mu}_a$ be the average reward for each action a . An exploitative strategy would involve *greedily* choosing $a = \arg \max \hat{\mu}_a$. However, this precludes exploration. Instead, by using an *optimistic estimate* of an action's value, i.e. $a = \arg \max \hat{\mu}_a + C\sigma_a$, where σ_a is some sort of variance estimate/measure of uncertainty. A common choice of that second term is $\sqrt{\frac{2 \ln H}{N(a)}}$, where N is the number of times a has been picked so far. This is the same as **UCB action selection** from the RL book notes. With this choice, $\text{Reg}(H) \in O(\log H)$, which is provably optimal!

Optimistic Exploration in Deep RL

As in UCB, we'd like to apply some sort of "exploration bonus" to actions less taken/actions we are more uncertain about.

For MDPs, we can use a similar **count-based exploration** method, i.e. measuring uncertainty inversely with $N(\mathbf{s}, \mathbf{a})$ or $N(\mathbf{s})$, a measure of visitations to that state-action pair/state. We can modify our reward function to be

$$r^+(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \mathcal{B}(N(\mathbf{s}))$$

where $\mathcal{B}$ is some function inversely proportional to $N(\mathbf{s})$. So, we can just augment any model-free RL algorithm with this reward function to promote exploration... right?

Unfortunately, the issue with this count-based exploration method for deep RL problems is inherently the *massive* state/action space. For instance, consider an RL problem that uses images as state; literally counting the number of visits to each image is... probably not smart considering the number of possible images. It gets even worse if you consider a continuous task with states represented by floating point numbers! However, we know that some states are more similar than others... can we use this notion of "*familiarity*" to promote exploration of only states for which we have not seen similar states?

Well, what if we fit a **generative model** to our replay buffer of observed states? Then, we can directly quantify the familiarity of a state by simply evaluating the probability density model on it.

However, we'd also like to convert the probability density of a state to a "*pseudo-count*." We can note that the true probability density is represented by

$$P(s) = \frac{N(\mathbf{s})}{n}$$

where $N(s)$ is the number of visits to state s and n is the total number of states visited. Then, after observing a visit to $\mathbf{s}$, the probability becomes

$$P'(\mathbf{s}) = \frac{N(\mathbf{s}) + 1}{n + 1}$$

So we'd like our density model to obey these rules, because then we can use the probability density estimates to solve for the counts.

The paper Unifying Count-Based Exploration by Bellemare et al. describes a solution.

1. Fit model $p_\theta(\mathbf{s})$ to $\mathcal{D}$.
2. Take step i and observe $\mathbf{s}$.
3. Fit new model $p_{\theta'}(\mathbf{s})$ to $\mathcal{D} \cap \{\mathbf{s}_i\}$.
4. Use $p_\theta(\mathbf{s}_i)$ and $p_{\theta'}(\mathbf{s}_i)$ to estimate $\hat{N}(\mathbf{s})$.
5. Set $r_i^+ = r_i + \mathcal{B}(\hat{N}(s))$.

Where we estimate $\hat{N}(\mathbf{s})$ by solving the system of equations

$$p_\theta(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i)}{\hat{n}} \quad p_{\theta'}(\mathbf{s}_i) = \frac{\hat{N}(\mathbf{s}_i) + 1}{\hat{n} + 1}$$

with unknown $\hat{N}(\mathbf{s}_i)$ and $\hat{n}$ turning out to be

$$\hat{N}(\mathbf{s}_i) = \hat{n} p_\theta(\mathbf{s}_i) \quad \hat{n} = \frac{1 - p_{\theta'}(\mathbf{s}_i)}{p_{\theta'}(\mathbf{s}_i) - p_\theta(\mathbf{s}_i)} p_\theta(\mathbf{s}_i)$$

One final consideration: what bonus function $\mathcal{B}$ to use? There are a lot that work fine, just make sure it's *inversely proportional* to $N(s)$. Here are a couple examples from literature.

- UCB: $\mathcal{B}(N) = \sqrt{\frac{2 \ln n}{N}}$.

- MBIE-EB (Strehl & Littman): $\mathcal{B}(N) = \sqrt{\frac{1}{N(s)}}$.
- BEB (Kolter & Ng): $\mathcal{B}(N) = \frac{1}{N(s)}$.

Heuristic Estimation of Counts via Errors

Using a generative model $p_\theta(\mathbf{s})$ is great, but, in practice, you may not even need a complex model. In reality, we primarily require $p_\theta(\mathbf{s})$ to, not necessarily be a true density model, but to simply be large for familiar states and small for unfamiliar states. We can develop some simple heuristics for this purpose!

Consider a target function $f^*(\mathbf{s}, \mathbf{a})$. Given a buffer $\mathcal{D} = \{(\mathbf{s}_i, \mathbf{a}_i)\}$, attempt to fit a function $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$ to model $f(\mathbf{s}, \mathbf{a})$. In particular, we expect that $\hat{f}_\theta$, to some extent, will **overfit** to the dataset, producing near-zero error for any $\mathbf{s}, \mathbf{a} \sim \mathcal{D}$ and some non-negligible error for unseen states. Thus, we can actually use the error of our model, e.g. $\mathcal{E}(\mathbf{s}, \mathbf{a}) = \|\hat{f}_\theta(\mathbf{s}, \mathbf{a}) - f^*(\mathbf{s}, \mathbf{a})\|^2$ as a bonus instead!

There are a number of choices for $f^*(\mathbf{s}, \mathbf{a})$. One common choice is $f^*(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$, i.e. next-state prediction. An even simpler choice is to literally just make f^* a neural network initialized with completely random parameters. The latter choice is known as **RND** (Random Network Distillation, Burda et al.) and is the most widely used deep RL exploration method used today.

Lecture 20: Reinforcement Learning Theory

Why study RL theory?

First, there are some interesting questions asked in RL theory.

- If I use algorithm $\mathcal{A}$ with N samples for k iterations, what is the worst case performance (under some set of assumptions)?
 - For instance, determining the number of iterations k of Q -learning to produce $\|\hat{Q}_k - Q^*\| \leq \epsilon$ with probability $\geq 1 - \delta$ if $N \geq f(\epsilon, \delta)$.
 - Or, determining the number of iterations k of Q -learning such that $\|Q^{\pi_k} - Q^*\| \leq \epsilon$.
- For some exploration algorithm $\mathcal{A}$, how high, in the worst-case, is the regret?
- And many others!

In analyzing RL theory, it's also necessary to make some strong assumptions to achieve any interesting results, while not deviating too far from reality. For...

- ...exploration: how likely are we to find (potentially sparse) rewards? In the worst case, it's extremely hard; thus, there are typically some assumptions made to produce interesting bounds.
- ...learning: how many samples do we need to effectively learn a model/value function? Often, a "generative model" assumption is made that assumes we can sample from $p(s' | s, a)$ for any (s, a) (assumes exploration is not too hard). Alternatively, an "oracle exploration" assumption assumes sampling from $p(s' | s, a)$ for each (s, a) up to N times.

The point of this is not to prove that an algorithm works perfectly every time (in fact, in deep RL, typically not even convergence is guaranteed). However, it does yield interesting conclusions on the effect of problem parameters/hyperparameters on error—RL theory produces *qualitative* conclusions about various factors under *strong assumptions* that are reasonable enough such that they may guide decisions in real RL problems..

Some Useful Identities

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim p(s'|s,a)}[V^\pi(s')] = r(s, a) + \gamma \sum_{s'} p(s' | s, a) V^\pi(s') \implies \\ Q^\pi &= r + \gamma P V^\pi \\ p^\pi(s', a' | s, a) &= \pi(a' | s') p(s' | s, a) \implies \\ Q^\pi &= r + \gamma P^\pi Q^\pi \\ Q^\pi - \gamma P^\pi Q^\pi &= r \\ (\mathbf{I} - \gamma P^\pi) Q^\pi &= r \\ Q^\pi &= (I - \gamma P^\pi)^{-1} r \end{aligned}$$

Note that r is a vector, and Q^π, P, P^π, V^π are all matrices.

That final equation actually gives us a method of recovering the Q -function, but is impractical not just for the typically high dimensionality of the state and action spaces but because we typically do not assume knowledge of P^π (involves state transition probabilities). Notably, though, that last equation tells us that policy evaluation is essentially just a *linear operation*.

Now, say $r \approx 1$. From this, we can note that $\sum_{t=0}^{\infty} \gamma^t r \approx \sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma}$. Thus, the total reward of a trajectory is approximately $\frac{1}{1-\gamma}$, and thus our Q -function matrix Q^π has elements that are roughly $\frac{1}{1-\gamma}$ in magnitude. Alternatively, for a finite horizon problem (and no discounting), we have that the total reward of a trajectory is approximately $\sum_{t=1}^H r \approx H$, and our Q -function matrix Q^π would have elements that are roughly H in magnitude. Therefore, for infinite horizon tasks with discounting, we can roughly term $\frac{1}{1-\gamma}$ as a "*finite horizon*" for the problem.

Convergence of Value Iteration

In linear algebra notation, value iteration is $V \leftarrow \max_a [r + \gamma PV]$. Let's define an operator T such that

$$TV = \max_a [r + \gamma PV]$$

this is known as the **Bellman optimality operator**. Also, let us note the following useful inequality

$$|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|$$

Consider, now, two value functions $V(s)$ and $U(s)$. We'll apply the Bellman optimality operator to both, and consider the difference.

$$\begin{aligned} |TV(s) - TU(s)| &= \left| \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s') \right] - \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) U(s') \right] \right| \\ &\leq \max_a \left| \sum_{s'} p(s' | s, a) (V(s') - U(s')) \right| \\ &\leq \gamma \max_{s'} |V(s') - U(s')| \\ &= \gamma \|V - U\|_\infty \end{aligned}$$

Thus, applying T to any two value functions V and U will "bring them closer" w.r.t. to the infinity norm.

Also, we will note that $TV^* = V^*$, or that the optimal value function does not change by application of the Bellman optimality operator. We will not prove this fact for brevity.

Therefore,

$$\begin{aligned} \|TV - V^*\|_\infty &= \|TV - TV^*\|_\infty \leq \|V - V^*\|_\infty \\ \|T^k V - V^*\|_\infty &\leq \gamma^k \|V - V^*\|_\infty \\ \lim_{k \rightarrow \infty} \|T^k V - V^*\|_\infty &= 0 \end{aligned}$$

And thus value iteration converges!

Sample Complexity without Exploration

First, let's define our algorithm and assumptions. We will assume "oracle exploration," i.e. we can sample $s' \sim p(s' | s, a)$ for each (s, a) up to N times. Our algorithm is a simple "model based" algorithm.

1. $\hat{p}(s' | s, a) = \frac{\#(s, a, s')}{N}$.
2. Given π , use $\hat{p}$ to estimate $\hat{Q}^\pi$.

Then, we'd like to answer the following questions.

1. How close is $\hat{Q}^\pi$ to Q^π ?
2. How close is $\hat{Q}^*$ to Q^* if we learn it using $\hat{p}$?
3. How close is the final policy $Q^{\hat{\pi}^*}$ to Q^* ?

We'll first address question 1, where we define (ϵ, δ) -closeness as $\|Q^\pi(s, a) - \hat{Q}^\pi(s, a)\|_\infty \leq \epsilon$ with probability at least $1 - \delta$ if $N \geq f(\epsilon, \delta)$. Note that we use $\|\cdot\|_\infty$ because it provides worst-case performance bounds.

Concentration Inequalities

Now, relating samples to errors is highly nontrivial. We will need a **concentration inequality**; Perhaps the simplest such inequality is **Hoeffding's inequality**.

Lemma A.1. (Hoeffding's inequality) Suppose $X_1, X_2, \dots, X_n$ are a sequence of independent, identically distributed (i.i.d.) random variables with mean μ . Let $\bar{X}_n = n^{-1} \sum_{i=1}^n X_i$. Suppose that $X_i \in [b_-, b_+]$ with probability 1, then

$$P(\bar{X}_n \geq \mu + \epsilon) \leq e^{-2n\epsilon^2 / (b_+ - b_-)^2}.$$

Similarly,

$$P(\bar{X}_n \leq \mu - \epsilon) \leq e^{-2n\epsilon^2 / (b_+ - b_-)^2}.$$

Intuitively, the interpretation is that if we estimate μ with n samples the probability we're off by more than ϵ is at most $2e^{-2n\epsilon^2 / (b_+ - b_-)^2}$.

Equivalently, if we want this probability bound to be δ , we can show that we require

$$\frac{b_+ - b_-}{\sqrt{2n}} \sqrt{\log \frac{2}{\delta}} \geq \epsilon$$

Importantly, $\epsilon \propto \frac{1}{\sqrt{n}}$.

However, we are estimating sample probabilities, not sample averages. For that, we need a different concentration inequality.

Proposition A.8. (Concentration for Discrete Distributions) Let z be a discrete random variable that takes values in $\{1, \dots, d\}$, distributed according to q . We write q as a vector where $\vec{q} = [\Pr(z = j)]_{j=1}^d$. Assume we have N iid samples, and that our empirical estimate of $\vec{q}$ is $[\hat{q}]_j = \sum_{i=1}^N \mathbf{1}[z_i = j] / N$.

We have that $\forall \epsilon > 0$:

$$\Pr(\|\hat{q} - \vec{q}\|_2 \geq 1/\sqrt{N} + \epsilon) \leq e^{-N\epsilon^2}.$$

which implies that:

$$\Pr(\|\hat{q} - \vec{q}\|_1 \geq \sqrt{d}(1/\sqrt{N} + \epsilon)) \leq e^{-N\epsilon^2}.$$

From this, we derive

$$\delta \leq \frac{1}{\sqrt{N}} \sqrt{\log \frac{1}{\delta}}$$

and we may apply this to our problem and write

$$\begin{aligned} \|\hat{p}(s' | s, a) - p(s' | s, a)\|_1 &\leq \sqrt{|S|} \left(\frac{1}{\sqrt{N}} + \epsilon \right), \text{ with probability } 1 - \delta \\ &= \sqrt{\frac{|S|}{N}} + \sqrt{\frac{|S| \log \frac{1}{\delta}}{N}} \\ &\leq c \sqrt{\frac{|S| \log \frac{1}{\delta}}{N}} \end{aligned}$$

for some constant c .

Useful Lemmas

We'd like to now relate the error of $\hat{p}$ to the error of $\hat{Q}^\pi$. We can note that

$$\begin{aligned}
Q^\pi &= r + \gamma P V^\pi \\
V^\pi &= \Pi Q^\pi \\
P^\pi &= P \Pi
\end{aligned}$$

Note that Π represents the matrix of $\pi(a \mid s)$ for all (s, a) , and P represents the matrix of $p(s' \mid s, a)$ for all (s, a, s') . These directly imply that

$$\begin{aligned}
Q^\pi &= r + \gamma P^\pi Q^\pi \\
Q^\pi &= (I - \gamma P^\pi)^{-1} r
\end{aligned}$$

Comparing this with the previous equation derived for $\hat{Q}^\pi$, we note the similarity

$$\begin{aligned}
Q^\pi &= (I - \gamma P^\pi)^{-1} r \\
\hat{Q}^\pi &= (I - \gamma \hat{P}^\pi)^{-1} r
\end{aligned}$$

Then, we claim the following **simulation lemma** is true.

$$Q^\pi - \hat{Q}^\pi = \gamma(I - \gamma \hat{P}^\pi)^{-1}(P - \hat{P})V^\pi$$

Proof.

$$\begin{aligned}
Q^\pi - \hat{Q}^\pi &= Q^\pi - (I - \gamma \hat{P}^\pi)^{-1} r \\
&= (I - \gamma \hat{P}^\pi)^{-1} (I - \gamma \hat{P}^\pi) Q^\pi - (I - \gamma \hat{P}^\pi)^{-1} r \\
&= (I - \gamma \hat{P}^\pi)^{-1} (I - \gamma \hat{P}^\pi) Q^\pi - (I - \gamma \hat{P}^\pi)^{-1} (I - \gamma P^\pi) Q^\pi \\
&= (I - \gamma \hat{P}^\pi)^{-1} ((I - \gamma \hat{P}^\pi) Q^\pi - (I - \gamma P^\pi) Q^\pi) \\
&= \gamma (I - \gamma \hat{P}^\pi)^{-1} (P^\pi - \hat{P}^\pi) Q^\pi \\
&= \gamma (I - \gamma \hat{P}^\pi)^{-1} (P \Pi - \hat{P} \Pi) Q^\pi \\
&= \gamma (I - \gamma \hat{P}^\pi)^{-1} (P - \hat{P}) \Pi Q^\pi \\
&= \gamma (I - \gamma \hat{P}^\pi)^{-1} (P - \hat{P}) V^\pi
\end{aligned}$$

Another useful lemma is that, for some P^π and any vector $v \in \mathbb{R}^{|S||A|}$, we have that

$$\|(I - \gamma P^\pi)^{-1} v\|_\infty \leq \|v\|_\infty / (1 - \gamma)$$

this is basically just a formalization of an obvious fact using the geometric series application with γ previously. It's really just stating that, in policy evaluation, the maximum Q -function value is bounded by the maximum reward seen across all time steps multiplied by the effective horizon $\frac{1}{1-\gamma}$.

Proof.

Let $w = (I - \gamma P^\pi)^{-1} v$.

$$\begin{aligned}
\|v\|_\infty &= \|(I - \gamma P^\pi)w\|_\infty \\
&\geq \|w\|_\infty - \gamma \|P^\pi w\|_\infty && \text{(Triangle Inequality)} \\
&\geq \|w\|_\infty - \gamma \|w\|_\infty && (\|P^\pi\|_\infty \leq 1) \\
&= (1 - \gamma) \|w\|_\infty \implies \\
\|v\|_\infty / (1 - \gamma) &\geq \|(I - \gamma P^\pi)^{-1} v\|_\infty
\end{aligned}$$

Putting it Together

Now, let's apply both lemmas and finish off the error bound.

$$\begin{aligned}
Q^\pi - \hat{Q}^\pi &= \gamma(I - \gamma \hat{P}^\pi)(P - \hat{P})V^\pi && \text{(Simulation Lemma)} \\
\|Q^\pi - \hat{Q}^\pi\|_\infty &= \|\gamma(I - \gamma \hat{P}^\pi)(P - \hat{P})V^\pi\|_\infty \\
&\leq \frac{\gamma}{1 - \gamma} \|(P - \hat{P})V^\pi\|_\infty && \text{Lemma 2} \\
&\leq \frac{\gamma}{1 - \gamma} (\max_{s,a} \|p(\cdot \mid s, a) - \hat{p}(\cdot \mid s, a)\|_1) \|V^\pi\|_\infty
\end{aligned}$$

Note that $\|V^\pi\|_\infty \leq \frac{1}{1-\gamma} R_{\max}$ because of the same geometric series trick, and we can actually assume WLOG $R_{\max} = 1$ because, if they aren't, we can simply rescale the rewards appropriately (if we multiply the rewards by any constant, the optimal policy doesn't change). Thus, $\|V^\pi\|_\infty \leq \frac{1}{1-\gamma}$. Thus,

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \frac{\gamma}{(1-\gamma)^2} (\max_{s,a} \|p(\cdot | s, a) - \hat{p}(\cdot | s, a)\|_1)$$

Now recall the concentration inequality.

$$\|\hat{p}(s' | s, a) - p(s' | s, a)\| \leq c \sqrt{\frac{|S| \log \frac{1}{\delta}}{N}}$$

Which allows us to conclude that our error bound is

$$\|Q^\pi - \hat{Q}^\pi\|_\infty \leq \frac{\gamma}{(1-\gamma)^2} c_2 \sqrt{\frac{|S| \log \frac{1}{\delta}}{N}} = \epsilon$$

Note that c_2 exists because there is some constant factor involved when substituting for the lemma.

Interpretation

So, what can we take away from our error bound ϵ ?

First, the error grows quadratically in the horizon and each backup "accumulates" error.

$$\epsilon \propto \frac{1}{(1-\gamma)^2}$$

Second, more samples translates to lower error.

$$\epsilon \propto \sqrt{\frac{1}{N}}$$

Implications about Other Questions

What about the other questions we asked previously?

Let's first consider the second, i.e. bounding $\|Q^* - \hat{Q}^*\|$. Note the following lemma

$$|\sup_x f(x) - \sup_x g(x)| \leq \sup_x |f(x) - g(x)|$$

(This is essentially the same as the previous inequality with max instead of sup).

Then,

$$\|Q^* - \hat{Q}^*\|_\infty = \|\sup_\pi Q^\pi - \sup_\pi \hat{Q}^\pi\|_\infty \leq \sup_\pi \|Q^\pi - \hat{Q}^\pi\|_\infty \leq \epsilon$$

Now, let's consider the third, i.e. bounding $\|Q^* - Q^{\hat{\pi}^*}\|_\infty$.

$$\begin{aligned} \|Q^* - Q^{\hat{\pi}^*}\|_\infty &= \|Q^* - \hat{Q}^{\hat{\pi}^*} + \hat{Q}^{\hat{\pi}^*} - Q^{\hat{\pi}^*}\|_\infty \\ &\leq \|Q^* - \hat{Q}^{\hat{\pi}^*}\|_\infty + \|\hat{Q}^{\hat{\pi}^*} - Q^{\hat{\pi}^*}\|_\infty \end{aligned}$$

We note that $\hat{Q}^{\hat{\pi}^*}$ is essentially just $\hat{Q}^*$, and so the first term is bounded by ϵ according to our second derived bound, $\|Q^* - \hat{Q}^*\|_\infty \leq \epsilon$. Meanwhile, the second infinity norm is bounded by ϵ according to our first derived bound, $\|Q^\pi - \hat{Q}^\pi\| \leq \epsilon$, where $\pi = \hat{\pi}^*$ here. Thus,

$$\|Q^* - Q^{\hat{\pi}^*}\|_\infty \leq 2\epsilon$$

Analysis of Model-Free RL

We'll now analyze fitted Q -iteration.

First, let T be the **Bellman operator** such that

$$TQ = r + \gamma P \max_a Q$$

Then, exact fitted Q -iteration may be defined as $Q_{k+1} \leftarrow TQ_k$. For approximate fitted Q -iteration that uses samples, it changes slightly to be $\hat{Q}_{k+1} \leftarrow \arg \min_Q \|\hat{Q} - \hat{T}\hat{Q}_k\|$. $\hat{T}$ is the Bellman operator, except it accounts for sampling by considering the effective frequency of observations of each s' . In particular, we define $\hat{T}$ as

$$\hat{T}Q = \hat{r} + \gamma \hat{P} \max_a Q$$

where

$$\begin{aligned} \hat{r} &= \frac{1}{N(s, a)} \sum_i \delta((s_i, a_i) = (s, a)) r_i \\ \hat{p}(s' | s, a) &= \frac{N(s, a, s')}{N(s, a)} \end{aligned}$$

However, our update rule now involves a norm—what norm should we use? Unfortunately, the algorithm won't actually converge if $\|\cdot\|_2$ is used; therefore, we will assume $\|\cdot\|_\infty$. Notably, *no such learning algorithm* exists that may train with the infinity norm, and mean squared error is of course used in practice. Some interesting properties of the MSE-based learning algorithm are provable; however, they are much more difficult. Thus, we proceed with a theoretical learning algorithm that works with the infinity norm.

Now, we'd like to answer the following question: as $k \rightarrow \infty$, $\hat{Q}_k \rightarrow ?$ or $\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty \leq ?$. Where does our error come from though? For approximate fitted Q -iteration, from *sampling error*, $T \neq \hat{T}$, and *approximation error*, $\hat{Q}_{k+1} \neq \hat{T}\hat{Q}_k$.

Sampling Error

Let's first understand the effect of sampling error.

$$\begin{aligned} |\hat{T}Q(s, a) - TQ(s, a)| &= |\hat{r}(s, a) - r(s, a) + \gamma(\mathbb{E}_{\hat{p}(s'|s, a)}[\max_{a'} Q(s', a')] - \mathbb{E}_{p(s'|s, a)}[\max_{a'} Q(s', a')])| \\ &\leq |\hat{r}(s, a) - r(s, a)| + \gamma \left| \sum_{s'} (\hat{p}(s' | s, a) - p(s' | s, a)) \max_{a'} Q(s', a') \right| \end{aligned}$$

Where the inequality applied is triangle inequality. For the first term, the estimation of a continuous random variable, we can use Hoeffding's inequality.

$$|\hat{r}(s, a) - r(s, a)| \leq 2R_{\max} \sqrt{\frac{\log \frac{1}{\delta}}{2N}}$$

Meanwhile, for the second term, we can use the other concentration inequality

$$\begin{aligned}
\left| \sum_{s'} (\hat{p}(s' | s, a) - p(s' | s, a)) \max_{a'} Q(s', a') \right| &\leq \sum_{s'} |\hat{p}(s' | s, a) - p(s' | s, a)| \max_{s', a'} Q(s', a') \\
&= \|\hat{p}(\cdot | s, a) - p(\cdot | s, a)\|_1 \|Q\|_\infty \\
&\leq c \|Q\|_\infty \sqrt{\frac{\log \frac{1}{\delta}}{N}}
\end{aligned}$$

Thus, we can bound our error as

$$|\hat{T}Q(s, a) - TQ(s, a)| \leq 2R_{\max} \sqrt{\frac{\log \frac{1}{\delta}}{2N}} + c \|Q\|_\infty \sqrt{\frac{\log \frac{1}{\delta}}{N}}$$

Using the union bound, we can derive

$$|\hat{T}Q(s, a) - TQ(s, a)| \leq 2R_{\max} c_1 \sqrt{\frac{\log \frac{|S||A|}{\delta}}{2N}} + c_2 \|Q\|_\infty \sqrt{\frac{\log \frac{|S|}{\delta}}{N}}$$

Approximation Error

We'll first analyze the exact backup operator. We will assume $\|\hat{Q}_{k+1} - T\hat{Q}_k\|_\infty \leq \epsilon_k$ for some ϵ_k . Note that this is a pretty strong assumption!

$$\begin{aligned}
\|\hat{Q}_k - Q^*\|_\infty &= \|\hat{Q}_k - T\hat{Q}_{k-1} + T\hat{Q}_{k-1} - Q^*\|_\infty \\
&= \|(\hat{Q}_k - T\hat{Q}_{k-1}) + (T\hat{Q}_{k-1} - TQ^*)\|_\infty \\
&\leq \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty + \|T\hat{Q}_{k-1} - TQ^*\|_\infty \\
&\leq \epsilon_{k-1} + \|T\hat{Q}_{k-1} - TQ^*\|_\infty \\
&\leq \epsilon_{k-1} + \gamma \|\hat{Q}_{k-1} - Q^*\|_\infty
\end{aligned}$$

Recall the substitution of Q^* with TQ^* in step 2 is possible because Q^* is a fixed point of the operator T . The last step leverages the fact that T is a γ -contraction. We can recurse through this last inequality to produce

$$\begin{aligned}
\|\hat{Q}_k - Q^*\|_\infty &\leq \sum_{i=0}^{k-1} \gamma^i \epsilon_{k-i-1} + \gamma^k \|\hat{Q}_0 - Q^*\|_\infty \\
\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_\infty &\leq \sum_{i=0}^{k-1} \gamma^i \max_k \epsilon_k \\
&= \frac{1}{1-\gamma} \|\epsilon\|_\infty \\
&= \frac{1}{1-\gamma} \max_k \|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty
\end{aligned}$$

In other words, the approximation error scales with horizon.

Putting it Together

Let's now combine our bounds on the approximation error and sampling error to produce a bound on total error.

$$\begin{aligned}
\|\hat{Q}_k - T\hat{Q}_{k-1}\|_\infty &= \|\hat{Q}_k - \hat{T}\hat{Q}_{k-1} + \hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty \\
&\leq \underbrace{\|\hat{Q}_k - \hat{T}\hat{Q}_{k-1}\|_\infty}_{\text{approximation error}} + \underbrace{\|\hat{T}\hat{Q}_{k-1} - T\hat{Q}_{k-1}\|_\infty}_{\text{sampling error}}
\end{aligned}$$

Here are the bounds again, for convenience.

$$|\hat{T}Q(s, a) - TQ(s, a)| \leq 2R_{\max}c_1 \sqrt{\frac{\log \frac{|S||A|}{\delta}}{2N}} + c_2 \|Q\|_{\infty} \sqrt{\frac{\log \frac{|S|}{\delta}}{N}}$$

$$\lim_{k \rightarrow \infty} \|\hat{Q}_k - Q^*\|_{\infty} \leq \frac{1}{1-\gamma} \max_k \|\hat{Q}_k - T\hat{Q}_{k-1}\|_{\infty}$$

Notably, the $\|Q\|_{\infty}$ in the sampling error bound is in $O\left(R_{\max} \frac{1}{1-\gamma}\right)$. (Recall from previously that the entries of the Q -function matrix are approximately the size of the horizon, assuming $r \approx 1$). Meanwhile, there exists a $\frac{1}{1-\gamma}$ scalar for the approximation error bound. Thus, error *compounds* quadratically with the horizon, since the error for each step $\|\hat{Q}_k - T\hat{Q}_{k-1}\|$ scales with the horizon and there are H steps for a horizon H .